DEVOPS ENGINEERING MASTERY

KUBERNETES & CONTAINER ORCHESTRATION

Kubernetes Architecture, Cluster Management, Workload Orchestration ☸ Production Operations

VOLUME 07 • FIRST EDITION

```
nahid@technova:~$ kubectl get pods -n production
✓ technova-api-7d9f4c8b6f-x2k9p   1/1   Running
nahid@technova:~$ kubectl rollout status deployment/technova-api
✓ deployment "technova-api" successfully rolled out
nahid@technova:~$ helm upgrade technova ./charts/technova
```

Kubernetes • kind • Helm • Kustomize • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Kubernetes Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 07 (Companion Workbook). First Edition, 9 November 2026. Kubernetes with kind, Minikube, Helm and Kustomize and a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your repository without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab step on a production cluster. The instructor-style solution guide for the capstone is at the end — attempt the capstone before reading it.

ল্যাব ওয়ার্কবুক — পরিচিতি

DEVOPS ENGINEERING MASTERY — Volume 07: Kubernetes & Container Orchestration

এই ওয়ার্কবুকটি Volume 07 বইয়ের হাতে-কলমে সঙ্গী। বইয়ে আপনি Kubernetes-এর ধারণা, আর্কিটেকচার, Pod, Deployment, Service, Ingress, Storage, RBAC, নিরাপত্তা, নেটওয়ার্কিং, Helm, Kustomize, মনিটরিং ও প্রোডাকশন পরিচালনা শিখেছেন। এই ওয়ার্কবুকে সেই ধারণাগুলোকে আপনি নিজের কম্পিউটারে একটি লোকাল ক্লাস্টারে চালিয়ে, ভেঙে, ঠিক করে এবং আবার চালিয়ে সত্যিকারের দক্ষতায় রূপান্তর করবেন। পড়া আর করা এক নয় — Kubernetes শেখা যায় কেবল `kubectl` চালিয়ে, YAML লিখে, ভুল করে এবং `kubectl describe` পড়ে।

এই ওয়ার্কবুকের উদ্দেশ্য

এই ওয়ার্কবুকের উদ্দেশ্য চারটি। প্রথমত, একটি লোকাল Kubernetes ক্লাস্টার নিজে তৈরি করা এবং প্রতিটি মৌলিক বস্তু — Pod, Deployment, Service, ConfigMap, Secret, PersistentVolumeClaim, Ingress, Job, CronJob — হাতে তৈরি করা। দ্বিতীয়ত, আপডেট, রোলব্যাক, স্কেলিং, প্রোব, রিসোর্স সীমা ও নিরাপত্তা নিয়ন্ত্রণের মতো দৈনন্দিন অপারেশনাল কাজে আত্মবিশ্বাস অর্জন করা। তৃতীয়ত, সমস্যা সমাধানের পদ্ধতি শেখা — CrashLoopBackOff, ImagePullBackOff, Pending Pod-এর মতো সাধারণ ব্যর্থতা দেখে লগ, ইভেন্ট ও বর্ণনা পড়ে মূল কারণ খুঁজে বের করা। চতুর্থত, Helm, Kustomize, ব্যাকআপ, মনিটরিং ও সম্পূর্ণ MERN অ্যাপ্লিকেশন ডেপ্লয়মেন্টের মাধ্যমে একটি ছোট প্রোডাকশন-ধাঁচের পরিবেশ গড়ে তোলা, যা ক্যাপস্টোন প্রকল্পে কাজে লাগবে।

কীভাবে ব্যবহার করবেন

ল্যাবগুলো ক্রমানুসারে করুন। প্রতিটি ল্যাব আগের ল্যাবের ওপর ভিত্তি করে তৈরি। প্রতিটি ল্যাবে প্রথমে উদ্দেশ্য ও পূর্বশর্ত পড়ুন, তারপর আর্কিটেকচার ডায়াগ্রামটি বুঝুন, তারপর ধাপে ধাপে কমান্ড চালান। YAML ফাইল কপি করার আগে প্রতিটি ফিল্ডের অর্থ বুঝুন — অন্ধভাবে কপি করবেন না। প্রতিটি ধাপের পর যাচাই ধাপটি চালান; যাচাই না করে এগোবেন না। কোনো ধাপে আটকে গেলে প্রথমে Troubleshooting অংশটি পড়ুন, তারপর `kubectl describe` ও `kubectl logs` দেখুন। প্রতিটি ল্যাব শেষে Cleanup অংশটি চালিয়ে ক্লাস্টার পরিষ্কার রাখুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। শেষে স্ব-মূল্যায়নের প্রশ্নগুলোর উত্তর মুখে বা লিখে দিন — উত্তর দিতে না পারলে সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন।

প্রয়োজনীয় টুলস

টুল	কাজ	নোট

kind / Minikube / k3d	লোকাল Kubernetes ক্লাস্টার	যেকোনো একটি যথেষ্ট; এই বইয়ে kind-কে প্রধান ধরা হয়েছে
kubectl	ক্লাস্টার পরিচালনার কমান্ড-লাইন টুল	ক্লাস্টারের সংস্করণের কাছাকাছি সংস্করণ ব্যবহার করুন
Docker	কন্টেইনার রানটাইম ও ইমেজ তৈরি	kind ও Minikube-এর জন্য প্রয়োজন
Helm	প্যাকেজ ম্যানেজার ও চার্ট	ল্যাব ২১ ও প্রজেক্টে প্রয়োজন
kustomize	পরিবেশভিত্তিক YAML ওভারলে	kubectl-এর ভেতরে অন্তর্ভুক্ত; আলাদা বাইনারিও চলে
Git, curl, একটি টেক্সট এডিটর	ফাইল ব্যবস্থাপনা ও যাচাই	সব ল্যাবে প্রয়োজন

আপনার মেশিনে কমপক্ষে ৪টি CPU কোর, ৮ GB RAM ও ২০ GB খালি ডিস্ক থাকা ভালো। ল্যাবগুলো Linux, macOS ও Windows (WSL 2) — সবখানেই চলে। কাল্পনিক প্রসঙ্গ হিসেবে এই বইয়ে TechNova কোম্পানি, `technova.example` ডোমেইন ও `203.0.113.x` ঠিকানা ব্যবহার করা হয়েছে। এগুলো কাল্পনিক — বাস্তব কোনো সার্ভার বা ডোমেইন নয়।

> **WARNING:** কখনো প্রোডাকশন ক্লাস্টারে অনুশীলন করবেন না। কখনো API সার্ভার পাবলিক ইন্টারনেটে উন্মুক্ত করবেন না। এই বইয়ের সব উদাহরণে আসল পাসওয়ার্ড, টোকেন বা API কী ব্যবহার করবেন না — কেবল `CHANGE-ME-...` ধরনের প্লেসহোল্ডার ব্যবহার করুন। আসল সিক্রেট কখনো Git-এ কমিট করবেন না, স্ক্রিনশটে দেখাবেন না, বা চ্যাটে পাঠাবেন না। কোনো ক্লাউড ধাপে খরচ হতে পারে এমন ক্ষেত্রে তা স্পষ্টভাবে চিহ্নিত করা হয়েছে; সম্ভব হলে ক্লাউড ধাপ এড়িয়ে লোকাল ক্লাস্টারেই কাজ করুন।

> **CONCEPT:** প্রতিটি ল্যাবের নমুনা টার্মিনাল আউটপুট *illustrative* — অর্থাৎ ধারণা দেওয়ার জন্য লেখা নমুনা, আপনার মেশিনের আউটপুট হুবহু একই নাও হতে পারে। নাম, সময়, IP ঠিকানা ও Pod-এর নাম ভিন্ন হতে পারে; গুরুত্বপূর্ণ হলো অবস্থা (STATUS), প্রস্তুতি (READY) ও ইভেন্টের অর্থ বোঝা।

সাধারণ নিয়ম ও নিরাপত্তা

প্রতিটি ল্যাবের আগে `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন যে আপনি সঠিক — লোকাল — ক্লাস্টারে আছেন। ভুল কনটেক্সটে `kubectl delete` চালানো সবচেয়ে সাধারণ ও বিপজ্জনক ভুল। `kubectl apply` করার আগে `kubectl apply --dry-run=client -f ফাইল.yaml` দিয়ে YAML যাচাই করুন। Secret তৈরি করার সময় `--from-literal` ব্যবহার করুন, টার্মিনালের ইতিহাসে আসল মান না রেখে। ল্যাব শেষে সবসময় পরিষ্কার করুন — `kubectl delete namespace` বা `kind delete cluster` — যাতে পরের ল্যাব পরিষ্কার পরিবেশে শুরু হয়।

ল্যাব ১ — লোকাল Kubernetes ক্লাস্টার সেটআপ

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য আপনার নিজের কম্পিউটারে একটি সম্পূর্ণ Kubernetes ক্লাস্টার চালু করা। Kubernetes শেখার প্রথম ধাপ হলো একটি নিরাপদ, ফেলে দেওয়া যায় এমন পরিবেশ পাওয়া, যেখানে ভুল করলে কোনো ক্ষতি হয় না। kind (Kubernetes in Docker) Docker কন্টেইনারের ভেতরে ক্লাস্টার চালায়, Minikube একটি ভার্চুয়াল মেশিন বা কন্টেইনারে চালায়, আর k3d হালকা k3s ক্লাস্টার দেয়। তিনটির যেকোনো একটি দিয়ে এই বইয়ের সব ল্যাব করা যায়। এই ল্যাবে আপনি kind দিয়ে একটি ক্লাস্টার তৈরি করবেন, control-plane ও worker নোড দেখবেন, `kubectl` দিয়ে ক্লাস্টারের তথ্য যাচাই করবেন এবং প্রয়োজনে ক্লাস্টার মুছে আবার তৈরি করার অভ্যাস গড়বেন। ক্লাস্টার তৈরি করা শুধু একটি কমান্ড নয় — এটি Kubernetes আর্কিটেকচারের প্রথম হাতে-কলমে পরিচয়: API সার্ভার, etcd, scheduler, controller-manager ও kubelet কীভাবে একসঙ্গে কাজ করে, তা আপনি নোড ও Pod দেখে বুঝতে শুরু করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

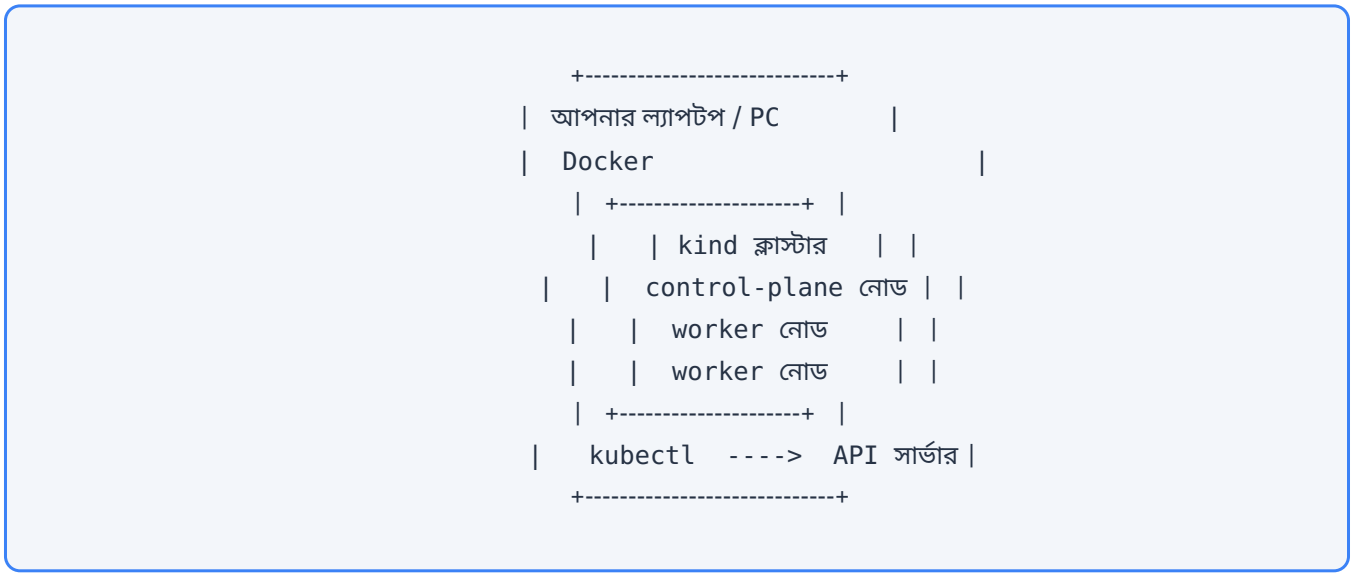
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, Docker। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



উপরে দেখা যাচ্ছে kind কীভাবে Docker কন্টেইনারের ভেতরে একাধিক নোড চালায়। আপনার `kubectl` কমান্ড API সার্ভারের সঙ্গে কথা বলে; API সার্ভারই ক্লাস্টারের একমাত্র প্রবেশদ্বার। নোডগুলো আসলে কন্টেইনার, তাই ক্লাস্টার মুছে ফেলা মানে কন্টেইনার মুছে ফেলা — সম্পূর্ণ নিরাপদ ও দ্রুত।

ধাপে ধাপে নির্দেশনা

1. **Docker ও kubectl যাচাই করুন:** প্রথমে `docker version` ও `kubectl version --client` চালিয়ে দেখুন দুটি টুলই ইনস্টল আছে কি না। Docker চলছে কি না তা `docker ps` দিয়ে যাচাই করুন। কোনো টুল না থাকলে অফিসিয়াল নির্দেশনা অনুযায়ী ইনস্টল করুন; সংস্করণ নিয়ে তাড়াহুড়ো করবেন না, কারণ kind ও kubectl-এর সংস্করণের মিল গুরুত্বপূর্ণ।
2. **kind ইনস্টল করুন:** অফিসিয়াল রিলিজ থেকে kind বাইনারি ডাউনলোড করে PATH-এ রাখুন। ইনস্টলের পর `kind version` চালিয়ে সংস্করণ দেখুন। Minikube বা k3d ব্যবহার করতে চাইলে সেই টুলটি ইনস্টল করুন — ধাপগুলোর ধারণা একই থাকবে, কেবল ক্লাস্টার তৈরির কমান্ড বদলাবে।
3. **ক্লাস্টার কনফিগারেশন ফাইল লিখুন:** নিচের YAML ফাইলে একটি control-plane নোড ও দুটি worker নোডের ক্লাস্টার বর্ণনা করা হয়েছে। ফাইলটি `kind-config.yaml` নামে সংরক্ষণ করুন। `kind: Cluster` ও `apiVersion: kind.x-k8s.io/v1alpha4` kind-এর নিজস্ব API — এটি Kubernetes-এর মূল API নয়, kind-কে ক্লাস্টার কেমন হবে তা জানায়।
4. **ক্লাস্টার তৈরি করুন:** `kind create cluster --name technova --config kind-config.yaml` চালান। তৈরি হতে এক থেকে তিন মিনিট লাগতে পারে, কারণ kind প্রথমে নোডের ইমেজ ডাউনলোড করে। ধৈর্য ধরুন; মাঝপথে বন্ধ করবেন না। তৈরি শেষে kind স্বয়ংক্রিয়ভাবে kubeconfig-এ কনটেক্সট যোগ করে।
5. **ক্লাস্টারের অবস্থা যাচাই করুন:** `kubectl cluster-info`, `kubectl get nodes` ও `kubectl get pods -A` চালান। সব নোড `Ready` অবস্থায় থাকা উচিত। `kube-system` নেমস্পেসের Pod-গুলো `Running` হলে বুঝবেন control-plane ঠিকমতো চলছে।
6. **একটি পরীক্ষামূলক Pod চালান:** `kubectl run hello --image=nginx:1.27 --port=80` চালিয়ে দেখুন Pod তৈরি হয় কি না, তারপর `kubectl get pods` ও `kubectl delete pod hello` দিয়ে মুছে ফেলুন। এটি প্রমাণ করে যে scheduler, kubelet ও ইমেজ টানা — পুরো চেইনটি কাজ করছে।

7. **কনটেক্সট বোঝা ও সংরক্ষণ:** `kubectl config get-contexts` চালিয়ে দেখুন `kind-technova` কনটেক্সটটি বর্তমান কি না। kubeconfig ফাইলটি কোথায় আছে তা `kubectl config view --minify` দিয়ে দেখুন। এই ফাইলে ক্লাস্টারের ঠিকানা ও শংসাপত্র থাকে — এটি কখনো শেয়ার করবেন না।
8. **ক্লাস্টার মুছে আবার তৈরি করার অভ্যাস:** `kind delete cluster --name technova` দিয়ে ক্লাস্টার মুছে আবার তৈরি করুন। ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সবচেয়ে বড় সুবিধা। কোনো ল্যাব এলোমেলো হয়ে গেলে নতুন ক্লাস্টার দিয়ে আবার শুরু করাই সঠিক পদ্ধতি।

কনফিগারেশন ফাইল

kind-config.yaml — তিন নোডের লোকাল ক্লাস্টার — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: technova
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kind create cluster --name technova --config kind-config.yaml
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -A
kubectl run hello --image=nginx:1.27 --port=80
kubectl get pods
kubectl delete pod hello
kind delete cluster --name technova
```

প্রত্যাশিত ফলাফল ও যাচাই

সব নোড `Ready` দেখাবে এবং `kubectl get pods -A`-এ `kube-system` নেমস্পেসের সব Pod `Running` অবস্থায় থাকবে। পরীক্ষামূলক nginx Pod তৈরি হয়ে `Running` হবে এবং মুছে ফেলার পর আর দেখা যাবে না। নমুনা আউটপুট (illustrative): `NAME STATUS ROLES AGE — technova-control-plane Ready control-plane 5m — technova-worker Ready <none> 5m — technova-worker2 Ready <none> 5m`। কোনো নোড `NotReady` থাকলে কয়েক মিনিট অপেক্ষা করে আবার দেখুন।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **ক্লাস্টার তৈরি আটকে গেছে:** প্রথমবার ইমেজ ডাউনলোডে সময় লাগে। ইন্টারনেট সংযোগ ও Docker চলছে কি না দেখুন; `docker ps`-এ kind নোড কন্টেইনার দেখা যাওয়া উচিত।
- **নোড NotReady দেখাচ্ছে:** `kubectl describe node <নোডের নাম>` ও `kubectl get pods -A` দেখুন। CNI Pod প্রস্তুত না হলে নোড NotReady থাকে; কয়েক মিনিট অপেক্ষা করুন।
- **kubectl: connection refused:** ভুল কনটেক্সট বা বন্ধ ক্লাস্টার — `kubectl config current-context` ও `kind get clusters` দিয়ে মিলিয়ে দেখুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab1` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. kind, Minikube ও k3d-এর মধ্যে পার্থক্য কী? কোন পরিস্থিতিতে কোনটি বেছে নেবেন?
2. control-plane নোডে সাধারণত ব্যবহারকারীর Pod চলে না কেন?
3. kubeconfig ফাইলে কী কী তথ্য থাকে এবং কেন এটি গোপন রাখতে হয়?
4. ক্লাস্টার মুছে আবার তৈরি করা কেন ল্যাব পরিবেশে নিরাপদ, কিন্তু প্রোডাকশনে নয়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২ — kubectl কনফিগারেশন ও কনটেক্সট

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য `kubectl`-এর কনফিগারেশন ব্যবস্থা — kubeconfig, ক্লাস্টার, ব্যবহারকারী (user/credential) ও কনটেক্সট — হাতে-কলমে বোঝা। Kubernetes-এ একটিই কমান্ড-লাইন টুল দিয়ে অনেক ক্লাস্টার পরিচালনা করা যায়, আর কনটেক্সটই বলে দেয় এই মুহূর্তে কোন ক্লাস্টারে, কোন নেমস্পেসে, কোন পরিচয়ে কাজ করছেন। ভুল কনটেক্সটে কাজ করা প্রোডাকশন দুর্ঘটনার সবচেয়ে সাধারণ কারণগুলোর একটি, তাই এই ল্যাবটি ছোট মনে হলেও এর গুরুত্ব অনেক বেশি। আপনি দুটি kind ক্লাস্টার তৈরি করে তাদের মধ্যে ঘুরবেন, নেমস্পেস বদলাবেন, kubeconfig ফাইলের গঠন দেখবেন এবং একটি নিরাপদ অভ্যাস গড়বেন: যেকোনো ধ্বংসাত্মক কমান্ডের আগে বর্তমান কনটেক্সট মুখে পড়ে নেওয়া।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

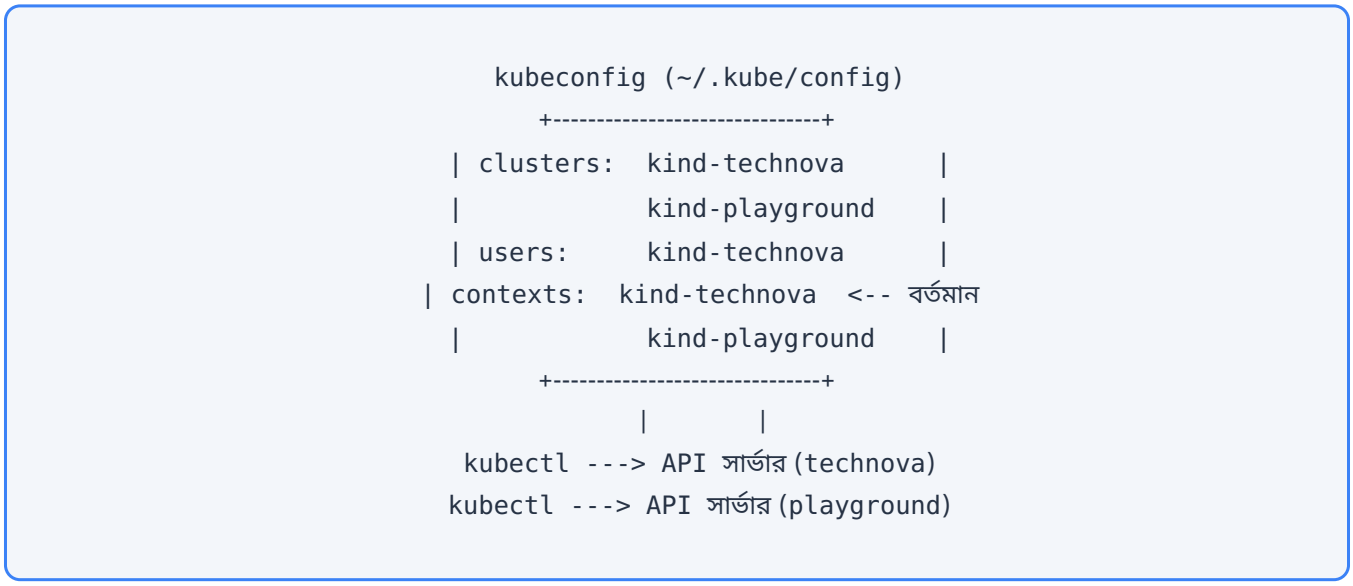
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



kubeconfig ফাইলে তিন ধরনের তথ্য থাকে — ক্লাস্টারের ঠিকানা ও শংসাপত্র, ব্যবহারকারীর পরিচয়পত্র, এবং কনটেক্সট যা একটি ক্লাস্টার ও একটি ব্যবহারকারীকে জোড়া দেয়। বর্তমান কনটেক্সটই নির্ধারণ করে আপনার পরের কমান্ড কোথায় যাবে।

ধাপে ধাপে নির্দেশনা

1. **বর্তমান কনফিগারেশন দেখুন:** `kubectl config view` ও `kubectl config get-contexts` চালান। কোন ক্লাস্টার, কোন ব্যবহারকারী ও কোন কনটেক্সট আছে তা লিখে রাখুন। `kubectl config current-context` দিয়ে বর্তমান কনটেক্সট নিশ্চিত করুন।
2. **দ্বিতীয় ক্লাস্টার তৈরি করুন:** `kind create cluster --name playground` চালান। এখন দুটি ক্লাস্টার থাকবে। `kind get clusters` ও `kubectl config get-contexts` দিয়ে দুটি কনটেক্সট দেখুন — kind স্বয়ংক্রিয়ভাবে নতুন কনটেক্সট যোগ করে ও সেটিকে বর্তমান করে দেয়।
3. **কনটেক্সট বদলান:** `kubectl config use-context kind-technova` চালিয়ে প্রথম ক্লাস্টারে ফিরুন, তারপর আবার `kind-playground`-এ যান। প্রতিবার `kubectl get nodes` চালিয়ে দেখুন নোডের নাম কীভাবে বদলায় — এটিই প্রমাণ যে আপনি আসলেই ভিন্ন ক্লাস্টারে আছেন।
4. **নেমস্পেস নির্ধারণ করুন:** `kubectl config set-context --current --namespace=technova-lab` চালিয়ে বর্তমান কনটেক্সটের ডিফল্ট নেমস্পেস বদলান। এরপর `kubectl get pods` কোন নেমস্পেসে চলে তা লক্ষ করুন। এই সেটিং কেবল সুবিধার জন্য; `-n` ফ্ল্যাগ সবসময় এটিকে ওভাররাইড করে।
5. **kubeconfig ফাইলের গঠন পড়ুন:** `~/.kube/config` ফাইলটি এডিটরে খুলে `clusters`, `contexts`, `users` ও `current-context` অংশগুলো পড়ুন। শংসাপত্রের মান (certificate-authority-data, client-key-data) base64-এ এনকোড করা থাকে — সেগুলো কখনো কপি করে শেয়ার করবেন না।
6. **অস্থায়ীভাবে অন্য কনটেক্সটে কমান্ড চালান:** `kubectl --context kind-playground get nodes` চালিয়ে দেখুন কনটেক্সট না বদলেও একটিমাত্র কমান্ড অন্য ক্লাস্টারে চালানো যায়। স্ক্রিপ্টে এই পদ্ধতি নিরাপদ, কারণ এতে বর্তমান কনটেক্সট বদলায় না।
7. **ভুল কনটেক্সট শনাক্ত করার অভ্যাস:** টার্মিনালে একটি ছোট পরীক্ষা করুন: ইচ্ছা করে ভুল কনটেক্সটে `kubectl get pods -n kube-system` চালিয়ে দেখুন কী ঘটে। তারপর সঠিক কনটেক্সটে ফিরুন। এই অভ্যাসটি প্রোডাকশনে ভুল কমান্ড চালানো থেকে বাঁচায়।

8. **playground ক্লাস্টার মুছে ফেলুন:** `kind delete cluster --name playground` দিয়ে দ্বিতীয় ক্লাস্টার মুছে ফেলুন এবং `kubectl config get-contexts` দিয়ে দেখুন তার কনটেক্সটও চলে গেছে কি না। প্রয়োজনে `kubectl config delete-context kind-playground` দিয়ে অবশিষ্ট কনটেক্সট মুছুন।

কনফিগারেশন ফাইল

কোনো YAML প্রয়োজন নেই — এই ল্যাবটি সম্পূর্ণ kubectl কনফিগারেশন কমান্ডের — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
# এই ল্যাবে কোনো YAML ফাইল নেই।
# সব কাজ kubectl config কমান্ড দিয়ে করা হয়।
# উদাহরণ: বর্তমান কনটেক্সট দেখা
# kubectl config current-context
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl config get-contexts
kubectl config current-context
kind create cluster --name playground
kubectl config use-context kind-technova
kubectl config use-context kind-playground
kubectl --context kind-playground get nodes
kubectl config set-context --current --namespace=technova-lab
kubectl config view --minify
kind delete cluster --name playground
```

প্রত্যাশিত ফলাফল ও যাচাই

দুটি কনটেক্সট তালিকায় দেখা যাবে, `use-context` চালালে বর্তমান কনটেক্সট বদলাবে এবং `get nodes`-এর আউটপুটে নোডের নাম বদলে যাবে। playground মুছে ফেলার পর তার কনটেক্সটও তালিকা থেকে যাবে না। নমুনা (illustrative): `CURRENT NAME CLUSTER — * kind-technova kind-technova — kind-playground kind-playground`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **কনটেক্সট বদলাচ্ছে না:** `kubectl config use-context`-এর নাম হুবহু মিলছে কি না দেখুন; `get-contexts` থেকে নাম কপি করুন।

- **kubeconfig ফাইল খুঁজে পাচ্ছেন না:** `echo $KUBECONFIG` দেখুন; সেট না থাকলে ডিফল্ট `~/.kube/config` ব্যবহৃত হয়।
- **দুটি ক্লাস্টারের নোড একই দেখাচ্ছে:** আপনি আসলে কনটেক্সট বদলাননি — `current-context` আবার যাচাই করুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab2` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. kubeconfig-এ cluster, user ও context — তিনটির ভূমিকা আলাদা করে বলুন।
2. `--context` ফ্ল্যাগ ব্যবহার করা কেন স্ক্রিপ্টে `use-context`-এর চেয়ে নিরাপদ?
3. বর্তমান কনটেক্সট না দেখে `kubectl delete` চালালে কী বিপদ হতে পারে?
4. একজন ডেভেলপার ও একজন ক্লাস্টার অ্যাডমিনের kubeconfig-এ পার্থক্য কী হওয়া উচিত?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৩ — Pod তৈরি ও জীবনচক্র

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এর সবচেয়ে ছোট ও মৌলিক একক — Pod — তৈরি করা, পর্যবেক্ষণ করা এবং তার জীবনচক্র বোঝা। Pod হলো এক বা একাধিক কন্টেইনারের একটি দল, যারা একই নেটওয়ার্ক নেমস্পেস ও স্টোরেজ ভাগ করে এবং সবসময় একই নোডে একসঙ্গে চলে। Deployment, Job বা DaemonSet — সবকিছুই শেষ পর্যন্ত Pod তৈরি করে, তাই Pod না বুঝলে Kubernetes-এর কিছুই বোঝা যায় না। এই ল্যাবে আপনি YAML দিয়ে একটি nginx Pod তৈরি করবেন, তার অবস্থা (Pending, Running, Succeeded, Failed) দেখবেন, লগ পড়বেন, Pod-এর ভেতরে ঢুকে কমান্ড চালাবেন এবং মুছে ফেলবেন। Pod অস্থায়ী — মুছে গেলে ফিরে আসে না, তার IP-ও বদলে যায় — এই সত্যটি হাতে-কলমে অনুভব করাই এই ল্যাবের মূল শিক্ষা।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

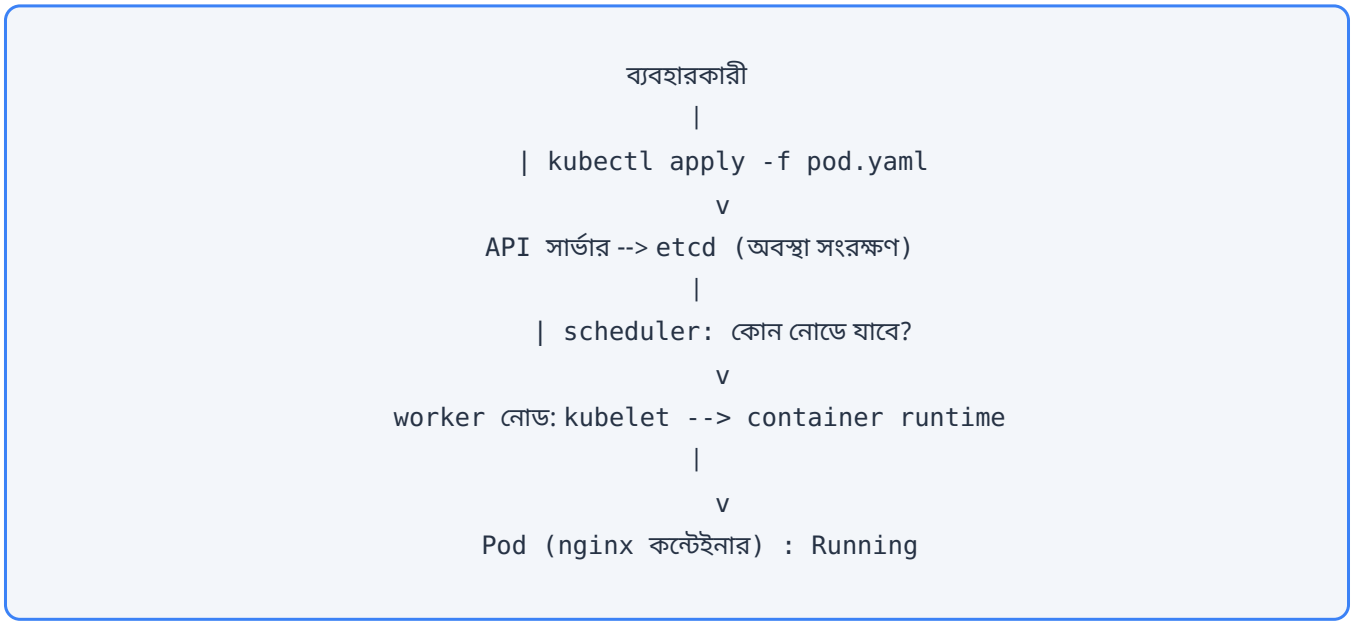
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Pod তৈরির পথটি দেখুন: আপনার YAML প্রথমে API সার্ভারে যায়, etcd-তে সংরক্ষিত হয়, scheduler একটি নোড বেছে নেয়, আর সেই নোডের kubelet কন্টেইনার চালু করে। কোনো ধাপে সমস্যা হলে Pod সেই ধাপের অবস্থায় আটকে থাকে — তাই অবস্থা দেখেই বোঝা যায় সমস্যাটা কোথায়।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab3` চালান। এই ল্যাবের সব বস্তু এই নেমস্পেসে রাখুন, যাতে শেষে এক কমান্ডে সব মুছে ফেলা যায়। `kubectl get namespaces` দিয়ে নিশ্চিত হোন।
2. **Pod-এর YAML লিখুন:** নিচের `pod.yaml` ফাইলটি লিখুন। `apiVersion: v1` ও `kind: Pod` লক্ষ করুন — Pod মূল (core) API-র অংশ। `containers`-এর ভেতরে ইমেজের নাম, পোর্ট ও নাম দেওয়া হয়েছে। `restartPolicy: Always` মানে কন্টেইনার বন্ধ হলে kubelet আবার চালু করবে।
3. **Pod তৈরি করুন:** `kubectl apply -f pod.yaml` চালান। তারপর `kubectl get pods -n lab3 -w` দিয়ে দেখুন Pod কীভাবে Pending থেকে ContainerCreating হয়ে Running-এ যায়। `-w` ফ্ল্যাগটি watch মোড — Ctrl+C দিয়ে থামান।
4. **Pod-এর বিস্তারিত দেখুন:** `kubectl describe pod web-pod -n lab3` চালান। Events অংশে দেখুন কোন ধাপে কী ঘটেছে — ইমেজ টানা, কন্টেইনার তৈরি, চালু হওয়া। এই আউটপুট পড়তে পারা Kubernetes-এর সবচেয়ে গুরুত্বপূর্ণ দক্ষতাগুলোর একটি।
5. **লগ পড়ুন:** `kubectl logs web-pod -n lab3` চালিয়ে nginx-এর লগ দেখুন। লগে কোনো অনুরোধ না থাকলে খালি লাগতে পারে — সেটিই স্বাভাবিক। লগ দেখার অভ্যাস করুন, কারণ অ্যাপ্লিকেশনের সমস্যা প্রথমে লগেই ধরা পড়ে।
6. **Pod-এর ভেতরে ঢুকুন:** `kubectl exec -it web-pod -n lab3 -- sh` চালিয়ে কন্টেইনারের শেল খুলুন। ভেতরে `hostname`, `cat /etc/os-release` ও `curl localhost` চালিয়ে দেখুন। `exit` দিয়ে বেরিয়ে আসুন। এই পদ্ধতি ডিবাগিংয়ে কাজে লাগে, কিন্তু প্রোডাকশনে সাবধানে ব্যবহার করতে হয়।
7. **Pod মুছে আবার তৈরি করুন:** `kubectl delete pod web-pod -n lab3` দিয়ে Pod মুছে `kubectl get pods -n lab3` দেখুন — Pod আর ফিরে আসে না। আবার `apply` করলে নতুন Pod তৈরি হয়, নতুন নাম ও নতুন IP নিয়ে। এটিই Pod-এর অস্থায়ী প্রকৃতি।

8. **একটি ব্যর্থ Pod পর্যবেক্ষণ করুন:** ইমেজের নাম বদলে ভুল ইমেজ দিয়ে একটি Pod তৈরি করে দেখুন কী হয় (ল্যাব ১৯-এ এর বিস্তারিত আছে)। আপাতত শুধু লক্ষ করুন: ভুল ইমেজের Pod Running হয় না — Kubernetes ভুল কিছু লুকায় না, অবস্থা দিয়েই জানিয়ে দেয়।

কনফিগারেশন ফাইল

pod.yaml — একটি nginx Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  namespace: lab3
  labels:
    app: web
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
  restartPolicy: Always
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab3
kubectl apply -f pod.yaml
kubectl get pods -n lab3 -w
kubectl describe pod web-pod -n lab3
kubectl logs web-pod -n lab3
kubectl exec -it web-pod -n lab3 -- sh
kubectl delete pod web-pod -n lab3
kubectl apply -f pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

Pod প্রথমে `Pending`, তারপর `Running` হবে; `READY` কলামে `1/1` দেখাবে। `describe`-এ Events অংশে `Pulled` ও `Created` ইভেন্ট থাকবে, `logs`-এ nginx-এর স্টার্টআপ লাইন দেখা যাবে। নমুনা (illustrative): `NAME READY STATUS RESTARTS AGE — web-pod 1/1 Running 0 2m`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Pod Pending-এ আটকে আছে:** `describe` দেখুন — ইমেজ টানতে দেরি হলে বা নোডে জায়গা না থাকলে Pending থাকে।
- **Pod CrashLoopBackOff দেখাচ্ছে:** কন্টেইনার চালু হয়েই বন্ধ হচ্ছে — `logs` দেখুন; বিস্তারিত ল্যাব ১৮-এ আছে।
- **exec কাজ করছে না:** Pod Running না হলে exec চলে না; আগে `get pods` দিয়ে অবস্থা নিশ্চিত করুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab3` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Pod ও কন্টেইনারের পার্থক্য কী? একটি Pod-এ একাধিক কন্টেইনার কখন রাখা হয়?
2. Pod মুছে গেলে তার IP কেন ফিরে পাওয়া যায় না? এই সমস্যার সমাধান কী?
3. `restartPolicy: Always` ও `Never`-এর পার্থক্য কোথায় কাজে লাগে?
4. `kubectl describe`-এর Events অংশ থেকে কী কী তথ্য পাওয়া যায়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৪ — Liveness, Readiness ও Startup Probe

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এর তিন ধরনের প্রোব — liveness, readiness ও startup — হাতে-কলমে শেখা। প্রোব হলো kubelet-এর নিয়মিত স্বাস্থ্য পরীক্ষা। liveness প্রোব ব্যর্থ হলে kubelet কন্টেইনারটি মেরে আবার চালু করে, readiness প্রোব ব্যর্থ হলে Pod-টিকে Service-এর ট্রাফিক থেকে সরিয়ে নেয়, আর startup প্রোব ধীরে চালু হওয়া অ্যাপ্লিকেশনকে সময় দেয়। প্রোব ছাড়া Kubernetes জানতে পারে না অ্যাপ্লিকেশনটি আসলে কাজ করছে কি না — কন্টেইনার চলছে মানেই অ্যাপ্লিকেশন সুস্থ নয়। এই ল্যাবে আপনি একটি ছোট অ্যাপ্লিকেশনে তিনটি প্রোব যোগ করবেন, ইচ্ছা করে অ্যাপ্লিকেশন অসুস্থ করবেন এবং দেখবেন Kubernetes কীভাবে প্রতিক্রিয়া জানায়।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

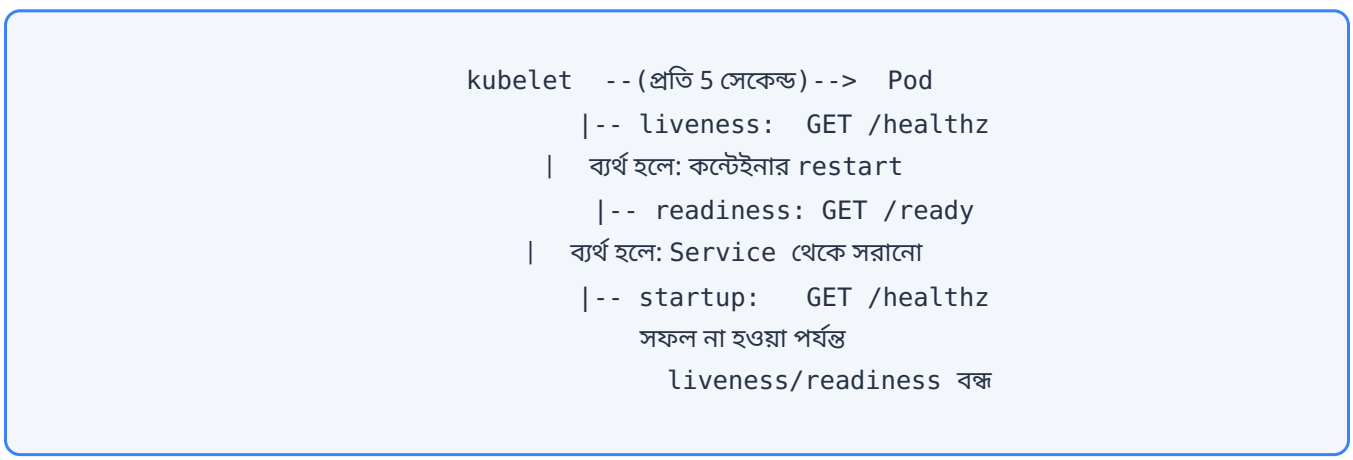
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



তিনটি প্রোবের ভূমিকা আলাদা। startup প্রোব চালু হওয়ার সময় রক্ষা করে, liveness প্রোব আটকে যাওয়া কন্টেইনারকে পুনরায় চালু করে, আর readiness প্রোব নিশ্চিত করে যে কেবল প্রস্তুত Pod-ই ট্রাফিক পায়।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab4` চালান। এই ল্যাবের সব বস্তু এই নেমস্পেসে রাখুন।
2. **প্রোবসহ Pod-এর YAML লিখুন:** নিচের YAML-এ একটি nginx Pod-এ তিনটি প্রোব যোগ করা হয়েছে। `httpGet` দিয়ে `/` পথে অনুরোধ পাঠানো হচ্ছে। `initialDelaySeconds`, `periodSeconds`, `failureThreshold` ও `successThreshold`-এর মানগুলো পড়ে বুঝুন — এগুলোই প্রোবের আচরণ নির্ধারণ করে।
3. **Pod তৈরি করে প্রোবের প্রভাব দেখুন:** `kubectl apply -f pod-probes.yaml` চালান, তারপর `kubectl describe pod probe-pod -n lab4` দেখুন। Events-এ প্রোব সফল হওয়ার কোনো আলাদা লাইন নাও থাকতে পারে — প্রোব নীরবে কাজ করে, ব্যর্থ হলেই ঘটনা ঘটে।
4. **readiness ব্যর্থতা পরীক্ষা করুন:** Pod-এর ভেতরে ঢুকে nginx বন্ধ করুন (`kubectl exec -it probe-pod -n lab4 -- nginx -s stop`) এবং `kubectl get pods -n lab4` দেখুন। READY কলাম `0/1` হয়ে যাওয়া উচিত, আর কিছুক্ষণ পর liveness প্রোব কন্টেইনারটি restart করবে।
5. **restart গণনা দেখুন:** `kubectl get pods -n lab4`-এ RESTARTS কলাম দেখুন — প্রতিটি liveness ব্যর্থতায় সংখ্যাটি বাড়ে। `describe`-এ `Liveness probe failed` ইভেন্ট দেখা যাবে। এটিই প্রোবের কাজের প্রমাণ।
6. **startup প্রোবের প্রয়োজন বুঝুন:** ধীরে চালু হওয়া অ্যাপ্লিকেশনের কথা ভাবুন: liveness প্রোব যদি শুরুতেই পরীক্ষা করে, তবে অ্যাপ্লিকেশন প্রস্তুত হওয়ার আগেই তাকে মেরে ফেলবে। startup প্রোব সেই সময় দেয়। YAML-এ `failureThreshold: 30` ও `periodSeconds: 10` মানে চালু হতে ৩০০ সেকেন্ড পর্যন্ত সময় দেওয়া হয়েছে।
7. **প্রোবের মান বদলে পরীক্ষা করুন:** `initialDelaySeconds` কমিয়ে বা `periodSeconds` বাড়িয়ে Pod আবার তৈরি করে দেখুন আচরণ কীভাবে বদলায়। খুব ঘন ঘন প্রোব অ্যাপ্লিকেশনের ওপর চাপ বাড়ায়, খুব কম ঘনত্বে ব্যর্থতা দেরিতে ধরা পড়ে — ভারসাম্য খুঁজে নেওয়াই দক্ষতা।
8. **Pod মুছে ফেলুন:** `kubectl delete pod probe-pod -n lab4` দিয়ে Pod মুছে ফেলুন। পরের ল্যাবে এই প্রোবগুলো Deployment-এ ব্যবহার করা হবে।

কনফিগারেশন ফাইল

pod-probes.yaml — তিনটি প্রোবসহ Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: probe-pod
  namespace: lab4
  labels:
    app: probe-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
```

```
      - containerPort: 80
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 3
      periodSeconds: 5
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 0
      periodSeconds: 10
      failureThreshold: 30
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab4
kubectl apply -f pod-probes.yaml
kubectl get pods -n lab4
kubectl describe pod probe-pod -n lab4
kubectl exec -it probe-pod -n lab4 -- nginx -s stop
kubectl get pods -n lab4
kubectl delete pod probe-pod -n lab4
```

প্রত্যাশিত ফলাফল ও যাচাই

Pod `Running` ও `1/1` READY হবে। nginx বন্ধ করার পর READY `0/1` হবে, তারপর RESTARTS বাড়বে এবং Pod আবার `1/1` হবে। `describe`-এ `Liveness probe failed` ও `Started container nginx` ইভেন্ট দেখা যাবে। নমুনা (illustrative): `probe-pod 1/1 Running 1 (30s ago) 3m`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **প্রোব সবসময় ব্যর্থ হচ্ছে:** পথ (`path`) বা পোর্ট ভুল কি না দেখুন; অ্যাপ্লিকেশন আসলে সেই পথে সাড়া দেয় কি না `curl` দিয়ে যাচাই করুন।
- **Pod বারবার restart হচ্ছে:** liveness প্রোব খুব কঠোর বা `initialDelaySeconds` খুব কম — অ্যাপ্লিকেশন চালু হতে যত সময় লাগে, তার চেয়ে বেশি সময় দিন।
- **readiness ব্যর্থ হলেও Pod Running দেখাচ্ছে:** এটিই স্বাভাবিক — readiness ব্যর্থতা Pod-কে Service থেকে সরায়, Pod-কে মারে না।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab4` (ল্যাব ২৪-এ `technova` , ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. liveness ও readiness প্রোবের পার্থক্য কী? একটির কাজ অন্যটি দিয়ে করা যায় কি?
2. startup প্রোব না থাকলে ধীর অ্যাপ্লিকেশনে কী সমস্যা হয়?
3. প্রোবের জন্য HTTP GET, TCP Socket ও Exec — কখন কোনটি ব্যবহার করবেন?
4. `failureThreshold` ও `periodSeconds` বদলালে ব্যর্থতা শনাক্তের সময় কীভাবে বদলায়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৫ — Deployment ও Replica

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Deployment — Kubernetes-এ অ্যাপ্লিকেশন চালানোর সবচেয়ে সাধারণ পদ্ধতি — তৈরি করা ও পরিচালনা করা। Pod একা থাকলে মুছে গেলে ফিরে আসে না, কিন্তু Deployment নিশ্চিত করে যে নির্দিষ্ট সংখ্যক Replica সবসময় চলছে। কোনো Pod মারা গেলে Deployment নতুন Pod তৈরি করে, কোনো নোড নষ্ট হলে অন্য নোডে Pod চালায়। Deployment-এর ভেতরে ReplicaSet থাকে, যা Replica গণনা বজায় রাখে। এই ল্যাবে আপনি তিনটি Replica-সহ একটি nginx Deployment তৈরি করবেন, একটি Pod মুছে দেখবেন Deployment কীভাবে তা ফিরিয়ে আনে, Replica সংখ্যা বাড়িয়ে-কমিয়ে স্কেল করবেন এবং Deployment-এর অবস্থা যাচাই করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Deployment সরাসরি Pod তৈরি করে না — এটি ReplicaSet তৈরি করে, আর ReplicaSet Pod তৈরি করে। এই স্তরবিন্যাসের কারণেই আপডেট ও রোলব্যাক সম্ভব হয়, যা পরের ল্যাবে দেখবেন।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab5` চালান।
2. **Deployment-এর YAML লিখুন:** নিচের YAML-এ `apiVersion: apps/v1` ও `kind: Deployment` ব্যবহার করা হয়েছে। `replicas: 3` মানে তিনটি Pod সবসময় চলবে। `selector.matchLabels` ও Pod টেমপ্লেটের `labels` মিলতে হবে — না মিললে Deployment তার Pod চিনতে পারবে না।
3. **Deployment তৈরি করুন:** `kubectl apply -f deployment.yaml` চালান। তারপর `kubectl get deployments -n lab5`, `kubectl get replicasets -n lab5` ও `kubectl get pods -n lab5` চালিয়ে তিনটি স্তরই দেখুন।
4. **Deployment-এর অবস্থা যাচাই করুন:** `kubectl rollout status deployment/web-deployment -n lab5` চালান। `successfully rolled out` বার্তা মানে সব Replica প্রস্তুত। `kubectl describe deployment web-deployment -n lab5`-এ Events দেখুন।
5. **একটি Pod মুছে পরীক্ষা করুন:** একটি Pod-এর নাম নিয়ে `kubectl delete pod <নাম> -n lab5` চালান, তারপর সঙ্গে সঙ্গে `kubectl get pods -n lab5 -w` দেখুন। নতুন Pod তৈরি হয়ে তিনটি Replica আবার পূর্ণ হবে — এটিই self-healing।
6. **স্কেল আপ ও স্কেল ডাউন করুন:** `kubectl scale deployment web-deployment --replicas=5 -n lab5` দিয়ে পাঁচটি করুন, তারপর `--replicas=2` দিয়ে দুটি করুন। প্রতিবার `get pods` দিয়ে গণনা মিলিয়ে দেখুন। স্কেলিং-এর জন্য YAML বদলানোর প্রয়োজন নেই।
7. **ReplicaSet দেখুন:** `kubectl get replicasets -n lab5` চালিয়ে দেখুন একটি ReplicaSet তিনটি (বা বর্তমান সংখ্যক) Pod পরিচালনা করছে। ReplicaSet-এর নামে Deployment-এর নাম ও একটি হ্যাশ থাকে।
8. **Deployment মুছে ফেলুন:** `kubectl delete deployment web-deployment -n lab5` চালালে Deployment, ReplicaSet ও সব Pod একসঙ্গে মুছে যাবে। `get pods` দিয়ে নিশ্চিত হোন।

কনফিগারেশন ফাইল

deployment.yaml — তিনটি Replica-সহ nginx Deployment — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: lab5
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
```

```
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab5
kubectl apply -f deployment.yaml
kubectl get deployments -n lab5
kubectl get replicasets -n lab5
kubectl get pods -n lab5
kubectl rollout status deployment/web-deployment -n lab5
kubectl scale deployment web-deployment --replicas=5 -n lab5
kubectl scale deployment web-deployment --replicas=2 -n lab5
kubectl delete deployment web-deployment -n lab5
```

প্রত্যাশিত ফলাফল ও যাচাই

তিনটি Pod `Running` ও `1/1` হবে; Deployment-এ `READY 3/3` দেখাবে। একটি Pod মুছলে কয়েক সেকেন্ডের মধ্যে নতুন Pod তৈরি হবে। স্কেলিংয়ের পর Pod সংখ্যা নির্দেশিত সংখ্যার সঙ্গে মিলবে। নমুনা (illustrative): `NAME READY UP-TO-DATE AVAILABLE AGE — web-deployment 3/3 3 3 4m`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Pod তৈরি হচ্ছে না:** selector ও টেমপ্লেটের labels মিলছে কি না দেখুন; না মিললে Deployment Pod খুঁজে পায় না।
- **Deployment-এ 0/3 READY:** `describe deployment` ও Pod-এর `describe` দেখুন — ইমেজ বা রিসোর্স সমস্যা হতে পারে।
- **স্কেল করলে Pod বাড়ছে না:** ক্লাস্টারে CPU বা মেমরি যথেষ্ট আছে কি না দেখুন; লোকাল ক্লাস্টারে বেশি Replica চাইলে নোডের সম্পদ সীমিত হতে পারে।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab5` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Deployment কেন সরাসরি Pod তৈরি করে না, ReplicaSet-এর ভূমিকা কী?
2. Pod মুছে গেলে Deployment কীভাবে বুঝতে পারে যে নতুন Pod দরকার?
3. `kubectl scale` ও YAML-এ replicas বদলানো — দুটির পার্থক্য কী?
4. কেবল Pod ব্যবহার করা কখন ঠিক, আর কখন Deployment আবশ্যক?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৬ — Rolling Update ও Rollback

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Deployment-এর rolling update ও rollback হাতে-কলমে শেখা। প্রোডাকশনে অ্যাপ্লিকেশন আপডেট করার সময় সার্ভিস বন্ধ করা যায় না — পুরোনো সংস্করণের Pod-গুলো একে একে নতুন সংস্করণ দিয়ে বদলাতে হয়, যাতে সবসময় কিছু Pod ট্রাফিক দিতে পারে। এটিই rolling update। নতুন সংস্করণে সমস্যা দেখা দিলে আগের সংস্করণে ফিরে যাওয়াই rollback। Kubernetes এই দুটি কাজই স্বয়ংক্রিয়ভাবে করে, তবে তার জন্য সঠিক কৌশল (strategy) ও ইতিহাস (revision history) বোঝা দরকার। এই ল্যাবে আপনি nginx-এর সংস্করণ বদলে আপডেট করবেন, আপডেটের অগ্রগতি দেখবেন, ইচ্ছা করে ভুল সংস্করণ দিয়ে ব্যর্থ আপডেট তৈরি করবেন এবং rollback করে আগের অবস্থায় ফিরবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

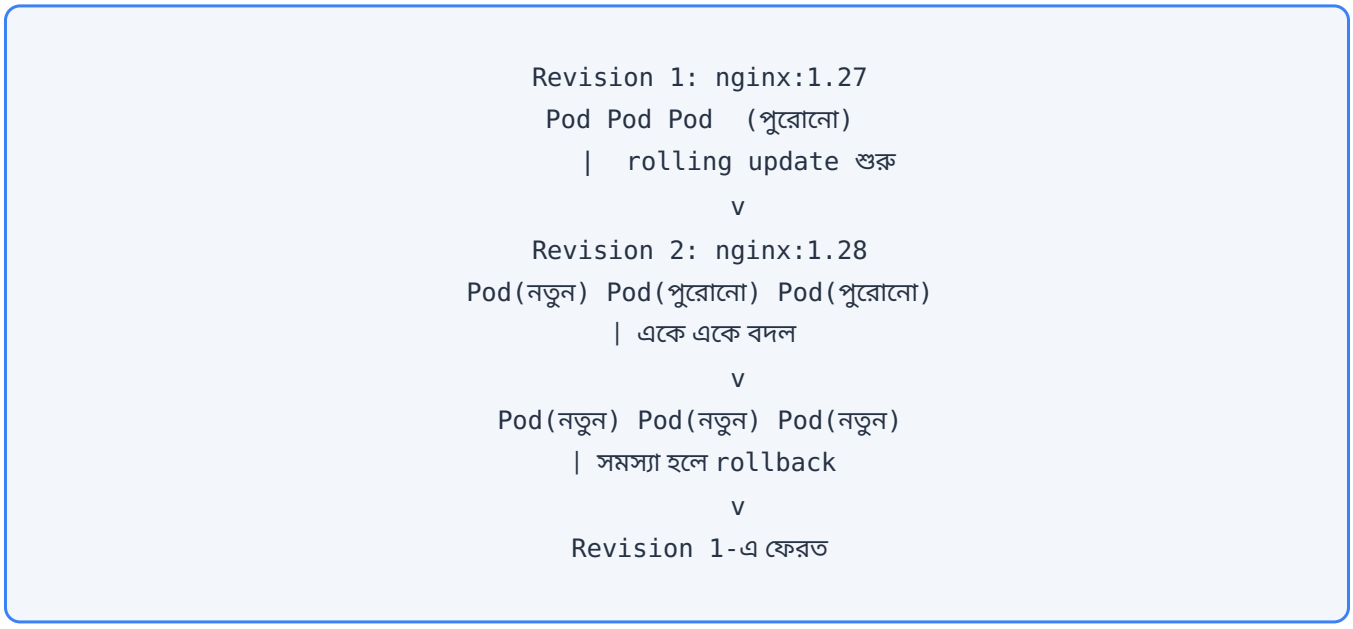
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



rolling update-এ পুরোনো ReplicaSet ধীরে ধীরে ছোট হয় আর নতুন ReplicaSet বড় হয়। দুটি ReplicaSet কিছুক্ষণ একসঙ্গে থাকে — তাই rollback মানে নতুন ReplicaSet বন্ধ করে পুরোনোটিকে আবার বড় করা, যা খুব দ্রুত হয়।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস ও Deployment তৈরি করুন:** `kubectl create namespace lab6` চালিয়ে ল্যাব ৫-এর Deployment YAML দিয়ে তিনটি Replica-সহ Deployment তৈরি করুন, তবে নেমস্পেস `lab6` করুন। `kubectl rollout status` দিয়ে প্রস্তুতি নিশ্চিত করুন।
2. **বর্তমান সংস্করণ দেখুন:** `kubectl get deployment web-deployment -n lab6 -o wide` ও `kubectl get pods -n lab6 -o wide` দিয়ে দেখুন কোন ইমেজ চলছে। `kubectl rollout history deployment/web-deployment -n lab6` দিয়ে revision ইতিহাস দেখুন — এখন কেবল revision 1 থাকবে।
3. **ইমেজ আপডেট করুন:** `kubectl set image deployment/web-deployment nginx=nginx:1.28 -n lab6` চালান। সঙ্গে সঙ্গে `kubectl get pods -n lab6 -w` দেখুন — নতুন Pod তৈরি হচ্ছে, পুরোনো Pod একে একে মুছে যাচ্ছে।
4. **আপডেটের অগ্রগতি দেখুন:** `kubectl rollout status deployment/web-deployment -n lab6` চালিয়ে `successfully rolled out` বার্তা পাওয়া পর্যন্ত অপেক্ষা করুন। `kubectl rollout history` আবার দেখুন — এখন revision 2 যোগ হয়েছে।
5. **ভুল সংস্করণ দিয়ে ব্যর্থ আপডেট করুন:** `kubectl set image deployment/web-deployment nginx=nginx:9.99 -n lab6` চালান — এমন কোনো সংস্করণ নেই। Pod-গুলো ImagePullBackOff অবস্থায় যাবে, আর rollout আটকে যাবে। `kubectl get pods -n lab6` ও `describe pod` দিয়ে দেখুন।
6. **আপডেট থামিয়ে rollback করুন:** `kubectl rollout undo deployment/web-deployment -n lab6` চালান। Kubernetes আগের revision-এ ফিরে যাবে। `kubectl rollout status` ও `get pods` দিয়ে দেখুন সব Pod আবার সঠিক ইমেজে চলছে।

7. **নির্দিষ্ট revision-এ ফেরত যান:** `kubectl rollout history deployment/web-deployment -n lab6` দেখে `kubectl rollout undo deployment/web-deployment --to-revision=1 -n lab6` চালিয়ে নির্দিষ্ট revision-এ ফেরার পদ্ধতি পরীক্ষা করুন।
8. **আপডেট কৌশল বুঝুন:** YAML-এ `strategy.type: RollingUpdate`, `maxUnavailable` ও `maxSurge` কী করে তা পড়ুন। `maxUnavailable: 1` মানে আপডেটের সময় সর্বোচ্চ একটি Pod অনুপলব্ধ থাকতে পারে; `maxSurge: 1` মানে একটি অতিরিক্ত Pod তৈরি করা যায়।

কনফিগারেশন ফাইল

deployment-update.yaml — rolling update কৌশলসহ Deployment — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
  namespace: lab6
  labels:
    app: web
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab6
kubectl apply -f deployment-update.yaml
kubectl rollout status deployment/web-deployment -n lab6
kubectl rollout history deployment/web-deployment -n lab6
kubectl set image deployment/web-deployment nginx=nginx:1.28 -n lab6
kubectl get pods -n lab6 -w
```

```
kubectl set image deployment/web-deployment nginx=nginx:9.99 -n lab6
kubectl rollout undo deployment/web-deployment -n lab6
kubectl rollout status deployment/web-deployment -n lab6
```

প্রত্যাশিত ফলাফল ও যাচাই

প্রথম আপডেট সফলভাবে শেষ হবে এবং সব Pod `nginx:1.28` চালাবে। ভুল সংস্করণের আপডেট আটকে যাবে, আর rollback-এর পর সব Pod আবার সঠিক ইমেজে `Running` হবে। ইতিহাসে একাধিক revision দেখা যাবে। নমুনা (illustrative): `deployment.apps/web-deployment successfully rolled out`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **rollout আটকে গেছে:** `rollout status` কিছুক্ষণ অপেক্ষা করে; নতুন Pod প্রস্তুত না হলে আপডেট এগোয় না — Pod-এর `describe` দেখুন।
- **rollback-এর পরও ভুল ইমেজ দেখাচ্ছে:** সঠিক revision-এ ফিরেছেন কি না `rollout history` দিয়ে যাচাই করুন; প্রয়োজনে `--to-revision` ব্যবহার করুন।
- **ইতিহাসে পুরোনো revision নেই:** `revisionHistoryLimit` কম থাকলে পুরোনো revision মুছে যায়; ডিফল্ট ১০টি রাখা হয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab6` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. rolling update-এর সময় সার্ভিস কেন বন্ধ হয় না? maxUnavailable ও maxSurge-এর ভূমিকা কী?
2. rollback আসলে কী করে — নতুন Deployment তৈরি করে, নাকি পুরোনো ReplicaSet-কে আবার বড় করে?
3. ভুল ইমেজের আপডেট কেন নিজে থেকে rollback হয় না? কী করলে স্বয়ংক্রিয় rollback সম্ভব?
4. Recreate কৌশল কখন ব্যবহার করা হয়, আর তার ঝুঁকি কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৭ — Namespace, Label ও Selector

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Namespace, Label ও Selector — Kubernetes-এ বস্তুগুলোকে সংগঠিত করার তিনটি মৌলিক উপায় — হাতে-কলমে শেখা। Namespace হলো ক্লাস্টারের ভেতরে একটি ভার্চুয়াল বিভাজন; একই ক্লাস্টারে dev, staging ও prod আলাদা Namespace-এ রাখা যায়, যাতে নামের সংঘাত না হয় ও অনুমতি আলাদা করা যায়। Label হলো বস্তুর গায়ে লাগানো কী-ভ্যালু ট্যাগ, আর Selector হলো সেই ট্যাগ দিয়ে বস্তু খুঁজে বের করার নিয়ম। Deployment তার Pod খুঁজে পায় Selector দিয়ে, Service তার Pod খুঁজে পায় Selector দিয়ে। এই তিনটি না বুঝলে Kubernetes-এর কোনো বড় ব্যবস্থাই বোঝা যায় না। এই ল্যাবে আপনি একাধিক Namespace তৈরি করবেন, Label দিয়ে Pod তৈরি করবেন এবং Selector দিয়ে সেগুলোকে খুঁজবেন, ফিল্টার করবেন ও মুছবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

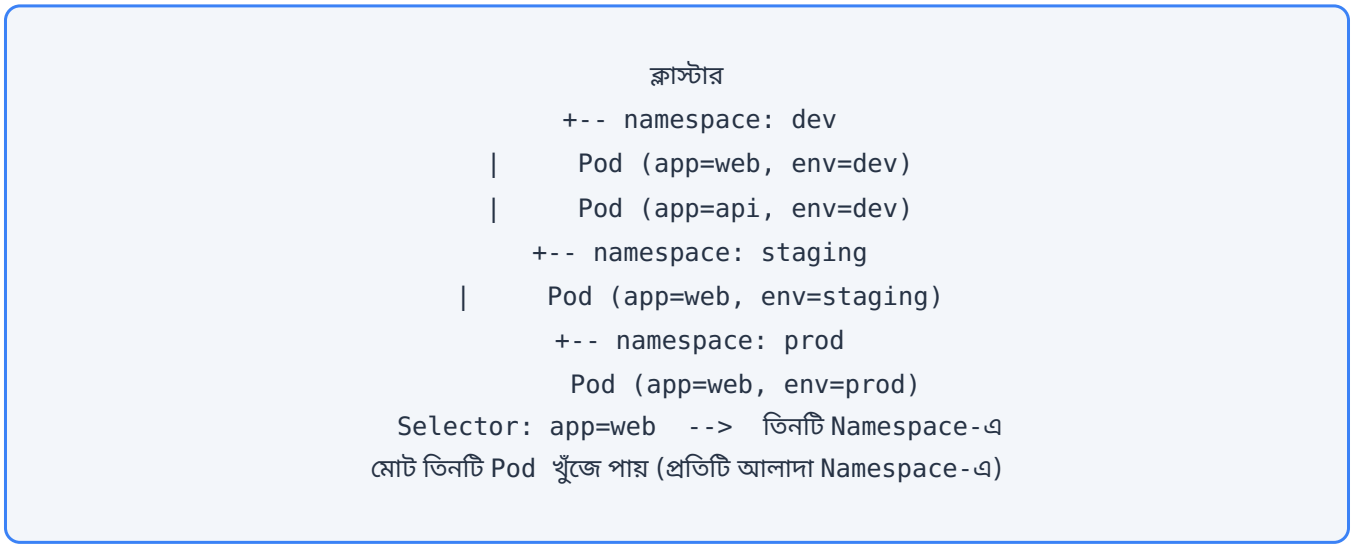
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Namespace নামের পরিসর আলাদা করে, Label বস্তুকে চিহ্নিত করে, Selector সেই চিহ্ন দিয়ে বস্তু বেছে নেয়। একই Label বিভিন্ন Namespace-এ থাকতে পারে, কিন্তু Selector কেবল নির্দিষ্ট Namespace-এর ভেতরে কাজ করে — যদি না `--all-namespaces` দেওয়া হয়।

ধাপে ধাপে নির্দেশনা

1. **তিনটি Namespace তৈরি করুন:** `kubectl create namespace dev`, `kubectl create namespace staging` ও `kubectl create namespace prod` চালান। `kubectl get namespaces` দিয়ে দেখুন — `default`, `kube-system` ছাড়াও আপনার তিনটি Namespace দেখা যাবে।
2. **Label-সহ Pod তৈরি করুন:** নিচের YAML-এ তিনটি Pod-এ আলাদা Label দেওয়া হয়েছে: `app`, `env` ও `tier`। `dev` Namespace-এ দুটি Pod তৈরি করুন — একটি web, একটি api। Label-এর মান ছোট, অর্থবহ ও সামঞ্জস্যপূর্ণ রাখুন।
3. **সব Pod Label-সহ দেখুন:** `kubectl get pods -n dev --show-labels` চালান। LABELS কলামে প্রতিটি Pod-এর সব Label দেখা যাবে। এটি Label ঠিকমতো লেগেছে কি না যাচাই করার দ্রুততম উপায়।
4. **Selector দিয়ে ফিল্টার করুন:** `kubectl get pods -n dev -l app=web` চালিয়ে দেখুন কেবল web Pod দেখা যায়। তারপর `-l app=web,env=dev` ও `-l tier=frontend` দিয়ে আরও ফিল্টার পরীক্ষা করুন। Selector-এ কমা মানে AND — সব শর্ত মিলতে হবে।
5. **সব Namespace-এ Selector চালান:** `kubectl get pods -A -l app=web` চালিয়ে দেখুন তিনটি Namespace-এর web Pod একসঙ্গে দেখা যায়। এটি বড় ক্লাস্টারে সমস্যা খুঁজতে খুব কাজে লাগে।
6. **Label যোগ ও বদল করুন:** `kubectl label pod web-pod -n dev version=v2` চালিয়ে চলমান Pod-এ নতুন Label যোগ করুন, তারপর `--overwrite` দিয়ে একটি Label-এর মান বদলান। Pod মুছে আবার তৈরি না করেও Label বদলানো যায় — তবে Deployment-এর Pod টেমপ্লেটের Label বদলালে নতুন Pod তৈরি হয়।
7. **Selector দিয়ে মুছে ফেলুন:** `kubectl delete pods -n dev -l app=api` চালিয়ে দেখুন কেবল api Pod মুছে যায়। Selector দিয়ে মোছা শক্তিশালী, তাই কমান্ড চালানোর আগে `get` দিয়ে কী মুছবে তা দেখে নিন।
8. **Namespace মুছে ফেলুন:** `kubectl delete namespace staging` চালিয়ে দেখুন সেই Namespace-এর সব বস্তু একসঙ্গে মুছে যায়। Namespace মোছা সবচেয়ে দ্রুত পরিষ্কার করার উপায়, তবে ভুল Namespace মুছলে সব হারিয়ে যাবে — সাবধানে করুন।

কনফিগারেশন ফাইল

labelled-pod.yaml — Label-সহ Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  namespace: dev
  labels:
    app: web
    env: dev
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: api-pod
  namespace: dev
  labels:
    app: api
    env: dev
    tier: backend
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace prod
kubectl apply -f labelled-pod.yaml
kubectl get pods -n dev --show-labels
kubectl get pods -n dev -l app=web
kubectl get pods -A -l app=web
kubectl label pod web-pod -n dev version=v2
kubectl delete pods -n dev -l app=api
kubectl delete namespace staging
```

প্রত্যাশিত ফলাফল ও যাচাই

তিনটি Namespace তৈরি হবে, দুটি Pod `Running` হবে এবং Selector দিয়ে ফিল্টার করলে কেবল মিলে যাওয়া Pod দেখা যাবে। `--show-labels`-এ সব Label দেখা যাবে, আর Namespace মুছলে তার ভেতরের সব বস্তু চলে যাবে। নমুনা (illustrative): `kubectl get pods -n dev -l app=web`-এ কেবল `web-pod` দেখাবে।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Selector-এ কিছু পাওয়া যাচ্ছে না:** Label-এর বানান ও মান হুবহু মিলছে কি না দেখুন; Label case-sensitive।
- **Namespace মুছতে অনেক সময় লাগছে:** Namespace `Terminating` অবস্থায় আটকে থাকতে পারে — ভেতরে কোনো বস্তু মুছতে বাধা দিচ্ছে কি না `kubectl get all -n <নাম>` দিয়ে দেখুন।
- **Label বদলাতে পারছেন না:** Deployment-এর Selector-এর সঙ্গে মিলে যাওয়া Label বদলাতে গেলে সমস্যা হয় — টেমপ্লেটের Label বদলানো নিরাপদ নয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab7` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Namespace ব্যবহারের সুবিধা কী? কখন আলাদা ক্লাস্টার, আর কখন আলাদা Namespace ব্যবহার করবেন?
2. Label ও Annotation-এর পার্থক্য কী? কোনটি Selector-এ ব্যবহার করা যায়?
3. Deployment-এর Selector কেন তৈরি হওয়ার পর বদলানো যায় না?

4. একটি Pod-এ একাধিক Label থাকলে Selector কীভাবে কাজ করে?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৮ — ConfigMap ও Secret

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য অ্যাপ্লিকেশনের কনফিগারেশন ও গোপন তথ্য Pod থেকে আলাদা করে পরিচালনা করা শেখা। ConfigMap-এ রাখা হয় সাধারণ কনফিগারেশন — যেমন অ্যাপ্লিকেশনের নাম, পরিবেশের নাম, ফিচার ফ্ল্যাগ বা পুরো কনফিগারেশন ফাইল। Secret-এ রাখা হয় সংবেদনশীল তথ্য — পাসওয়ার্ড, টোকেন, API কী, ডাটাবেসের শংসাপত্র। দুটির ব্যবহার প্রায় একই, কিন্তু Secret base64-এ এনকোড করে সংরক্ষিত হয় এবং আরও সাবধানে পরিচালনা করতে হয়। ইমেজের ভেতরে কনফিগারেশন ঢুকিয়ে দিলে প্রতিটি পরিবেশের জন্য আলাদা ইমেজ বানাতে হয়; ConfigMap ও Secret ব্যবহার করলে একই ইমেজ dev, staging ও prod — সবখানে চলে, কেবল কনফিগারেশন বদলায়। এই ল্যাবে আপনি দুটি তৈরি করবেন, পরিবেশ চলক (environment variable) ও ভলিউম হিসেবে Pod-এ যুক্ত করবেন এবং Secret-এর নিরাপত্তা নিয়মগুলো অনুশীলন করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

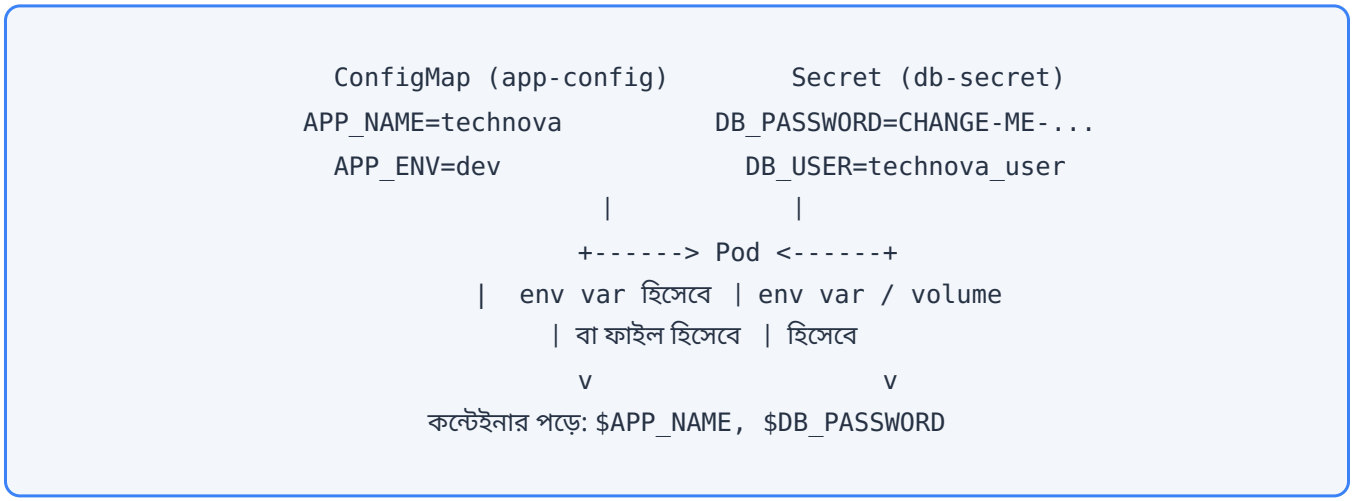
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



ConfigMap ও Secret Pod-এর সঙ্গে দুইভাবে যুক্ত হয় — পরিবেশ চলক হিসেবে, অথবা ফাইল হিসেবে মাউন্ট করে। পরিবেশ চলক সহজ, ফাইল হিসেবে মাউন্ট করলে কনফিগারেশন ফাইল সরাসরি ব্যবহার করা যায়। Secret কখনো ইমেজে বা Git-এ রাখা যাবে না।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab8` চালান।
2. **ConfigMap তৈরি করুন:** নিচের YAML দিয়ে একটি ConfigMap তৈরি করুন, অথবা `kubectl create configmap app-config --from-literal=APP_NAME=technova --from-literal=APP_ENV=dev -n lab8` চালান। দুটি পদ্ধতিই শিখুন — YAML Git-এ রাখা যায়, কমান্ড দ্রুত পরীক্ষার জন্য ভালো।
3. **Secret তৈরি করুন:** `kubectl create secret generic db-secret --from-literal=DB_USER=technova_user --from-literal=DB_PASSWORD=CHANGE-ME-db-password -n lab8` চালান। লক্ষ করুন: কমান্ডে আসল পাসওয়ার্ডের বদলে প্লেসহোল্ডার ব্যবহার করা হয়েছে। YAML-এ Secret লিখলে মান base64-এ এনকোড করতে হয় — `echo -n 'CHANGE-ME-db-password' | base64` দিয়ে মান বের করা যায়।
4. **ConfigMap ও Secret দেখুন:** `kubectl get configmaps -n lab8` ও `kubectl get secrets -n lab8` চালান। তারপর `kubectl describe configmap app-config -n lab8` দিয়ে মান দেখুন, কিন্তু Secret-এর ক্ষেত্রে `kubectl get secret db-secret -n lab8 -o yaml` দিয়ে দেখুন — মান base64-এ থাকবে, সরাসরি পড়া যাবে না।
5. **Pod-এ পরিবেশ চলক হিসেবে যুক্ত করুন:** নিচের Pod YAML-এ ConfigMap ও Secret থেকে পরিবেশ চলক নেওয়া হয়েছে। Pod তৈরি করে `kubectl exec -it config-pod -n lab8 -- env | grep -E 'APP_|DB_'` চালিয়ে দেখুন মানগুলো Pod-এর ভেতরে পৌঁছেছে।
6. **ফাইল হিসেবে মাউন্ট করুন:** একই ConfigMap-কে ভলিউম হিসেবে মাউন্ট করে Pod-এর ভেতরে `/etc/config` পথে ফাইল হিসেবে পড়ুন। `kubectl exec` দিয়ে `ls /etc/config` ও `cat /etc/config/APP_NAME` চালান। কনফিগারেশন ফাইলের ক্ষেত্রে এই পদ্ধতিই বেশি ব্যবহৃত হয়।

7. **ConfigMap বদলে Pod আপডেট হয় কি না পরীক্ষা করুন:** ConfigMap-এর মান বদলে দেখুন চলমান Pod-এর পরিবেশ চলক বদলায় কি না। উত্তর: পরিবেশ চলক বদলায় না — Pod আবার তৈরি করতে হয়। ভলিউম হিসেবে মাউন্ট করা মান কিছুক্ষণ পর আপডেট হয়, তবে অ্যাপ্লিকেশনকে ফাইলটি আবার পড়তে হয়।
8. **Secret মুছে ফেলুন ও সতর্কতা মনে রাখুন:** ল্যাব শেষে Secret মুছে ফেলুন। মনে রাখবেন: Secret base64 এনকোডিং কোনো এনক্রিপশন নয় — যে কেউ ডিকোড করতে পারে। প্রোডাকশনে Secret এনক্রিপশন ও অ্যাক্সেস নিয়ন্ত্রণ (RBAC) আবশ্যক।

কনফিগারেশন ফাইল

config-pod.yaml — ConfigMap ও Secret ব্যবহার করা Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: lab8
data:
  APP_NAME: technova
  APP_ENV: dev
  FEATURE_NEW_UI: "true"
---
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: lab8
type: Opaque
stringData:
  DB_USER: technova_user
  DB_PASSWORD: CHANGE-ME-db-password
---
apiVersion: v1
kind: Pod
metadata:
  name: config-pod
  namespace: lab8
  labels:
    app: config-demo
spec:
  containers:
    - name: app
      image: nginx:1.27
      ports:
        - containerPort: 80
      env:
        - name: APP_NAME
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_NAME
        - name: DB_PASSWORD
          valueFrom:
```

```
            secretKeyRef:
              name: db-secret
              key: DB_PASSWORD
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
          readOnly: true
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab8
kubectl apply -f config-pod.yaml
kubectl get configmaps -n lab8
kubectl get secrets -n lab8
kubectl describe configmap app-config -n lab8
kubectl exec -it config-pod -n lab8 -- env | grep -E 'APP_|DB_'
kubectl exec -it config-pod -n lab8 -- ls /etc/config
kubectl delete -f config-pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

ConfigMap ও Secret তৈরি হবে, Pod `Running` হবে এবং `env` কমান্ডে `APP_NAME=technova` ও `DB_PASSWORD=CHANGE-ME-db-password` দেখা যাবে। `/etc/config`-এ ConfigMap-এর প্রতিটি কী একটি ফাইল হিসেবে থাকবে। নমুনা (illustrative): `APP_NAME=technova – DB_PASSWORD=CHANGE-ME-db-password`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Pod তৈরি হচ্ছে না — ConfigMap পাওয়া যাচ্ছে না:** ConfigMap ও Pod একই Namespace-এ আছে কি না দেখুন; নামের বানান মিলিয়ে নিন।
- **Secret-এর মান দেখতে পাচ্ছেন না:** এটিই স্বাভাবিক — Secret base64-এ থাকে; `echo '<মান>' | base64 --decode` দিয়ে ডিকোড করা যায়, তবে তা কেবল পরীক্ষার জন্য।
- **ConfigMap বদলালেও Pod-এর মান বদলাচ্ছে না:** পরিবেশ চলক Pod তৈরির সময় নির্ধারিত হয়; মান বদলাতে Pod আবার তৈরি করতে হয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab8` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. ConfigMap ও Secret-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
2. Secret-এর base64 এনকোডিং কেন এনক্রিপশন নয়? প্রোডাকশনে Secret কীভাবে সুরক্ষিত রাখা হয়?
3. পরিবেশ চলক ও ভলিউম মাউন্ট — কখন কোন পদ্ধতি বেছে নেবেন?
4. ConfigMap-এর মান বদলালে চলমান Pod-এ কী ঘটে, আর কেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ৯ — Resource Request ও Limit

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Pod-এর জন্য CPU ও মেমরির request ও limit নির্ধারণ করা শেখা। request হলো scheduler-কে দেওয়া প্রতিশ্রুতি — এই Pod-কে অন্তত এতটুকু সম্পদ দিতে হবে; scheduler কেবল সেই নোডেই Pod বসায় যেখানে এতটুকু খালি সম্পদ আছে। limit হলো সর্বোচ্চ সীমা — Pod এর বেশি CPU বা মেমরি ব্যবহার করতে পারবে না; মেমরির সীমা ছাড়ালে Pod-কে OOMKilled করে মেরে ফেলা হয়। request ও limit না দিলে একটি Pod পুরো নোডের সম্পদ খেয়ে ফেলতে পারে, আর অন্য Pod-গুলো ক্ষতিগ্রস্ত হয়। প্রোডাকশনে প্রতিটি ওয়ার্কলোডের জন্য request ও limit নির্ধারণ করা মৌলিক শৃঙ্খলা। এই ল্যাবে আপনি সীমাসহ Pod তৈরি করবেন, সীমা ছাড়িয়ে OOMKilled দেখবেন এবং নোডের সম্পদ কীভাবে বরাদ্দ হয় তা পরীক্ষা করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

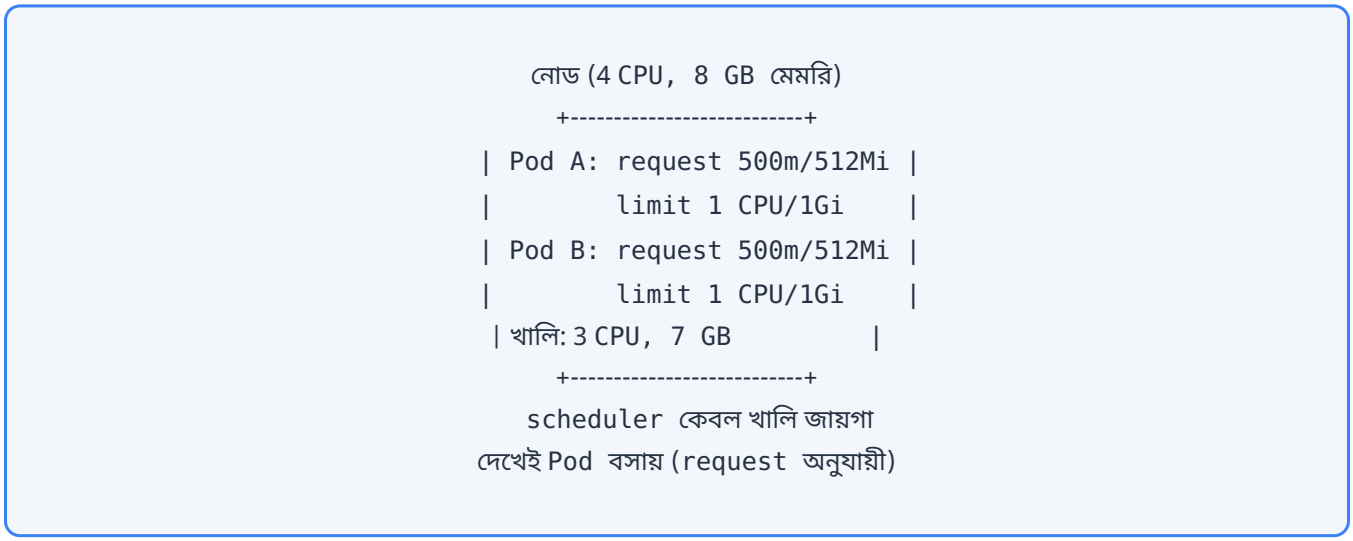
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



request দিয়ে scheduler সিদ্ধান্ত নেয় Pod কোথায় বসবে, limit দিয়ে kubelet Pod-এর ব্যবহার নিয়ন্ত্রণ করে। CPU-র একক `m` (millicpu) — 1000m মানে ১টি CPU কোর; মেমরির একক Mi (mebibyte) বা Gi।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab9` চালান।
2. **নোডের সম্পদ দেখুন:** `kubectl describe node` চালিয়ে Allocatable অংশে দেখুন নোডে মোট কত CPU ও মেমরি ব্যবহারের জন্য খালি আছে। লোকাল ক্লাস্টারে এই সংখ্যা ছোট হতে পারে — তাই request-এর মান ছোট রাখুন।
3. **সীমাসহ Pod তৈরি করুন:** নিচের YAML-এ একটি Pod-এ request ও limit দেওয়া হয়েছে। `kubectl apply -f pod-resources.yaml` চালান। `requests` ও `limits`-এর মানগুলো পড়ে বুঝুন — request limit-এর চেয়ে ছোট বা সমান হতে হয়।
4. **Pod-এর সম্পদ বরাদ্দ যাচাই করুন:** `kubectl describe pod resource-pod -n lab9`-এ Limits ও Requests অংশ দেখুন। তারপর `kubectl top pod -n lab9` চালিয়ে বর্তমান ব্যবহার দেখুন (metrics-server ইনস্টল থাকলে; না থাকলে এই ধাপটি ল্যাব ২০-এ আবার করবেন)।
5. **মেমরি সীমা ছাড়িয়ে OOMKilled দেখুন:** নিচের দ্বিতীয় YAML-এ খুব ছোট মেমরি limit (64Mi) দিয়ে একটি Pod তৈরি করুন, যেটি তার চেয়ে বেশি মেমরি চায়। Pod-টি `OOMKilled` অবস্থায় যাবে। `kubectl get pods -n lab9` ও `describe pod`-এর Last State অংশে `Reason: OOMKilled` দেখুন।
6. **বড় request দিয়ে Pending Pod দেখুন:** অসম্ভব বড় request (যেমন 100 CPU) দিয়ে একটি Pod তৈরি করে দেখুন সেটি Pending-এ আটকে থাকে — কোনো নোডেই এত সম্পদ নেই। `kubectl describe pod`-এ `FailedScheduling` ইভেন্টে কারণ লেখা থাকবে।
7. **LimitRange ও ResourceQuota-র ধারণা নিন:** একটি Namespace-এ সব Pod-এর জন্য ডিফল্ট সীমা দিতে LimitRange, আর পুরো Namespace-এর মোট সীমা বাঁধতে ResourceQuota ব্যবহার করা হয় — এগুলোর বিস্তারিত প্রজেক্ট ২-এ আছে। আপাতত ধারণাটি মনে রাখুন।
8. **Pod-গুলো মুছে ফেলুন:** `kubectl delete -f pod-resources.yaml` ও পরীক্ষামূলক Pod-গুলো মুছে ফেলুন, যাতে নোডের সম্পদ খালি হয়।

কনফিগারেশন ফাইল

pod-resources.yaml — request ও limit-সহ Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
  namespace: lab9
  labels:
    app: resource-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
      resources:
        requests:
          memory: "128Mi"
          cpu: "250m"
        limits:
          memory: "256Mi"
          cpu: "500m"
---
apiVersion: v1
kind: Pod
metadata:
  name: oom-pod
  namespace: lab9
  labels:
    app: oom-demo
spec:
  containers:
    - name: busybox
      image: busybox:1.36
      command: ["sh", "-c", "while true; do a=$(cat /dev/zero | head -c 1000000); sleep 1;
done"]
      resources:
        requests:
          memory: "32Mi"
          cpu: "100m"
        limits:
          memory: "64Mi"
          cpu: "200m"
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab9
kubectl describe node
```

```
kubectl apply -f pod-resources.yaml
kubectl get pods -n lab9
kubectl describe pod resource-pod -n lab9
kubectl describe pod oom-pod -n lab9
kubectl top pod -n lab9
kubectl delete -f pod-resources.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

প্রথম Pod `Running` হবে এবং `describe`-এ request ও limit দেখা যাবে। OOM Pod `OOMKilled` হয়ে restart হতে থাকবে, আর অতিরিক্ত বড় request-এর Pod `Pending` থাকবে। নমুনা (illustrative): `Last State: Terminated – Reason: OOMKilled – Exit Code: 137`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Pod Pending-এ আটকে আছে:** request নোডের খালি সম্পদের চেয়ে বড় কি না দেখুন; `describe pod`-এর FailedScheduling বার্তা পড়ুন।
- **Pod বারবার OOMKilled হচ্ছে:** limit খুব কম, অথবা অ্যাপ্লিকেশন সত্যিই বেশি মেমরি চায় — limit বাড়ান অথবা অ্যাপ্লিকেশনের মেমরি ব্যবহার কমান।
- **kubectl top কাজ করছে না:** metrics-server ইনস্টল নেই — ল্যাব ২০-এ ইনস্টল করার পদ্ধতি আছে।

TROUBLE: কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab9` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

WARNING: Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. request ও limit-এর পার্থক্য কী? scheduler কোনটি দেখে সিদ্ধান্ত নেয়?
2. মেমরির limit ছাড়ালে Pod মারা যায়, কিন্তু CPU-র limit ছাড়ালে কী হয়?
3. request না দিলে কী সমস্যা হয়? একটি Pod কীভাবে পুরো নোড দখল করতে পারে?
4. OOMKilled-এর Exit Code 137-এর অর্থ কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১০ — Job ও CronJob

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য একবার চলা ও নিয়মিত সময়সূচিতে চলা কাজ — Job ও CronJob — শেখা। Deployment দীর্ঘস্থায়ী সার্ভিসের জন্য, কিন্তু অনেক কাজ একবার চালিয়ে শেষ হয়: ডাটা মাইগ্রেশন, রিপোর্ট তৈরি, ব্যাকআপ, ইমেইল পাঠানো। Job এমন একটি Pod তৈরি করে যা কাজ শেষ করে সফলভাবে বন্ধ হয়; কাজ ব্যর্থ হলে Job আবার চেষ্টা করে। CronJob হলো সময়সূচিভুক্ত Job — প্রতি রাতে ২টায় ব্যাকআপ, প্রতি ঘণ্টায় পরিষ্কারকরণ — cron অভিব্যক্তি দিয়ে সময় নির্ধারণ করা হয়। এই ল্যাবে আপনি একটি Job তৈরি করে তার সফল সমাপ্তি দেখবেন, একটি ব্যর্থ Job-এর পুনরায় চেষ্টা দেখবেন এবং একটি CronJob তৈরি করে তার সময়সূচি পরীক্ষা করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

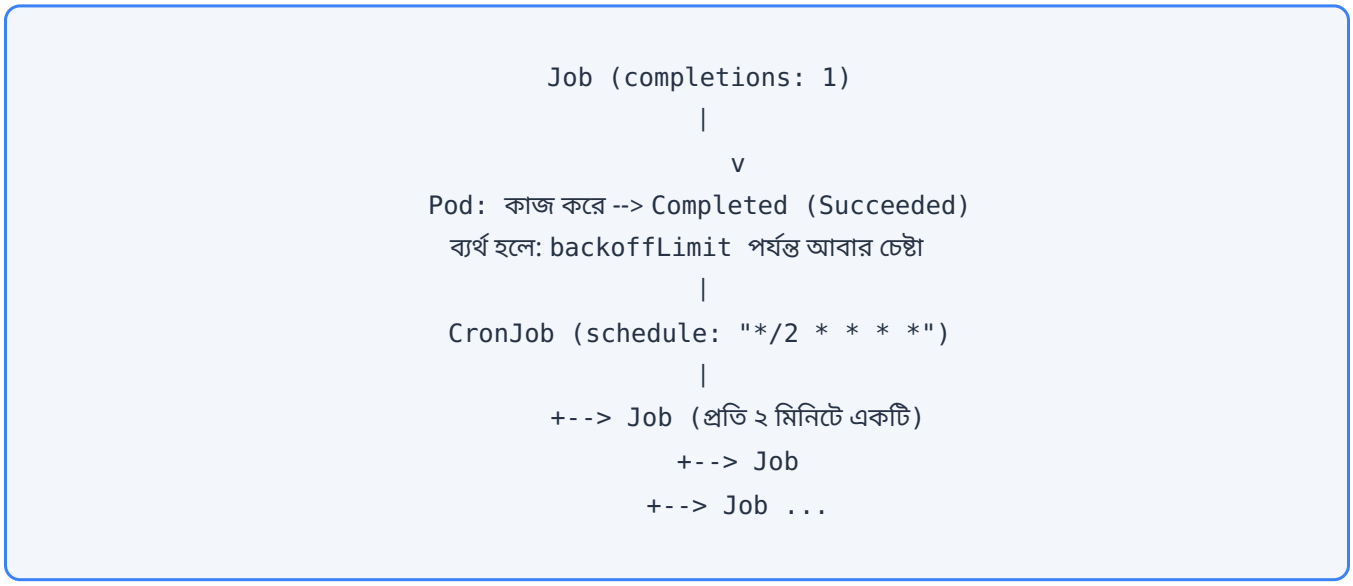
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Job Pod-এর `restartPolicy` হতে হয় `Never` বা `OnFailure` — `Always` নয়, কারণ Job-এর Pod কাজ শেষ করে থেমে যাওয়া স্বাভাবিক। CronJob-এর সময়সূচি পাঁচটি ক্ষেত্রের cron অভিব্যক্তি: মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab10` চালান।
2. **একটি সাধারণ Job তৈরি করুন:** নিচের YAML-এ একটি Job আছে যা একটি সংখ্যা গণনা করে শেষ হয়। `kubectl apply -f job.yaml` চালান। `completions: 1` মানে একটি সফল Pod হলেই Job শেষ; `backoffLimit: 3` মানে ব্যর্থ হলে তিনবার আবার চেষ্টা করবে।
3. **Job-এর অগ্রগতি দেখুন:** `kubectl get jobs -n lab10 -w` চালিয়ে দেখুন Job কীভাবে Active থেকে Complete হয়। তারপর `kubectl get pods -n lab10` দেখুন — Pod-টি `Completed` অবস্থায় থাকবে, মুছে যাবে না।
4. **Job-এর লগ পড়ুন:** `kubectl logs job/pi-job -n lab10` চালিয়ে Job-এর আউটপুট দেখুন। Job-এর লগই তার ফলাফল — Pod মুছে গেলে লগও হারিয়ে যায়, তাই গুরুত্বপূর্ণ ফলাফল কোথাও সংরক্ষণ করতে হয়।
5. **ব্যর্থ Job পরীক্ষা করুন:** ইচ্ছা করে ভুল কমান্ড দিয়ে একটি Job তৈরি করুন (যেমন `exit 1`)। `kubectl get jobs` ও `get pods` দেখুন — Pod বারবার তৈরি হয়ে ব্যর্থ হচ্ছে, যতক্ষণ না backoffLimit শেষ হয়। তারপর Job `Failed` হবে।
6. **CronJob তৈরি করুন:** নিচের CronJob YAML-এ প্রতি দুই মিনিটে একটি Job চলে। `kubectl apply -f cronjob.yaml` চালান, তারপর `kubectl get cronjobs -n lab10` ও `kubectl get jobs -n lab10 -w` দিয়ে দেখুন সময়মতো Job তৈরি হচ্ছে কি না।
7. **CronJob-এর সময়সূচি বুঝুন:** `schedule: '*/2 * * * *'` মানে প্রতি দুই মিনিটে। `concurrencyPolicy: Forbid` মানে আগের Job চলতে থাকলে নতুন Job শুরু হবে না। `successfulJobsHistoryLimit` ও `failedJobsHistoryLimit` দিয়ে কতগুলো পুরোনো Job রাখা হবে তা নির্ধারণ করা হয়।
8. **CronJob বন্ধ ও মুছে ফেলুন:** পরীক্ষা শেষে `kubectl patch cronjob hello-cron -n lab10 -p '{"spec":{"suspend":true}}'` দিয়ে CronJob থামান, তারপর `kubectl delete -f cronjob.yaml` ও `kubectl`

`delete -f job.yaml` দিয়ে সব মুছে ফেলুন। CronJob চালু রেখে ভুলে গেলে সেটি অনির্দিষ্টকাল Job তৈরি করতে থাকবে।

কনফিগারেশন ফাইল

job.yaml ও cronjob.yaml — Job ও CronJob — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-job
  namespace: lab10
spec:
  completions: 1
  parallelism: 1
  backoffLimit: 3
  template:
    metadata:
      labels:
        app: pi-job
    spec:
      restartPolicy: Never
      containers:
        - name: pi
          image: perl:5.38
          command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(200)"]
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cron
  namespace: lab10
spec:
  schedule: "*/2 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        metadata:
          labels:
            app: hello-cron
        spec:
          restartPolicy: OnFailure
          containers:
            - name: hello
              image: busybox:1.36
              command: ["sh", "-c", "date; echo Hello from TechNova CronJob"]
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab10
kubectl apply -f job.yaml
kubectl get jobs -n lab10 -w
kubectl get pods -n lab10
kubectl logs job/pi-job -n lab10
kubectl apply -f cronjob.yaml
kubectl get cronjobs -n lab10
kubectl get jobs -n lab10 -w
kubectl delete -f cronjob.yaml
kubectl delete -f job.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

Job `Complete` হবে এবং Pod `Completed` দেখাবে; লগে গণনার ফল দেখা যাবে। CronJob প্রতি দুই মিনিটে একটি নতুন Job তৈরি করবে, প্রতিটি Job-এর Pod `Completed` হবে। নমুনা (illustrative): `NAME COMPLETIONS DURATION AGE — pi-job 1/1 12s 2m`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Job Complete হচ্ছে না:** Pod-এর লগ দেখুন — কমান্ডটি কি শেষ হচ্ছে, নাকি অনন্তকাল চলছে? Job-এর কমান্ড অবশ্যই শেষ হতে হবে।
- **CronJob-এ Job তৈরি হচ্ছে না:** সময়সূচি (schedule) ঠিক আছে কি না দেখুন; `suspend: true` থাকলে Job তৈরি হয় না।
- **ব্যর্থ Job বারবার Pod তৈরি করছে:** এটিই backoffLimit-এর কাজ — সীমা শেষ হলে থামবে; `describe job` দেখুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab10` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` /

`prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Job ও Deployment-এর পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
2. Job-এর Pod-এর restartPolicy কেন Always হতে পারে না?
3. CronJob-এর concurrencyPolicy-র তিনটি মান (Allow, Forbid, Replace) কখন কোনটি ব্যবহার করবেন?
4. Job-এর লগ মুছে গেলে তার ফলাফল কীভাবে সংরক্ষণ করা যায়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১১ — Service

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes Service — Pod-গুলোর সামনে একটি স্থিতিশীল ঠিকানা দেওয়ার উপায় — শেখা। Pod-এর IP অস্থায়ী: Pod মুছে গেলে নতুন Pod নতুন IP পায়। Service একটি স্থিতিশীল ClusterIP ও DNS নাম দেয় এবং Selector দিয়ে মিলে যাওয়া সব Pod-এর মধ্যে ট্রাফিক ভাগ করে দেয়। ClusterIP কেবল ক্লাস্টারের ভেতরে কাজ করে, NodePort ক্লাস্টারের বাইরে থেকে নোডের পোর্ট দিয়ে পৌঁছানো যায়, আর LoadBalancer ক্লাউডে একটি বাহ্যিক লোড ব্যালান্সার চায়। এই ল্যাবে আপনি তিনটি Replica-সহ Deployment তৈরি করে তার সামনে একটি ClusterIP Service বসাবেন, Service-এর ভেতর থেকে curl দিয়ে পরীক্ষা করবেন, একটি Pod মুছে দেখবেন Service কীভাবে ট্রাফিক নতুন Pod-এ পাঠায় এবং NodePort Service-ও পরীক্ষা করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

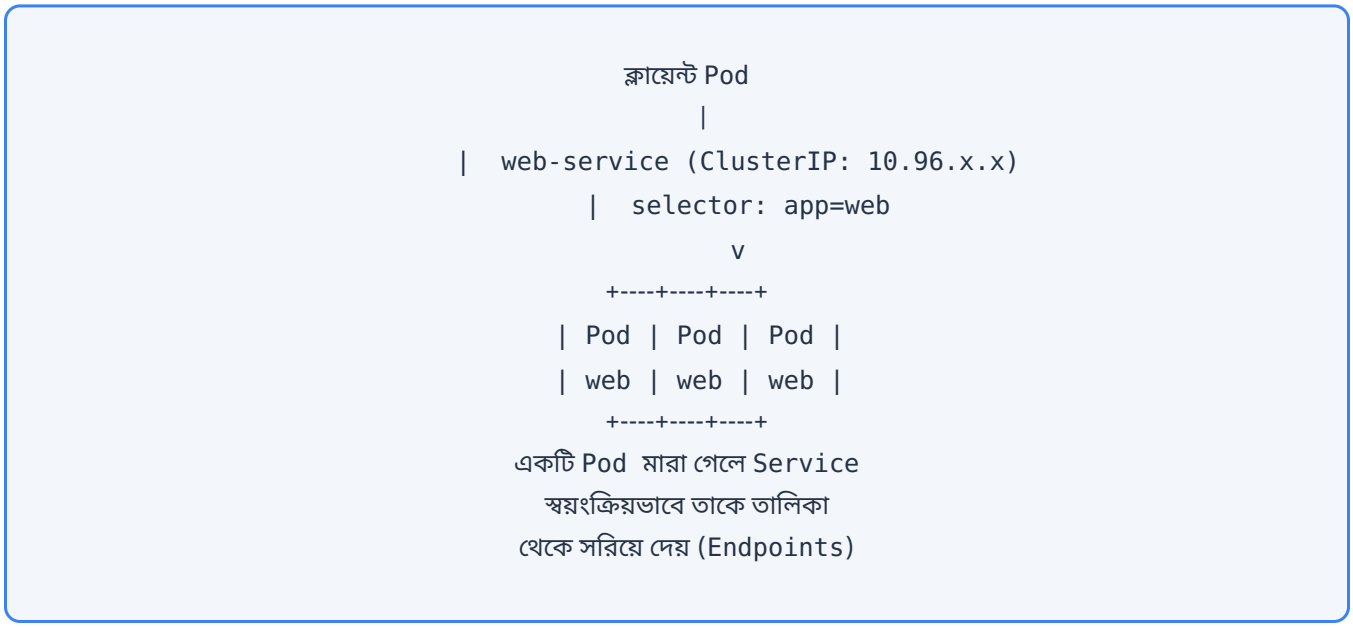
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Service-এর পেছনে Endpoints নামে একটি বস্তু থাকে, যাতে মিলে যাওয়া Pod-গুলোর IP থাকে। Pod প্রস্তুত না হলে বা মারা গেলে Endpoints থেকে সরে যায় — তাই ক্লায়েন্টকে Pod-এর IP জানতে হয় না, কেবল Service-এর নাম জানলেই চলে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস ও Deployment তৈরি করুন:** `kubectl create namespace lab11` চালিয়ে তিনটি Replica-সহ nginx Deployment তৈরি করুন (ল্যাব ৫-এর YAML, নেমস্পেস lab11)। `kubectl get pods -n lab11` দিয়ে তিনটি Pod চলছে কি না নিশ্চিত হোন।
2. **ClusterIP Service তৈরি করুন:** নিচের YAML-এ `type: ClusterIP` Service আছে। `selector: app: web` মানে এই Service কেবল web Label-এর Pod-গুলোতে ট্রাফিক পাঠাবে। `port: 80` হলো Service-এর পোর্ট, `targetPort: 80` হলো Pod-এর পোর্ট।
3. **Service ও Endpoints দেখুন:** `kubectl get services -n lab11` ও `kubectl get endpoints web-service -n lab11` চালান। Endpoints-এ তিনটি Pod-এর IP দেখা যাওয়া উচিত। `kubectl describe service web-service -n lab11`-এ Selector ও Endpoints মিলিয়ে দেখুন।
4. **ক্লাস্টারের ভেতর থেকে পরীক্ষা করুন:** একটি অস্থায়ী Pod থেকে curl চালান: `kubectl run curl-test --image=busybox:1.36 -n lab11 --rm -it -- sh` — ভেতরে `wget -qO- http://web-service` চালিয়ে nginx-এর স্বাগত পাতা দেখুন। Service-এর নাম দিয়েই পৌঁছানো যাচ্ছে — এটিই DNS-এর কাজ, বিস্তারিত ল্যাব ১২-এ আছে।
5. **একটি Pod মুছে Service পরীক্ষা করুন:** একটি Pod মুছে `kubectl get endpoints web-service -n lab11` আবার দেখুন — নতুন Pod-এর IP Endpoints-এ যোগ হয়েছে, পুরোনোটি সরে গেছে। Service-এর ClusterIP কিন্তু একই আছে — এটিই স্থিতিশীলতা।
6. **NodePort Service পরীক্ষা করুন:** নিচের দ্বিতীয় YAML-এ `type: NodePort` Service আছে। তৈরি করে `kubectl get services -n lab11` দেখুন — 30000–32767 পরিসরের একটি পোর্ট বরাদ্দ হয়েছে। kind-এ `kubectl port-forward` দিয়ে বা নোডের পোর্টে পৌঁছে পরীক্ষা করুন।

7. **port-forward দিয়ে পরীক্ষা করুন:** `kubectl port-forward service/web-service 8080:80 -n lab11` চালিয়ে ব্রাউজারে `http://localhost:8080` খুলুন। port-forward কেবল পরীক্ষার জন্য — প্রোডাকশনে Service বা Ingress ব্যবহার করা হয়।
8. **Service মুছে ফেলুন:** `kubectl delete -f service.yaml` দিয়ে Service মুছে ফেলুন। Service মুছলে Pod-গুলো মুছে যায় না — কেবল তাদের সামনের ঠিকানাটি চলে যায়।

কনফিগারেশন ফাইল

service.yaml — ClusterIP ও NodePort Service — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
  namespace: lab11
  labels:
    app: web
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      protocol: TCP
---
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
  namespace: lab11
  labels:
    app: web
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
      protocol: TCP
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab11
kubectl apply -f service.yaml
```

```
kubectl get services -n lab11
kubectl get endpoints web-service -n lab11
kubectl describe service web-service -n lab11
kubectl run curl-test --image=busybox:1.36 -n lab11 --rm -it -- sh
kubectl port-forward service/web-service 8080:80 -n lab11
kubectl delete -f service.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

Service-এর ClusterIP বরাদ্দ হবে, Endpoints-এ তিনটি Pod-এর IP থাকবে এবং অস্থায়ী Pod থেকে `http://web-service`-এ curl করলে nginx-এর পাতা ফিরবে। একটি Pod মুছলে Endpoints আপডেট হবে, কিন্তু ClusterIP বদলাবে না। নমুনা (illustrative): `NAME TYPE CLUSTER-IP PORT(S) — web-service ClusterIP 10.96.123.45 80/TCP`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Endpoints খালি দেখাচ্ছে:** Selector ও Pod-এর Label মিলছে কি না দেখুন; Pod Running ও Ready কি না দেখুন — প্রস্তুত নয় এমন Pod Endpoints-এ আসে না।
- **curl-এ সংযোগ হচ্ছে না:** Service-এর নাম সঠিক Namespace-এ ব্যবহার করছেন কি না দেখুন; ভিন্ন Namespace থেকে পুরো নাম লাগে।
- **NodePort-এ পৌঁছাতে পারছেন না:** kind-এ NodePort সরাসরি হোস্ট থেকে নাও পৌঁছাতে পারে — port-forward দিয়ে পরীক্ষা করুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab11` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev`/`staging`/`prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Service ছাড়া Pod-এর IP দিয়ে সরাসরি যোগাযোগ করলে কী সমস্যা হয়?
2. ClusterIP, NodePort ও LoadBalancer — কখন কোনটি ব্যবহার করবেন?
3. Endpoints কী? Pod প্রস্তুত না হলে Endpoints-এ কী ঘটে?
4. Service-এর Selector ভুল হলে কীভাবে বুঝবেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১২ — অভ্যন্তরীণ DNS

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এর অভ্যন্তরীণ DNS ব্যবস্থা — CoreDNS — বোঝা ও ব্যবহার করা। ক্লাস্টারের ভেতরে প্রতিটি Service একটি DNS নাম পায়: `<service>.<namespace>.svc.cluster.local`। একই Namespace-এর ভেতর থেকে কেবল Service-এর নাম লিখলেই চলে, ভিন্ন Namespace থেকে পুরো নাম লিখতে হয়। Pod-এরও DNS নাম থাকে, তবে Service-এর নামই বেশি ব্যবহৃত হয়। DNS ছাড়া মাইক্রোসার্ভিস আর্কিটেকচার কাজ করত না — প্রতিটি সার্ভিস অন্যটিকে নাম দিয়ে ডাকে, IP দিয়ে নয়। এই ল্যাবে আপনি দুটি Namespace-এ Service তৈরি করে নাম দিয়ে যোগাযোগ পরীক্ষা করবেন, CoreDNS Pod দেখবেন এবং DNS ব্যর্থতার সমস্যা সমাধান শিখবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

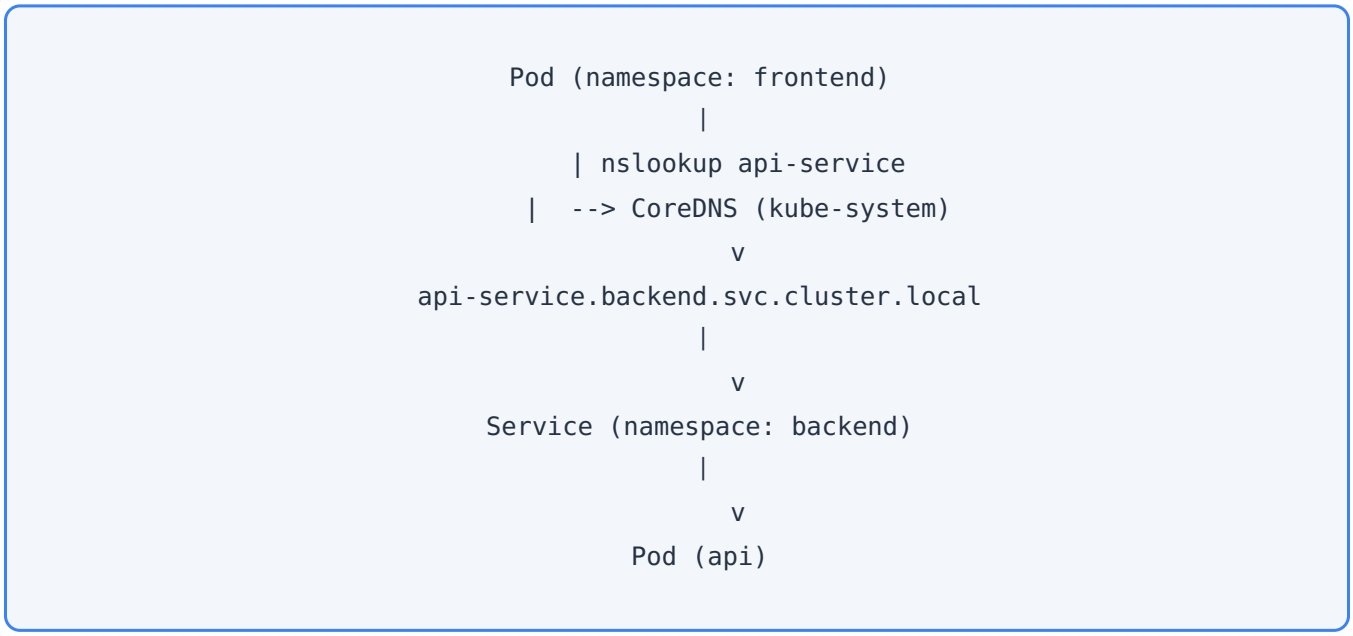
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



CoreDNS ক্লাস্টারের DNS সার্ভার হিসেবে `kube-system` Namespace-এ চলে। Pod-এর `/etc/resolv.conf`-এ CoreDNS-এর ঠিকানা লেখা থাকে, তাই Pod-এর যেকোনো নামের অনুরোধ প্রথমে CoreDNS-এর কাছে যায়।

ধাপে ধাপে নির্দেশনা

1. **দুটি Namespace ও Service তৈরি করুন:** `kubectl create namespace frontend` ও `kubectl create namespace backend` চালান। backend-এ একটি nginx Deployment ও Service (`api-service`) তৈরি করুন; frontend-এ একটি অস্থায়ী busybox Pod চালান।
2. **একই Namespace-এ নাম দিয়ে পরীক্ষা করুন:** backend Namespace-এর ভেতর থেকে একটি অস্থায়ী Pod চালিয়ে `nslookup api-service` ও `wget -qO- http://api-service` চালান। সংক্ষিপ্ত নামেই কাজ হওয়া উচিত।
3. **ভিন্ন Namespace থেকে পুরো নাম ব্যবহার করুন:** frontend Namespace-এর Pod থেকে `nslookup api-service.backend.svc.cluster.local` চালান। সংক্ষিপ্ত নাম এখানে কাজ করবে না — কারণ DNS প্রথমে নিজের Namespace-এ খোঁজে। পুরো নাম (FQDN) সবসময় কাজ করে।
4. **Pod-এর resolv.conf দেখুন:** অস্থায়ী Pod-এর ভেতরে `cat /etc/resolv.conf` চালিয়ে দেখুন nameserver হিসেবে CoreDNS-এর ClusterIP লেখা আছে। `search` লাইনে Namespace-এর নাম দেখুন — এটিই সংক্ষিপ্ত নামকে পুরো নামে রূপান্তর করে।
5. **CoreDNS Pod দেখুন:** `kubectl get pods -n kube-system -l k8s-app=kube-dns` চালিয়ে CoreDNS Pod-গুলো দেখুন। `kubectl logs -n kube-system -l k8s-app=kube-dns` দিয়ে তার লগ দেখুন — কোনো ত্রুটি আছে কি না।
6. **ভুল নাম পরীক্ষা করুন:** ইচ্ছা করে ভুল Service নাম দিয়ে `nslookup wrong-service` চালিয়ে দেখুন কী ত্রুটি আসে। এই ত্রুটি চিনে রাখুন — অ্যাপ্লিকেশন লগে `no such host` বা `NXDOMAIN` দেখলে প্রথমে Service-এর নাম ও Namespace যাচাই করতে হয়।

7. **headless Service-এর ধারণা নিন:** `clusterIP: None` দিয়ে একটি headless Service তৈরি করে `nslookup` চালিয়ে দেখুন — এখন Service-এর IP-র বদলে সরাসরি Pod-গুলোর IP ফেরত আসে। StatefulSet ও ডাটাবেস ক্লাস্টারে এই পদ্ধতি ব্যবহৃত হয়।
8. **পরীক্ষার Pod ও Namespace মুছে ফেলুন:** অস্থায়ী Pod মুছে `kubectl delete namespace frontend backend` চালিয়ে পরিষ্কার করুন।

কনফিগারেশন ফাইল

dns-demo.yaml — backend Service (DNS পরীক্ষার জন্য) — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
  namespace: backend
  labels:
    app: api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: api-service
  namespace: backend
  labels:
    app: api
spec:
  type: ClusterIP
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: api-headless
  namespace: backend
```

```
spec:
  clusterIP: None
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace frontend
kubectl create namespace backend
kubectl apply -f dns-demo.yaml
kubectl run dns-test --image=busybox:1.36 -n backend --rm -it -- sh
kubectl run dns-test --image=busybox:1.36 -n frontend --rm -it -- sh
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl delete namespace frontend backend
```

প্রত্যাশিত ফলাফল ও যাচাই

একই Namespace-এ সংক্ষিপ্ত নামে ও ভিন্ন Namespace-এ পুরো নামে DNS রেজলভ হবে; `wget` করলে nginx-এর পাতা ফিরবে। ভুল নামে `NXDOMAIN` ত্রুটি আসবে। headless Service-এ Pod-গুলোর IP সরাসরি ফিরবে। নমুনা (illustrative): `Server: 10.96.0.10 — Address: api-service.backend.svc.cluster.local 10.96.x.x`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **নাম রেজলভ হচ্ছে না:** Service তৈরি হয়েছে কি না, নাম ও Namespace ঠিক আছে কি না দেখুন; CoreDNS Pod চলছে কি না যাচাই করুন।
- **সংক্ষিপ্ত নাম ভিন্ন Namespace-এ কাজ করছে না:** এটিই স্বাভাবিক — ভিন্ন Namespace থেকে পুরো FQDN ব্যবহার করুন।
- **CoreDNS Pod চলছে না:** `kubectl get pods -n kube-system` দেখুন; CoreDNS না চললে ক্লাস্টারের সব নাম রেজলভ বন্ধ হয়ে যাবে।

TROUBLE: কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab12` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Service-এর DNS নামের পূর্ণ রূপটি লিখুন ও প্রতিটি অংশের অর্থ বলুন।
2. একই Namespace-এ সংক্ষিপ্ত নাম কেন কাজ করে, ভিন্ন Namespace-এ কেন করে না?
3. headless Service কখন ব্যবহার করা হয়? সাধারণ Service-এর সঙ্গে পার্থক্য কী?
4. অ্যাপ্লিকেশন লগে `no such host` দেখলে প্রথমে কী কী যাচাই করবেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৩ — Ingress Controller ও TLS

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Ingress — ক্লাস্টারের বাইরে থেকে HTTP ও HTTPS ট্রাফিক ভেতরের Service-গুলোতে পৌঁছে দেওয়ার ব্যবস্থা — স্থাপন ও ব্যবহার করা। Service-এর NodePort দিয়ে বাইরে থেকে পৌঁছানো গেলেও প্রতিটি সার্ভিসের জন্য আলাদা পোর্ট মনে রাখা অসুবিধাজনক। Ingress একটি প্রবেশদ্বার: একটি ডোমেইন নাম ও পথ (path) দেখে ট্রাফিক সঠিক Service-এ পাঠায়। Ingress কাজ করার জন্য একটি Ingress Controller প্রয়োজন — যেমন NGINX Ingress Controller — যা আসলে একটি বিশেষ Pod, যা Ingress নিয়ম পড়ে নিজের কনফিগারেশন আপডেট করে। এই ল্যাবে আপনি kind-এ NGINX Ingress Controller ইনস্টল করবেন, দুটি Service-এর জন্য Ingress নিয়ম লিখবেন, TLS-এর জন্য স্ব-স্বাক্ষরিত শংসাপত্র দিয়ে Secret তৈরি করবেন এবং HTTPS-এ পরীক্ষা করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

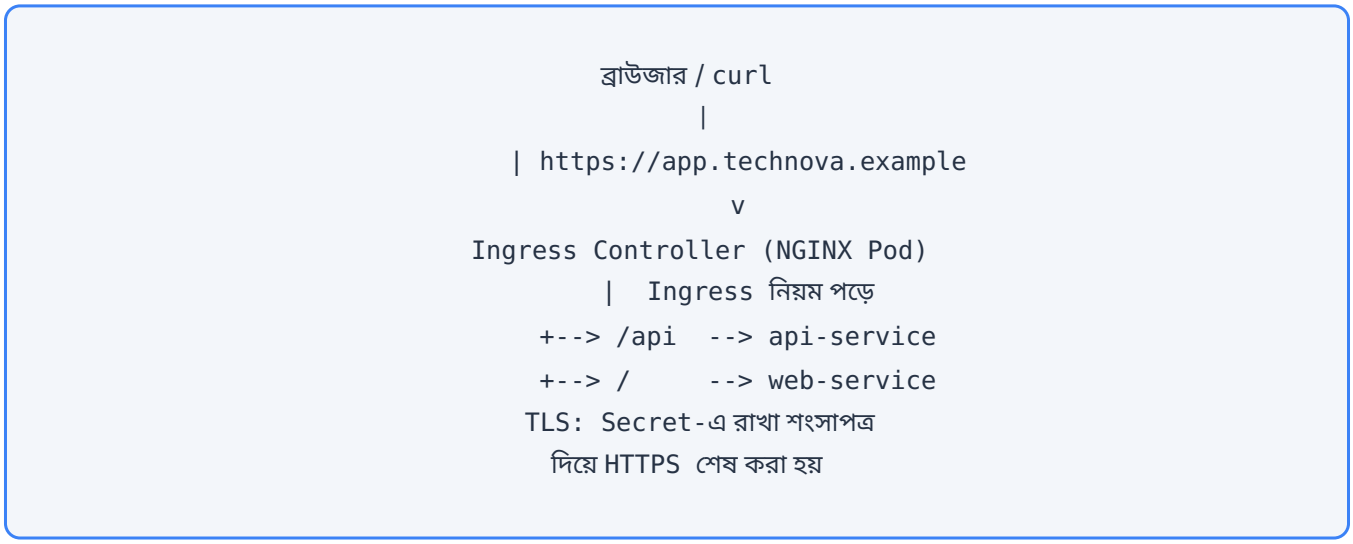
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, openssl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Ingress নিজে কোনো সার্ভার নয় — এটি একটি নিয়মের সেট। Ingress Controller সেই নিয়ম বাস্তবায়ন করে। Controller ছাড়া Ingress তৈরি করলেও কোনো ট্রাফিক আসবে না — এটি নতুনদের সবচেয়ে সাধারণ ভুল।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab13` চালিয়ে দুটি Deployment ও Service তৈরি করুন — একটি web, একটি api। দুটি Service-ই ClusterIP ধরনের হবে; বাইরের ট্রাফিক কেবল Ingress দিয়ে আসবে।
2. **NGINX Ingress Controller ইনস্টল করুন:** kind-এর জন্য অফিসিয়াল NGINX Ingress Controller ম্যানিফেস্ট প্রয়োগ করুন (kind-এর জন্য তৈরি সংস্করণ)। ইনস্টলের পর `kubectl get pods -n ingress-nginx` চালিয়ে controller Pod `Running` হওয়া পর্যন্ত অপেক্ষা করুন। এই ধাপে ইন্টারনেট থেকে ইমেজ ডাউনলোড হয়।
3. **Ingress নিয়ম লিখুন:** নিচের YAML-এ একটি Ingress আছে: `app.technova.example` হোস্টে `/` পথে web-service-এ ও `/api` পথে api-service-এ ট্রাফিক যাবে। `pathType: Prefix` মানে পথের শুরু মিললেই হবে। `ingressClassName: nginx` দিয়ে বোঝানো হয়েছে কোন controller এই নিয়মটি পড়বে।
4. **Ingress তৈরি করে যাচাই করুন:** `kubectl apply -f ingress.yaml` চালান, তারপর `kubectl get ingress -n lab13` দেখুন। ADDRESS কলামে controller-এর ঠিকানা দেখা যাওয়া উচিত। `kubectl describe ingress -n lab13`-এ নিয়মগুলো মিলিয়ে দেখুন।
5. **হোস্ট ফাইলে নাম যোগ করে পরীক্ষা করুন:** আপনার কম্পিউটারের hosts ফাইলে `127.0.0.1 app.technova.example` যোগ করুন (এটি কাল্পনিক ডোমেইন, কেবল লোকাল পরীক্ষার জন্য)। তারপর `kubectl port-forward` দিয়ে controller-এর পোর্টে পৌঁছে `curl -H 'Host: app.technova.example' http://localhost:8080/` চালিয়ে পরীক্ষা করুন।
6. **TLS-এর জন্য Secret তৈরি করুন:** স্ব-স্বাক্ষরিত শংসাপত্র তৈরি করুন: `openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out tls.crt -subj '/CN=app.technova.example'`। তারপর `kubectl create secret tls technova-tls --cert=tls.crt --key=tls.key -n lab13` চালান। এটি কেবল ল্যাবের জন্য — প্রোডাকশনে Let's Encrypt বা প্রতিষ্ঠানের শংসাপত্র ব্যবহার করা হয়।

7. **TLS-সহ Ingress আপডেট করুন:** নিচের দ্বিতীয় YAML-এ `tls` অংশ যোগ করা হয়েছে। প্রয়োগ করে `curl -k https://app.technova.example` দিয়ে HTTPS পরীক্ষা করুন। `-k` ফ্ল্যাগ স্ব-স্বাক্ষরিত শংসাপত্রের সতর্কতা উপেক্ষা করে — কেবল ল্যাবের জন্য।
8. **পরিষ্কার করুন:** Ingress, Secret ও শংসাপত্র ফাইল মুছে ফেলুন। hosts ফাইলে যোগ করা লাইনটি মুছে দিন, যাতে ভবিষ্যতে বিভ্রান্তি না হয়।

কনফিগারেশন ফাইল

ingress.yaml — Ingress ও TLS — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-ingress
  namespace: lab13
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - app.technova.example
      secretName: technova-tls
  rules:
    - host: app.technova.example
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab13
kubectl get pods -n ingress-nginx
kubectl apply -f ingress.yaml
```

```
kubectl get ingress -n lab13
kubectl describe ingress technova-ingress -n lab13
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout tls.key -out tls.crt -subj '/
CN=app.technova.example'
kubectl create secret tls technova-tls --cert=tls.crt --key=tls.key -n lab13
curl -H 'Host: app.technova.example' http://localhost:8080/
```

প্রত্যাশিত ফলাফল ও যাচাই

Ingress-এ ADDRESS দেখা যাবে, `/` পথে web-service-এর পাতা ও `/api` পথে api-service-এর সাড়া ফিরবে। TLS প্রয়োগের পর HTTPS-এ সংযোগ হবে (স্ব-স্বাক্ষরিত বলে ব্রাউজার সতর্ক করবে — সেটিই প্রত্যাশিত)। নমুনা (illustrative): `NAME CLASS HOSTS ADDRESS PORTS — technova-ingress nginx app.technova.example 172.18.0.2 80, 443`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Ingress-এ ADDRESS খালি:** Ingress Controller চলছে কি না দেখুন; `ingressClassName` controller-এর সঙ্গে মিলছে কি না যাচাই করুন।
- **404 আসছে:** পথ (path) ও Service-এর নাম মিলছে কি না দেখুন; `describe ingress`-এ নিয়ম দেখুন।
- **TLS-এ ত্রুটি:** Secret-এর নাম ও হোস্টের নাম মিলছে কি না দেখুন; শংসাপত্রের CN হোস্টের সঙ্গে মিলতে হবে।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab13` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Ingress ও Service-এর পার্থক্য কী? Ingress ছাড়া বাইরে থেকে ট্রাফিক আনা যায় কি?
2. Ingress Controller না থাকলে Ingress তৈরি করলে কী ঘটে?
3. TLS termination মানে কী? শংসাপত্র কোথায় থাকে?
4. একটি Ingress দিয়ে একাধিক Service-এ ট্রাফিক পাঠানো যায় কীভাবে?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৪ — PersistentVolume ও PersistentVolumeClaim

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এ স্থায়ী স্টোরেজ — PersistentVolume (PV) ও PersistentVolumeClaim (PVC) — শেখা। Pod-এর ভেতরের ফাইল Pod মুছে গেলে হারিয়ে যায়; ডাটাবেসের ডেটা, আপলোড করা ফাইল বা লগ — এগুলো Pod-এর জীবনের চেয়ে দীর্ঘস্থায়ী হতে হয়। PV হলো ক্লাস্টারের স্টোরেজের একটি অংশ, যা অ্যাডমিন তৈরি করেন বা StorageClass দিয়ে স্বয়ংক্রিয়ভাবে তৈরি হয়। PVC হলো ব্যবহারকারীর অনুরোধ — এতটুকু জায়গা, এই ধরনের অ্যাক্সেস চাই। PVC Pod-এ মাউন্ট করা হয়, আর Pod মুছে গেলেও ডেটা PV-তে থেকে যায়। এই ল্যাবে আপনি kind-এ একটি hostPath PV তৈরি করবেন, PVC দিয়ে তা দাবি করবেন, Pod-এ মাউন্ট করে ফাইল লিখবেন, Pod মুছে আবার তৈরি করে দেখবেন ডেটা থেকে গেছে কি না।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

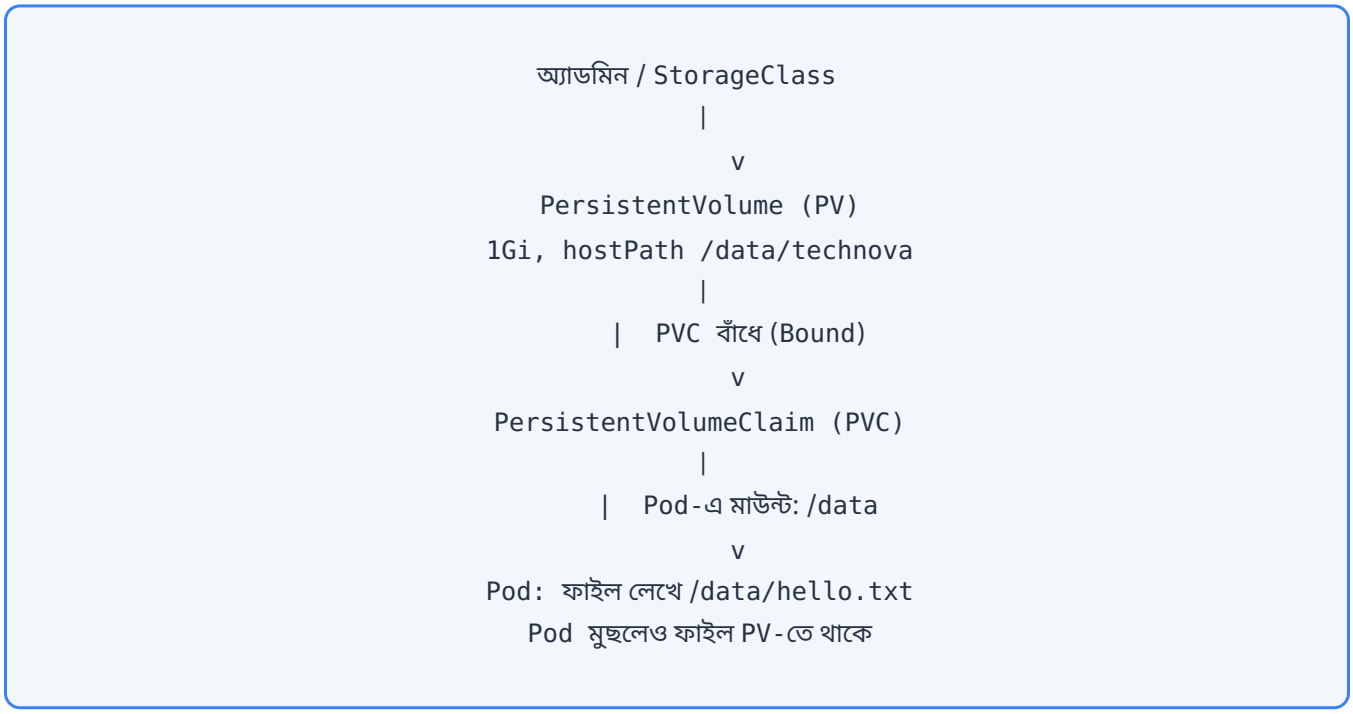
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



PV ও PVC আলাদা করার কারণ হলো ভূমিকা আলাদা করা: অ্যাডমিন স্টোরেজ সরবরাহ করেন, ডেভেলপার শুধু চান। StorageClass থাকলে PVC তৈরি করলেই স্বয়ংক্রিয়ভাবে PV তৈরি হয় — একে dynamic provisioning বলে, আর হাতে PV তৈরি করাকে static provisioning বলে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab14` চালান।
2. **PV ও PVC-এর YAML লিখুন:** নিচের YAML-এ একটি hostPath PV ও একটি PVC আছে। hostPath মানে নোডের ডিস্কের একটি পথ — কেবল লোকাল পরীক্ষার জন্য, প্রোডাকশনে নয়। `storage: 1Gi` ও `accessModes: ReadWriteOnce` মানে একটি নোড থেকে পড়া-লেখা করা যাবে। PVC-র অনুরোধ PV-র ক্ষমতার সঙ্গে মিলতে হবে।
3. **PV ও PVC তৈরি করুন:** `kubectl apply -f pv-pvc.yaml` চালান। তারপর `kubectl get pv` ও `kubectl get pvc -n lab14` দেখুন — PVC-র STATUS `Bound` হওয়া উচিত। Bound না হলে PVC-র অনুরোধের সঙ্গে কোনো PV মিলছে না।
4. **PVC মাউন্ট করা Pod তৈরি করুন:** নিচের দ্বিতীয় YAML-এ Pod-টি PVC-কে `/data` পথে মাউন্ট করেছে। Pod তৈরি করে `kubectl exec -it storage-pod -n lab14 -- sh -c 'echo Hello TechNova > /data/hello.txt && cat /data/hello.txt'` চালিয়ে ফাইল লিখুন ও পড়ুন।
5. **Pod মুছে ডেটা টিকে আছে কি না পরীক্ষা করুন:** `kubectl delete pod storage-pod -n lab14` দিয়ে Pod মুছে আবার তৈরি করুন। নতুন Pod-এর ভেতরে `cat /data/hello.txt` চালিয়ে দেখুন — ফাইলটি এখনো আছে। এটিই স্থায়ী স্টোরেজের প্রমাণ।
6. **StorageClass দেখুন:** `kubectl get storageclass` চালিয়ে দেখুন kind-এ কোন StorageClass আছে। `storageClassName: ''` না দিলে PVC ডিফল্ট StorageClass থেকে স্বয়ংক্রিয়ভাবে PV চাইতে পারে — তাই এই ল্যাবে PV-তে `storageClassName: manual` ও PVC-তে একই নাম দেওয়া হয়েছে।

7. **reclaimPolicy বুঝুন:** PV-র `persistentVolumeReclaimPolicy: Retain` মানে PVC মুছে গেলেও PV-র ডেটা থেকে যাবে। `Delete` হলে PVC মুছলেই PV ও ডেটা মুছে যাবে। প্রোডাকশনে ভুল reclaimPolicy ডেটা হারানোর কারণ হতে পারে।
8. **পরিষ্কার করুন:** Pod, PVC ও PV মুছে ফেলুন। Retain নীতির PV মুছতে হলে প্রথমে PVC মুছুন, তারপর PV মুছুন; প্রয়োজনে `kubectl patch pv <নাম> -p '{"spec":{"persistentVolumeReclaimPolicy":"Delete"}}'` ব্যবহার করুন।

কনফিগারেশন ফাইল

pv-pvc.yaml — PV, PVC ও মাউন্ট করা Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: technova-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /tmp/technova-data
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: technova-pvc
  namespace: lab14
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
  namespace: lab14
  labels:
    app: storage-demo
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: data-volume
```

```
          mountPath: /data
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: technova-pvc
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab14
kubectl apply -f pv-pvc.yaml
kubectl get pv
kubectl get pvc -n lab14
kubectl exec -it storage-pod -n lab14 -- sh -c 'echo Hello TechNova > /data/hello.txt && cat /
data/hello.txt'
kubectl delete pod storage-pod -n lab14
kubectl apply -f pv-pvc.yaml
kubectl exec -it storage-pod -n lab14 -- cat /data/hello.txt
kubectl delete -f pv-pvc.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

PVC `Bound` হবে, Pod-এর ভেতরে ফাইল লেখা ও পড়া যাবে এবং Pod মুছে আবার তৈরি করার পরও `/data/hello.txt` ফাইলটি থাকবে। নমুনা (illustrative): `NAME STATUS VOLUME CAPACITY ACCESS MODES — technova-pvc Bound technova-pv 1Gi RWO`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **PVC Pending-এ আটকে আছে:** কোনো PV-র ক্ষমতা, accessMode বা storageClassName মিলছে না — `describe pvc` দেখুন।
- **Pod-এ ফাইল লেখা যাচ্ছে না:** ভলিউম মাউন্টের পথ ও PVC-র নাম ঠিক আছে কি না দেখুন; hostPath-এর পথ নোডে আছে কি না যাচাই করুন।
- **PV মুছতে পারছেন না:** PVC আগে মুছুন; Retain নীতির PV-তে finalizer আটকে থাকতে পারে।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab14` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. PV ও PVC আলাদা কেন? সরাসরি Pod-এ hostPath মাউন্ট করলে কী সমস্যা?
2. static ও dynamic provisioning-এর পার্থক্য কী? StorageClass-এর ভূমিকা কী?
3. ReadWriteOnce, ReadOnlyMany ও ReadWriteMany — কখন কোনটি দরকার?
4. reclaimPolicy Retain ও Delete-এর পার্থক্য কী? ভুল নীতি কী বিপদ ডেকে আনতে পারে?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৫ — RBAC ও ServiceAccount

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এর অনুমতি ব্যবস্থা — RBAC (Role-Based Access Control) ও ServiceAccount — শেখা। Kubernetes-এ কে কী করতে পারে, তা RBAC দিয়ে নিয়ন্ত্রণ করা হয়। ServiceAccount হলো Pod-এর পরিচয় — Pod যখন API সার্ভারের সঙ্গে কথা বলে, তখন সে ServiceAccount-এর পরিচয়ে কথা বলে। Role বলে দেয় একটি Namespace-এর ভেতরে কী অনুমতি আছে, ClusterRole বলে দেয় পুরো ক্লাস্টারে কী অনুমতি আছে, আর RoleBinding ও ClusterRoleBinding সেই অনুমতি নির্দিষ্ট ব্যবহারকারী বা ServiceAccount-এর সঙ্গে জোড়া দেয়। এই ল্যাবে আপনি একটি ServiceAccount তৈরি করবেন, তাকে কেবল Pod পড়ার অনুমতি দেবেন, সেই পরিচয়ে Pod চালিয়ে পরীক্ষা করবেন যে সে Pod মুছতে পারে না, এবং ভুল অনুমতি দিলে কী ঘটে তা দেখবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



RBAC-এর মূল নীতি হলো least privilege — যতটুকু দরকার, ঠিক ততটুকু অনুমতি দেওয়া। ডিফল্ট ServiceAccount-কে বেশি অনুমতি দেওয়া সবচেয়ে সাধারণ নিরাপত্তা ভুলগুলোর একটি।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab15` চালান।
2. **ServiceAccount তৈরি করুন:** নিচের YAML-এ একটি ServiceAccount (`app-reader`) আছে। `kubectl apply -f rbac.yaml` চালান। তারপর `kubectl get serviceaccounts -n lab15` দিয়ে দেখুন।
3. **Role ও RoleBinding তৈরি করুন:** একই YAML-এ একটি Role আছে যা কেবল Pod-এর `get`, `list` ও `watch` অনুমতি দেয়, আর একটি RoleBinding সেই Role-কে ServiceAccount-এর সঙ্গে জোড়া দেয়। `kubectl describe role pod-reader -n lab15` দিয়ে অনুমতি দেখুন।
4. **অনুমতি যাচাই করুন:** `kubectl auth can-i list pods --as=system:serviceaccount:lab15:app-reader -n lab15` চালিয়ে দেখুন উত্তর `yes` আসে। তারপর `kubectl auth can-i delete pods --as=system:serviceaccount:lab15:app-reader -n lab15` চালিয়ে দেখুন উত্তর `no` আসে। এই কমান্ডটি অনুমতি পরীক্ষার সবচেয়ে দ্রুত উপায়।
5. **সেই ServiceAccount দিয়ে Pod চালান:** নিচের দ্বিতীয় YAML-এ Pod-টি `serviceAccountName: app-reader` ব্যবহার করে। Pod তৈরি করে তার ভেতরে ঢুকে `kubectl` না থাকলে `curl` দিয়ে API সার্ভারে অনুরোধ পাঠিয়ে দেখুন — Pod পড়া যাবে, মোছা যাবে না। (লোকাল পরীক্ষায় টোকেন মাউন্টের বিস্তারিত `describe pod`-এ দেখা যায়।)
6. **ভুল অনুমতি পরীক্ষা করুন:** RoleBinding মুছে `auth can-i` আবার চালিয়ে দেখুন উত্তর `no` হয়ে গেছে। তারপর আবার প্রয়োগ করে অনুমতি ফিরিয়ে আনুন। এতে বোঝা যায় অনুমতি Binding ছাড়া কাজ করে না।
7. **ClusterRole-এর ধারণা নিন:** ClusterRole পুরো ক্লাস্টারে প্রযোজ্য — যেমন সব Namespace-এর Pod পড়া। ClusterRoleBinding দিয়ে তাকে ServiceAccount-এর সঙ্গে জোড়া দেওয়া হয়। এই ল্যাবে ClusterRole তৈরি করবেন না — কেবল ধারণাটি মনে রাখুন, প্রজেক্ট ২-এ এটি ব্যবহার করা হবে।
8. **পরিষ্কার করুন:** `kubectl delete -f rbac.yaml` দিয়ে সব মুছে ফেলুন।

কনফিগারেশন ফাইল

rbac.yaml — ServiceAccount, Role ও RoleBinding — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-reader
  namespace: lab15
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: lab15
rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-reader-binding
  namespace: lab15
subjects:
  - kind: ServiceAccount
    name: app-reader
    namespace: lab15
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: v1
kind: Pod
metadata:
  name: reader-pod
  namespace: lab15
  labels:
    app: rbac-demo
spec:
  serviceAccountName: app-reader
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab15
kubectl apply -f rbac.yaml
kubectl get serviceaccounts -n lab15
kubectl auth can-i list pods --as=system:serviceaccount:lab15:app-reader -n lab15
kubectl auth can-i delete pods --as=system:serviceaccount:lab15:app-reader -n lab15
kubectl describe role pod-reader -n lab15
kubectl delete -f rbac.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

ServiceAccount তৈরি হবে, Role-এ কেবল পড়ার অনুমতি দেখা যাবে, `can-i list pods`-এ `yes` ও `can-i delete pods`-এ `no` আসবে। নমুনা (illustrative): `yes — no`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **can-i-তে সবকিছু no আসছে:** RoleBinding-এর subject-এর নাম ও Namespace ঠিক আছে কি না দেখুন; Role-এর নাম মিলছে কি না যাচাই করুন।
- **Pod-এ ServiceAccount যুক্ত হচ্ছে না:** Pod YAML-এ `serviceAccountName` ঠিকমতো লেখা আছে কি না দেখুন; Pod তৈরি হওয়ার পর আর বদলানো যায় না।
- **ClusterRole ও Role গুলিয়ে যাচ্ছে:** Role একটি Namespace-এ, ClusterRole পুরো ক্লাস্টারে — নামের শেষে Cluster থাকলেই বুঝবেন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab15` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. ServiceAccount ও ব্যবহারকারী (User)-এর পার্থক্য কী?
2. Role, ClusterRole, RoleBinding ও ClusterRoleBinding — চারটির ভূমিকা আলাদা করে বলুন।
3. least privilege নীতি কেন গুরুত্বপূর্ণ? ডিফল্ট ServiceAccount-কে বেশি অনুমতি দিলে কী বিপদ?
4. `kubectl auth can-i` দিয়ে কীভাবে অনুমতি পরীক্ষা করা যায়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৬ — SecurityContext ও Non-Root কন্টেইনার

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য কন্টেইনারের নিরাপত্তা — SecurityContext দিয়ে non-root ব্যবহারকারী হিসেবে চালানো, ক্ষমতা (capabilities) কমানো ও ফাইল সিস্টেমকে কেবল-পড়ার (read-only) করা — শেখা। ডিফল্টভাবে অনেক কন্টেইনার root ব্যবহারকারী হিসেবে চলে; কন্টেইনার থেকে বেরিয়ে আসার দুর্বলতা থাকলে আক্রমণকারী হোস্টের root পেয়ে যেতে পারে। non-root ব্যবহারকারী হিসেবে চালালে, অপ্রয়োজনীয় Linux capability সরিয়ে দিলে ও root ফাইল সিস্টেম read-only করলে আক্রমণের সুযোগ অনেক কমে যায়। এই ল্যাবে আপনি একটি Pod-কে non-root হিসেবে চালাবেন, root হিসেবে চালানোর চেষ্টা করলে কী বাধা আসে তা দেখবেন, `allowPrivilegeEscalation: false` ও `readOnlyRootFilesystem: true` পরীক্ষা করবেন এবং Pod Security Standards-এর ধারণা নেবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



SecurityContext দুই স্তরে দেওয়া যায় — Pod স্তরে (সব কন্টেইনারের জন্য) ও কন্টেইনার স্তরে (নির্দিষ্ট কন্টেইনারের জন্য)। কন্টেইনার স্তরের সেটিং Pod স্তরের সেটিংকে ওভাররাইড করে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab16` চালান।
2. **সুরক্ষিত Pod-এর YAML লিখুন:** নিচের YAML-এ Pod স্তরে `runAsNonRoot: true` ও `runAsUser: 1000` দেওয়া হয়েছে, আর কন্টেইনার স্তরে `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true` ও `capabilities: drop: [ALL]` দেওয়া হয়েছে। প্রতিটি ফিল্ডের অর্থ পড়ে বুঝুন।
3. **Pod তৈরি করে ব্যবহারকারী যাচাই করুন:** `kubectl apply -f secure-pod.yaml` চালান, তারপর `kubectl exec -it secure-pod -n lab16 -- id` চালিয়ে দেখুন ব্যবহারকারী `uid=1000` দেখাচ্ছে, root (0) নয়।
4. **root ফাইল সিস্টেমে লেখার চেষ্টা করুন:** Pod-এর ভেতরে `touch /test.txt` চালিয়ে দেখুন অনুমতি মিলবে না — read-only ফাইল সিস্টেমের প্রমাণ। তারপর `touch /tmp/test.txt` চালিয়ে দেখুন `/tmp`-এ লেখা যায়, কারণ সেখানে একটি খালি ভলিউম (emptyDir) মাউন্ট করা হয়েছে।
5. **root হিসেবে চালানোর চেষ্টা করুন:** একটি দ্বিতীয় Pod-এ `runAsUser: 0` ও `runAsNonRoot: true` একসঙ্গে দিয়ে দেখুন Pod তৈরি হয় না — kubelet বাধা দেয়। এই বাধাটিই নিরাপত্তা নীতির কাজ।
6. **capabilities-এর প্রভাব দেখুন:** `capabilities: drop: [ALL]` থাকায় কন্টেইনার `ping`-এর মতো raw socket দরকার এমন কাজ করতে পারবে না। প্রয়োজনে নির্দিষ্ট capability (যেমন `NET_BIND_SERVICE`) যোগ করা যায় — তবে কেবল যতটুকু দরকার, ততটুকুই।
7. **Pod Security Standards-এর ধারণা নিন:** Kubernetes-এ Namespace-এ লেবেল দিয়ে Pod Security Standards প্রয়োগ করা যায়: `privileged`, `baseline` ও `restricted`। `restricted` প্রোফাইলে non-root, read-only ফাইল সিস্টেম ও capability বাদ দেওয়া বাধ্যতামূলক। এই ল্যাবের Pod `restricted` প্রোফাইলের কাছাকাছি।

8. **Pod মুছে ফেলুন:** `kubectl delete -f secure-pod.yaml` দিয়ে Pod মুছে ফেলুন।

কনফিগারেশন ফাইল

secure-pod.yaml — non-root ও read-only কন্টেইনার — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
  namespace: lab16
  labels:
    app: secure-demo
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: app
      image: nginx:1.27
      ports:
        - containerPort: 8080
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        runAsNonRoot: true
        runAsUser: 1000
        capabilities:
          drop:
            - ALL
      volumeMounts:
        - name: tmp-volume
          mountPath: /tmp
        - name: cache-volume
          mountPath: /var/cache/nginx
  volumes:
    - name: tmp-volume
      emptyDir: {}
    - name: cache-volume
      emptyDir: {}
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab16
kubectl apply -f secure-pod.yaml
```

```
kubectl get pods -n lab16
kubectl exec -it secure-pod -n lab16 -- id
kubectl exec -it secure-pod -n lab16 -- touch /test.txt
kubectl exec -it secure-pod -n lab16 -- touch /tmp/test.txt
kubectl delete -f secure-pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

Pod `Running` হবে, `id` কমান্ডে `uid=1000` দেখাবে, `/test.txt` লিখতে গেলে অনুমতির ত্রুটি আসবে, কিন্তু `/tmp/test.txt` লেখা যাবে। নমুনা (illustrative): `uid=1000 gid=3000 groups=2000`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **Pod তৈরি হচ্ছে না — CreateContainerConfigError:** ইমেজটি root ব্যবহারকারী হিসেবে চালাতে চাইছে, কিন্তু runAsNonRoot বাধা দিচ্ছে — ইমেজটি non-root-সমর্থিত কি না দেখুন।
- **nginx চালু হচ্ছে না:** nginx ডিফল্টভাবে 80 পোর্টে শোনে, যার জন্য root দরকার — তাই এই ল্যাবে 8080 পোর্ট ব্যবহার করা হয়েছে; ইমেজের কনফিগারেশন দেখুন।
- **read-only ফাইল সিস্টেমে অ্যাপ্লিকেশন লিখতে চাইছে:** যেসব পথে লেখা দরকার, সেখানে emptyDir ভলিউম মাউন্ট করুন — যেমন /tmp ও ক্যাশে পথ।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab16` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. কন্টেইনার root হিসেবে চললে কী ঝুঁকি থাকে? non-root চালালে সেই ঝুঁকি কীভাবে কমে?
2. allowPrivilegeEscalation: false-এর অর্থ কী?
3. readOnlyRootFilesystem চালু করলে অ্যাপ্লিকেশনের কোন কোন পথে সমস্যা হতে পারে, আর সমাধান কী?
4. Pod Security Standards-এর তিনটি প্রোফাইলের পার্থক্য কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৭ — NetworkPolicy

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য NetworkPolicy — Pod-গুলোর মধ্যে নেটওয়ার্ক ট্রাফিক নিয়ন্ত্রণ — শেখা। ডিফল্টভাবে Kubernetes-এ সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে। এটি সুবিধাজনক, কিন্তু নিরাপত্তার দিক থেকে ঝুঁকিপূর্ণ: একটি Pod আক্রান্ত হলে আক্রমণকারী সব Pod-এ পৌঁছাতে পারে। NetworkPolicy দিয়ে বলা যায় কোন Pod কোন Pod-এর সঙ্গে, কোন পোর্টে কথা বলতে পারবে। এই ল্যাবে আপনি একটি frontend Pod-কে কেবল backend Pod-এর সঙ্গে কথা বলতে দেবেন, ডাটাবেস Pod-কে কেবল backend থেকে সংযোগ নিতে দেবেন, আর অন্য সব ট্রাফিক বন্ধ করবেন। তারপর নীতি ভেঙে পরীক্ষা করে দেখবেন বাধাটি কাজ করছে কি না।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

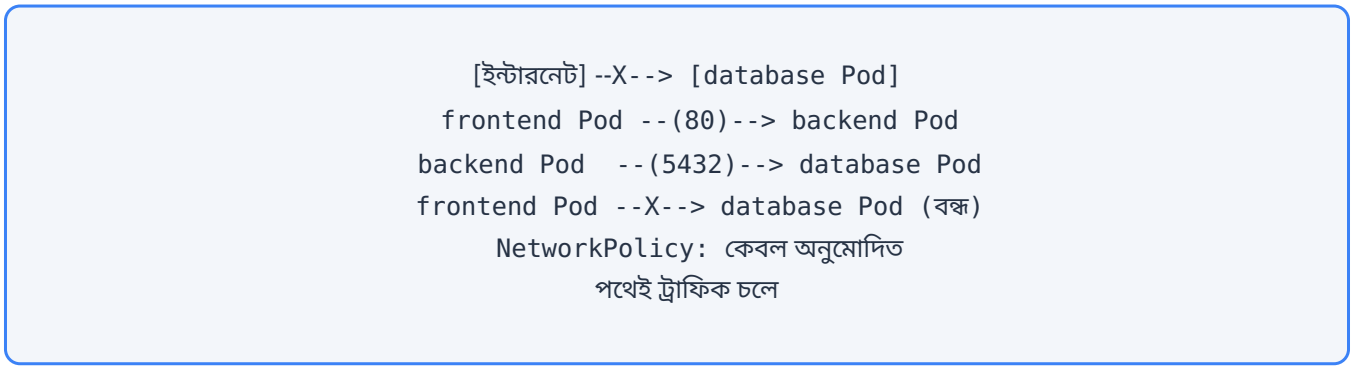
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind/Minikube, kubectl, Calico/Cilium CNI। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



NetworkPolicy কেবল তখনই কাজ করে যখন ক্লাস্টারের CNI প্লাগইন তা সমর্থন করে। kind-এর ডিফল্ট CNI NetworkPolicy সমর্থন করে না — তাই এই ল্যাবে প্রথমে Calico বা Cilium-এর মতো নীতিসমর্থিত CNI ইনস্টল করতে হবে,

অথবা Minikube-এ CNI সক্রিয় করতে হবে। CNI ছাড়া নীতি তৈরি করলেও কোনো বাধা কাজ করবে না — এটি এই ল্যাবের সবচেয়ে গুরুত্বপূর্ণ সতর্কতা।

ধাপে ধাপে নির্দেশনা

1. **CNI যাচাই করুন:** প্রথমে আপনার ক্লাস্টারের CNI NetworkPolicy সমর্থন করে কি না যাচাই করুন। kind-এ ডিফল্ট CNI তা করে না — প্রয়োজনে Calico ইনস্টল করুন অথবা এই ল্যাবটি Minikube-এ `--cni=calico` দিয়ে চালান। CNI ছাড়া নীতি পরীক্ষা করবেন না — ফলাফল ভুল হবে।
2. **কাজের নেমস্পেস ও তিনটি Pod তৈরি করুন:** `kubectl create namespace lab17` চালিয়ে frontend, backend ও database নামে তিনটি nginx Pod তৈরি করুন (প্রতিটিতে আলাদা Label: `app: frontend`, `app: backend`, `app: database`)। প্রথমে নীতি ছাড়া সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে কি না পরীক্ষা করুন।
3. **ডাটাবেসের জন্য NetworkPolicy লিখুন:** নিচের YAML-এ database Pod-এর জন্য একটি নীতি আছে: কেবল `app: backend` Label-এর Pod 5432 পোর্টে সংযোগ করতে পারবে। এই নীতি প্রয়োগ করলেই database Pod-এ অন্য সব ইনকামিং ট্রাফিক বন্ধ হয়ে যাবে।
4. **নীতি প্রয়োগ করে পরীক্ষা করুন:** `kubectl apply -f networkpolicy.yaml` চালান। তারপর backend Pod থেকে database-এ curl করুন — সফল হওয়া উচিত। frontend Pod থেকে database-এ curl করুন — সময় শেষ হয়ে যাওয়া (timeout) উচিত। এই পার্থক্যই নীতির কাজের প্রমাণ।
5. **frontend-এর জন্য নীতি যোগ করুন:** frontend Pod-এর জন্য একটি নীতি যোগ করুন: frontend কেবল backend-এর 80 পোর্টে যেতে পারবে, অন্য কোথাও নয়। প্রয়োগ করে আবার পরীক্ষা করুন।
6. **নীতি দেখুন ও বুঝুন:** `kubectl get networkpolicies -n lab17` ও `kubectl describe networkpolicy db-policy -n lab17` চালিয়ে দেখুন কোন Pod নির্বাচিত হয়েছে, কোন পোর্ট অনুমোদিত। `podSelector` কোন Pod-গুলোর ওপর নীতি প্রযোজ্য তা বলে, `ingress.from` কারা আসতে পারবে তা বলে।
7. **নীতি মুছে পরীক্ষা করুন:** `kubectl delete -f networkpolicy.yaml` দিয়ে নীতি মুছে আবার frontend থেকে database-এ curl করুন — এখন সংযোগ সফল হওয়া উচিত। এতে প্রমাণ হয় বাধাটি নীতির কারণেই ছিল।
8. **পরিষ্কার করুন:** সব Pod ও Namespace মুছে ফেলুন। মনে রাখবেন: NetworkPolicy ডিফল্টভাবে কিছু বন্ধ করে না — নীতি প্রয়োগ করলেই কেবল নির্বাচিত Pod-গুলোর ওপর নিয়ন্ত্রণ শুরু হয়।

কনফিগারেশন ফাইল

networkpolicy.yaml — ডাটাবেসের জন্য NetworkPolicy — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
  namespace: lab17
spec:
  podSelector:
    matchLabels:
      app: database
  policyTypes:
    - Ingress
  ingress:
```

```
    - from:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 5432
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
  namespace: lab17
spec:
  podSelector:
    matchLabels:
      app: frontend
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: backend
      ports:
        - protocol: TCP
          port: 80
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
          podSelector:
            matchLabels:
              k8s-app: kube-dns
      ports:
        - protocol: UDP
          port: 53
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab17
kubectl apply -f networkpolicy.yaml
kubectl get networkpolicies -n lab17
kubectl describe networkpolicy db-policy -n lab17
kubectl exec -it frontend-pod -n lab17 -- wget -qO- --timeout=5 http://database-service
kubectl delete -f networkpolicy.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

নীতি প্রয়োগের আগে সব Pod সব Pod-এর সঙ্গে কথা বলতে পারবে; নীতি প্রয়োগের পর backend থেকে database-এ সংযোগ সফল হবে, কিন্তু frontend থেকে database-এ সংযোগ timeout হবে। নীতি মুছলে আবার সব সংযোগ খুলে যাবে। নমুনা (illustrative): frontend থেকে database-এ `wget: download timed out`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **নীতি প্রয়োগের পরও সব ট্রাফিক চলছে:** আপনার CNI NetworkPolicy সমর্থন করে না — Calico বা Cilium ইনস্টল করুন; kind-এর ডিফল্ট CNI-তে নীতি কাজ করে না।
- **DNS কাজ করছে না:** Egress নীতিতে DNS (UDP 53, kube-dns) অনুমতি দিতে হয় — না হলে নাম রেজলভ বন্ধ হয়ে যায়; উপরের YAML-এ সেই অনুমতি যোগ করা হয়েছে।
- **নীতি মুছেও ট্রাফিক বন্ধ আছে:** CNI-এর ক্যাশে আপডেট হতে কিছু সময় লাগতে পারে — কয়েক সেকেন্ড অপেক্ষা করে আবার পরীক্ষা করুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab17` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. ডিফল্টভাবে সব Pod সব Pod-এর সঙ্গে কথা বলতে পারে কেন? এতে ঝুঁকি কী?
2. NetworkPolicy-র podSelector ও ingress.from-এর পার্থক্য কী?

3. Egress নীতিতে DNS-এর অনুমতি কেন দরকার হয়?
4. আপনার CNI NetworkPolicy সমর্থন করে কি না — তা না জেনে নীতি পরীক্ষা করলে কী ভুল হবে?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৮ — CrashLoopBackOff সমস্যা সমাধান

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এর সবচেয়ে সাধারণ ব্যর্থতা — CrashLoopBackOff — নির্ণয় ও সমাধান করা। CrashLoopBackOff মানে কন্টেইনার চালু হয়েই বারবার বন্ধ হয়ে যাচ্ছে, আর kubelet প্রতিবার আরও বেশি সময় অপেক্ষা করে আবার চালু করার চেষ্টা করছে। কারণ অনেক হতে পারে: অ্যাপ্লিকেশন চালু হতেই ত্রুটি করছে, কনফিগারেশন ফাইল নেই, পোর্ট ইতিমধ্যে ব্যবহৃত হচ্ছে, মেমরি সীমা ছাড়িয়ে OOMKilled হচ্ছে, অথবা liveness প্রোব ব্যর্থ হচ্ছে। এই ল্যাবে আপনি ইচ্ছা করে চারটি ভিন্ন কারণে CrashLoopBackOff তৈরি করবেন, প্রতিটির লক্ষণ দেখবেন এবং ধাপে ধাপে মূল কারণ খুঁজে বের করার পদ্ধতি অনুশীলন করবেন। সমস্যা সমাধানের পদ্ধতিই এই ল্যাবের আসল শিক্ষা — কারণটি নয়।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



CrashLoopBackOff নিজে কোনো কারণ নয় — এটি একটি লক্ষণ। লগে অ্যাপ্লিকেশনের ত্রুটি, describe-এ Events ও Last State, আর RESTARTS গণনা — এই তিনটি মিলিয়ে কারণ বোঝা যায়। `--previous` ফ্ল্যাগটি গুরুত্বপূর্ণ, কারণ বর্তমান কন্টেইনারটি বন্ধ হয়ে গেলে তার লগ দেখতে আগের চেষ্টার লগ লাগে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab18` চালান।
2. **ভুল কমান্ড দিয়ে CrashLoopBackOff তৈরি করুন:** নিচের প্রথম YAML-এ কন্টেইনার এমন একটি কমান্ড চালায় যা সঙ্গে সঙ্গে ব্যর্থ হয় (`exit 1`)। Pod তৈরি করে `kubectl get pods -n lab18 -w` দেখুন — Pod `CrashLoopBackOff` অবস্থায় যাবে, RESTARTS বাড়তে থাকবে।
3. **লগ দেখুন:** `kubectl logs crash-pod -n lab18` ও `kubectl logs crash-pod -n lab18 --previous` চালান। লগে কী ত্রুটি আছে তা পড়ুন। এই ল্যাবে লগে হয়তো কিছুই থাকবে না — কারণ কমান্ডটি কোনো আউটপুট ছাড়াই ব্যর্থ হয়েছে; তখন describe-এর Last State দেখতে হয়।
4. **describe দিয়ে কারণ খুঁজুন:** `kubectl describe pod crash-pod -n lab18` চালান। Containers অংশে Last State-এ Exit Code দেখুন, Events অংশে `Back-off restarting failed container` দেখুন। Exit Code 1 মানে অ্যাপ্লিকেশন নিজে ব্যর্থ হয়েছে।
5. **কনফিগারেশন অনুপস্থিতির কারণে ব্যর্থতা দেখুন:** দ্বিতীয় Pod-এ অ্যাপ্লিকেশন এমন একটি ফাইল পড়তে চায় যা ConfigMap-এ নেই। Pod তৈরি করে লগ দেখুন — `file not found` ধরনের ত্রুটি আসবে। সমাধান: ConfigMap ঠিক করুন অথবা অ্যাপ্লিকেশনের পথ ঠিক করুন।
6. **OOMKilled-এর কারণে CrashLoopBackOff দেখুন:** ল্যাব ৯-এর মতো ছোট মেমরি limit দিয়ে একটি Pod তৈরি করুন। `describe`-এ Last State-এ `Reason: OOMKilled` ও Exit Code 137 দেখুন — এটি মেমরির সমস্যা, অ্যাপ্লিকেশনের ত্রুটি নয়।
7. **সমস্যা সমাধানের চেকলিস্ট তৈরি করুন:** আপনার নোটে লিখুন: প্রথমে get pods, তারপর logs, তারপর logs --previous, তারপর describe pod, তারপর Events ও Last State। এই ক্রমটি প্রতিটি CrashLoopBackOff-এ একইভাবে ব্যবহার করবেন।
8. **Pod-গুলো ঠিক করে মুছে ফেলুন:** ভুল কমান্ড ঠিক করে Pod আবার তৈরি করুন — Pod `Running` হওয়া উচিত। তারপর সব Pod মুছে ফেলুন।

কনফিগারেশন ফাইল

crash-pod.yaml — ইচ্ছাকৃতভাবে ব্যর্থ Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: crash-pod
  namespace: lab18
  labels:
    app: crash-demo
spec:
  restartPolicy: Always
  containers:
    - name: app
```

```
      image: busybox:1.36
      command: ["sh", "-c", "echo Starting app; exit 1"]
---
apiVersion: v1
kind: Pod
metadata:
  name: missing-config-pod
  namespace: lab18
  labels:
    app: crash-demo
spec:
  restartPolicy: Always
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "cat /etc/config/missing.conf || exit 1; sleep 3600"]
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab18
kubectl apply -f crash-pod.yaml
kubectl get pods -n lab18 -w
kubectl logs crash-pod -n lab18
kubectl logs crash-pod -n lab18 --previous
kubectl describe pod crash-pod -n lab18
kubectl delete -f crash-pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

Pod `CrashLoopBackOff` অবস্থায় যাবে, RESTARTS বাড়তে থাকবে, `describe`-এ `Back-off restarting failed container` ইভেন্ট ও Last State-এ Exit Code 1 দেখা যাবে। কমান্ড ঠিক করলে Pod `Running` হবে। নমুনা (illustrative): `NAME READY STATUS RESTARTS AGE — crash-pod 0/1 CrashLoopBackOff 5 (20s ago) 3m`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **লগ খালি দেখাচ্ছে:** কন্টেইনার কোনো আউটপুট ছাড়াই ব্যর্থ হয়েছে — `--previous` দিয়ে আগের চেষ্টার লগ দেখুন, আর describe-এর Last State দেখুন।
- **RESTARTS বাড়ছে না:** Pod হয়তো `restartPolicy: Never` দিয়ে তৈরি — Job-এর Pod-এ এমন হয়; Deployment-এর Pod-এ Always থাকে।

- **সমস্যা ঠিক করেও Pod Running হচ্ছে না:** Pod-এর YAML বদলে `apply` করলে Pod আপডেট হয় না — Pod মুছে আবার তৈরি করতে হয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab18` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. CrashLoopBackOff মানে কী? এটি কেন একটি লক্ষণ, কারণ নয়?
2. `kubectl logs --previous` কখন দরকার হয়?
3. Exit Code 1 ও Exit Code 137-এর পার্থক্য কী? প্রতিটি কী নির্দেশ করে?
4. CrashLoopBackOff-এর সমস্যা সমাধানে আপনার ধাপগুলো কী কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ১৯ — ImagePullBackOff ও Pending Pod সমস্যা সমাধান

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Pod তৈরির আগের দুটি সাধারণ ব্যর্থতা — ImagePullBackOff ও Pending — নির্ণয় ও সমাধান করা। ImagePullBackOff মানে kubelet কন্টেইনারের ইমেজ ডাউনলোড করতে পারছে না: ইমেজের নাম ভুল, ট্যাগ নেই, রেজিস্ট্রিতে অনুমতি নেই, অথবা নেটওয়ার্ক সমস্যা। Pending মানে Pod কোনো নোডে বসতে পারছে না: সম্পদ নেই, নোডে taint আছে, Pod-এর অনুরোধ পূরণ হচ্ছে না, অথবা PVC বাঁধা যাচ্ছে না। দুটি ক্ষেত্রেই Pod-এর কন্টেইনার একবারও চালু হয় না, তাই লগে কিছু থাকে না — সমস্যাটি `describe pod`-এর Events-এ খুঁজতে হয়। এই ল্যাবে আপনি দুটি ব্যর্থতাই ইচ্ছা করে তৈরি করবেন এবং Events পড়ে কারণ খুঁজে বের করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

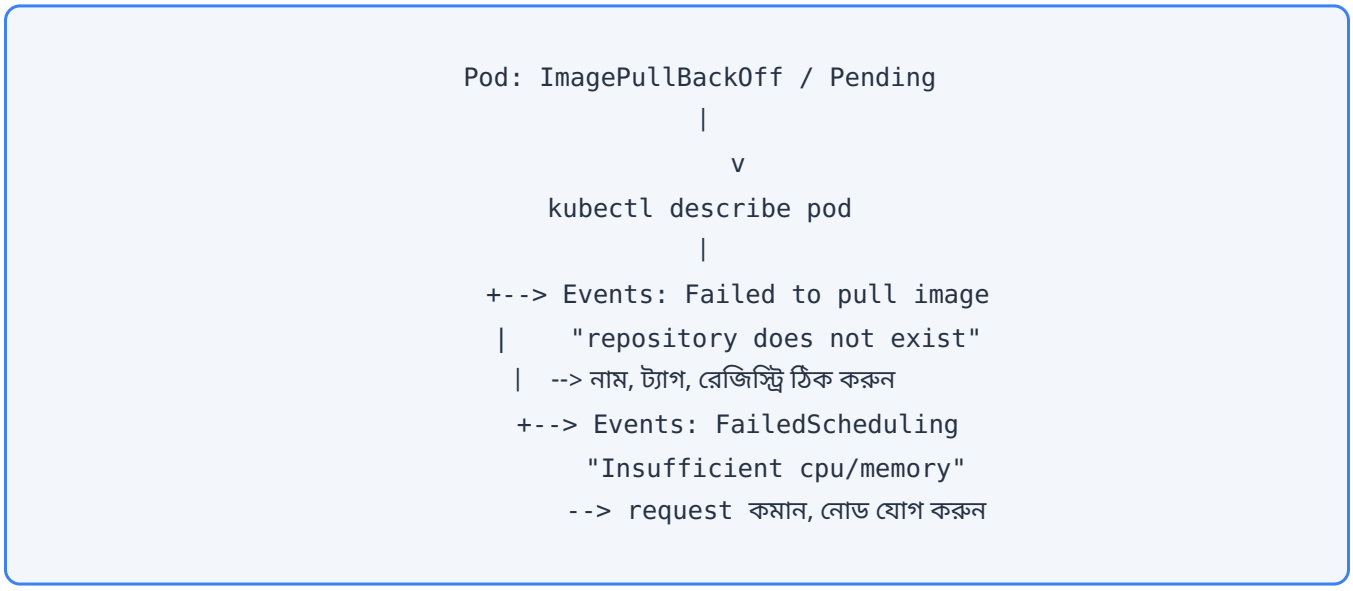
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



ImagePullBackOff-এ সমস্যা ইমেজে, Pending-এ সমস্যা সময়সূচি (scheduling)-এ। দুটিরই লক্ষণ এক — Pod Running হচ্ছে না — কিন্তু কারণ ও সমাধান সম্পূর্ণ আলাদা, তাই Events পড়া ছাড়া উপায় নেই।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace lab19` চালান।
2. **ভুল ইমেজ দিয়ে ImagePullBackOff তৈরি করুন:** নিচের প্রথম YAML-এ এমন একটি ইমেজের নাম দেওয়া হয়েছে যা নেই (`nginx:9.99`)। Pod তৈরি করে `kubectl get pods -n lab19 -w` দেখুন — Pod `ImagePullBackOff` অবস্থায় যাবে।
3. **Events পড়ুন:** `kubectl describe pod bad-image-pod -n lab19` চালান। Events-এ `Failed to pull image` ও `repository does not exist` বা `manifest unknown` বার্তা দেখুন। এটিই মূল কারণ — ইমেজের নাম বা ট্যাগ ভুল।
4. **ইমেজের নাম ঠিক করুন:** ইমেজের নাম `nginx:1.27` করে Pod মুছে আবার তৈরি করুন। Pod `Running` হওয়া উচিত। এই অভ্যাসটি গড়ুন: ইমেজের নাম টাইপ করার আগে রেজিস্ট্রিতে সেই ট্যাগ আছে কি না যাচাই করুন।
5. **ব্যক্তিগত রেজিস্ট্রির ধারণা নিন:** ব্যক্তিগত (private) রেজিস্ট্রির ইমেজ টানতে `imagePullSecrets` দরকার হয় — একটি Secret-এ রেজিস্ট্রির ব্যবহারকারী নাম ও পাসওয়ার্ড (CHANGE-ME-... প্লেসহোল্ডার) রেখে Pod-এ তা উল্লেখ করা হয়। এই ল্যাবে কেবল ধারণাটি শিখুন; আসল শংসাপত্র কখনো উদাহরণে লিখবেন না।
6. **অসম্ভব বড় request দিয়ে Pending তৈরি করুন:** নিচের দ্বিতীয় YAML-এ Pod 100 CPU চাইছে। তৈরি করে `kubectl get pods -n lab19` দেখুন — Pod `Pending`-এ আটকে থাকবে। `describe pod`-এ `FailedScheduling` ইভেন্টে `Insufficient cpu` লেখা থাকবে।
7. **PVC না বাঁধায় Pending দেখুন:** এমন একটি PVC দাবি করা Pod তৈরি করুন যার সঙ্গে কোনো PV মিলছে না। Pod Pending থাকবে এবং Events-এ `persistentvolumeclaim not found` বা `waiting for volume` বার্তা দেখা যাবে। PVC তৈরি করে বা নাম ঠিক করে সমস্যা সমাধান করুন।
8. **সমস্যা সমাধানের চেকলিস্ট লিখুন:** আপনার নোটে লিখুন: ImagePullBackOff হলে — নাম, ট্যাগ, রেজিস্ট্রি, imagePullSecrets যাচাই; Pending হলে — request, নোডের সম্পদ, taint/toleration, PVC যাচাই। দুটি ক্ষেত্রেই প্রথম ধাপ `describe pod`।

কনফিগারেশন ফাইল

bad-image-pod.yaml — ভুল ইমেজ ও অতিরিক্ত request-এর Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Pod
metadata:
  name: bad-image-pod
  namespace: lab19
  labels:
    app: troubleshoot-demo
spec:
  containers:
    - name: nginx
      image: nginx:9.99
      ports:
        - containerPort: 80
---
apiVersion: v1
kind: Pod
metadata:
  name: huge-request-pod
  namespace: lab19
  labels:
    app: troubleshoot-demo
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      resources:
        requests:
          cpu: "100"
          memory: "100Gi"
        limits:
          cpu: "100"
          memory: "100Gi"
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab19
kubectl apply -f bad-image-pod.yaml
kubectl get pods -n lab19 -w
kubectl describe pod bad-image-pod -n lab19
kubectl describe pod huge-request-pod -n lab19
kubectl delete -f bad-image-pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

ভুল ইমেজের Pod `ImagePullBackOff` হবে, Events-এ `Failed to pull image` দেখা যাবে; বড় request-এর Pod `Pending` থাকবে, Events-এ `FailedScheduling: Insufficient cpu` দেখা যাবে। নাম ঠিক করলে প্রথম Pod `Running` হবে। নমুনা (illustrative): `Failed to pull image 'nginx:9.99': manifest unknown`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **ImagePullBackOff বারবার আসছে:** ইমেজের নাম ও ট্যাগ হুবহু মিলছে কি না দেখুন; ব্যক্তিগত রেজিস্ট্রি হলে imagePullSecrets যাচাই করুন।
- **Pending Pod-এর Events খালি:** কয়েক সেকেন্ড অপেক্ষা করুন — scheduler-এর সিদ্ধান্ত Events-এ আসতে সময় লাগতে পারে; `describe pod` আবার চালান।
- **ইমেজ ঠিক করেও Pod Running হচ্ছে না:** Pod-এর ইমেজ বদলে apply করলে Pod আপডেট হয় না — Pod মুছে আবার তৈরি করতে হয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab19` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. ImagePullBackOff ও ErrImagePull-এর পার্থক্য কী?
2. Pending Pod-এর সম্ভাব্য কারণগুলো কী কী? প্রতিটির লক্ষণ কী?

3. ব্যক্তিগত রেজিস্ট্রির ইমেজ টানতে কী দরকার হয়?
4. Pod-এর লগে কিছু না থাকলে সমস্যার কারণ কোথায় খুঁজবেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২০ — রিসোর্স মনিটরিং

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য ক্লাস্টার ও Pod-এর সম্পদ ব্যবহার পর্যবেক্ষণ করা — metrics-server, `kubectl top` ও লগ দেখা শেখা। Kubernetes-এ Pod কত CPU ও মেমরি ব্যবহার করছে, তা না জানলে request ও limit নির্ধারণ করা যায় না, স্কেলিং সিদ্ধান্ত নেওয়া যায় না, আর সমস্যার কারণ বোঝা যায় না। metrics-server ক্লাস্টারের মেট্রিক সংগ্রহ করে API-র মাধ্যমে দেয়; `kubectl top` সেই তথ্য সহজভাবে দেখায়। প্রোডাকশনে Prometheus ও Grafana ব্যবহৃত হয়, তবে এই ল্যাবে metrics-server দিয়েই মূল ধারণা শেখা হবে। এই ল্যাবে আপনি metrics-server ইনস্টল করবেন, নোড ও Pod-এর ব্যবহার দেখবেন, চাপ তৈরি করে ব্যবহার বাড়তে দেখবেন এবং লগ দিয়ে সমস্যা খুঁজবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

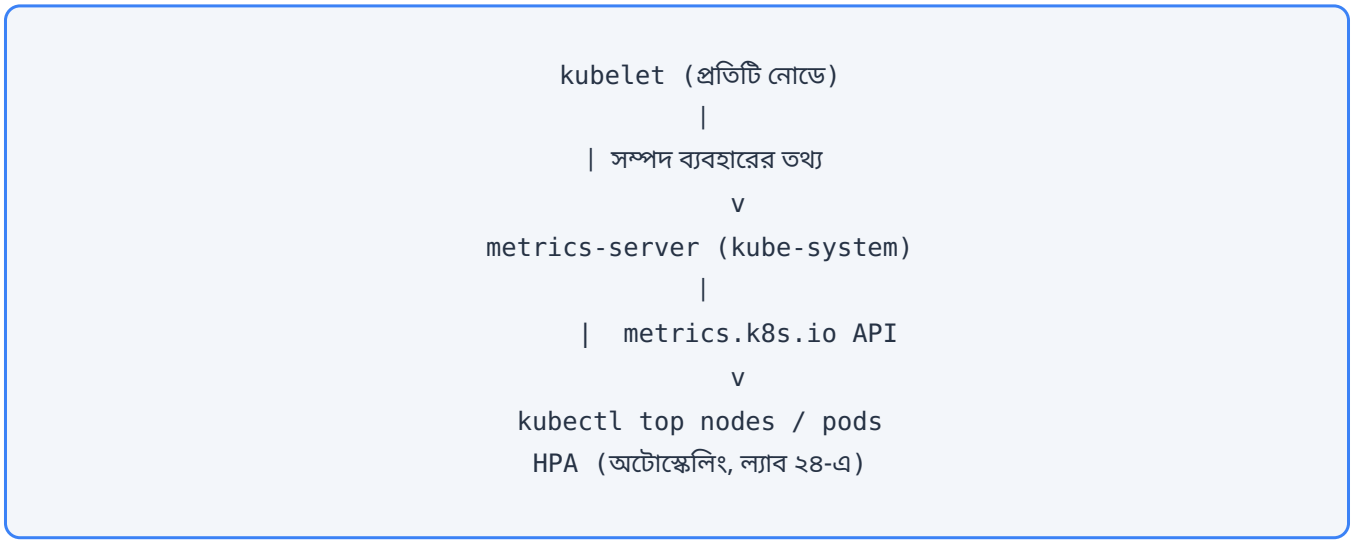
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, metrics-server। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



metrics-server কেবল বর্তমান ব্যবহারের তথ্য দেয়, ইতিহাস সংরক্ষণ করে না। দীর্ঘমেয়াদি ইতিহাস, সতর্কবার্তা ও ড্যাশবোর্ডের জন্য Prometheus ও Grafana দরকার — সেগুলোর ধারণা ক্যাপস্টোনে আছে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস ও লোড তৈরি করুন:** `kubectl create namespace lab20` চালিয়ে একটি nginx Deployment তৈরি করুন। তারপর একটি ব্যস্ত Pod তৈরি করুন যা CPU ব্যবহার করে — যেমন busybox দিয়ে গণনার লুপ চালানো।
2. **metrics-server ইনস্টল করুন:** অফিসিয়াল metrics-server ম্যানিফেস্ট প্রয়োগ করুন। kind-এ TLS যাচাইয়ের জন্য অতিরিক্ত ফ্ল্যাগ (`--kubelet-insecure-tls`) দরকার হতে পারে — অফিসিয়াল নির্দেশনা দেখে নিন। ইনস্টলের পর `kubectl get pods -n kube-system -l k8s-app=metrics-server` দিয়ে Pod `Running` হওয়া পর্যন্ত অপেক্ষা করুন।
3. **নোডের ব্যবহার দেখুন:** `kubectl top nodes` চালান। প্রতিটি নোডের CPU ও মেমরি ব্যবহার (শতাংশসহ) দেখা যাবে। নোডের ব্যবহার বেশি হলে নতুন Pod বসানো কঠিন হয় — এটি স্কেলিং সিদ্ধান্তের প্রথম তথ্য।
4. **Pod-এর ব্যবহার দেখুন:** `kubectl top pods -n lab20` ও `kubectl top pods -A` চালান। কোন Pod কত CPU ও মেমরি ব্যবহার করছে তা দেখুন। ব্যস্ত Pod-এর CPU ব্যবহার বাড়তে দেখা যাওয়া উচিত।
5. **চাপ বাড়িয়ে ব্যবহার দেখুন:** ব্যস্ত Pod-এর সংখ্যা বাড়িয়ে (`kubectl scale`) আবার `kubectl top pods -n lab20` চালান। মোট ব্যবহার কীভাবে বাড়ে তা লক্ষ করুন। এই পর্যবেক্ষণই request ও limit নির্ধারণের ভিত্তি।
6. **লগ দিয়ে সমস্যা খুঁজুন:** `kubectl logs deployment/nginx -n lab20 --tail=50` চালিয়ে সাম্প্রতিক লগ দেখুন। `--tail` ও `-f` (follow) ফ্ল্যাগের ব্যবহার শিখুন। লগে ত্রুটি খুঁজতে `kubectl logs ... | grep -i error` ব্যবহার করুন।
7. **ইভেন্ট দেখুন:** `kubectl get events -n lab20 --sort-by=.lastTimestamp` চালিয়ে সাম্প্রতিক ঘটনাগুলো সময়ের ক্রমে দেখুন। Events-এ Pod তৈরি, স্কেলিং, ব্যর্থতা — সবকিছুর ইতিহাস থাকে।
8. **metrics-server-এর সীমা বুঝুন:** metrics-server কেবল বর্তমান মান দেয় — এক ঘণ্টা আগের ব্যবহার জানতে চাইলে তা দিতে পারে না। সেই প্রয়োজনে Prometheus দরকার। এই সীমাটি মনে রাখুন, যাতে ভুল টুল দিয়ে ভুল প্রশ্ন না করেন।

কনফিগারেশন ফাইল

load-pod.yaml — CPU ব্যবহার করা Pod — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: load-deployment
  namespace: lab20
  labels:
    app: load-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: load-demo
  template:
    metadata:
      labels:
        app: load-demo
    spec:
      containers:
        - name: busybox
          image: busybox:1.36
          command: ["sh", "-c", "while true; do echo $((13*37)); done"]
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
              memory: "128Mi"
              cpu: "500m"
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab20
kubectl apply -f load-pod.yaml
kubectl top nodes
kubectl top pods -n lab20
kubectl top pods -A
kubectl logs deployment/load-deployment -n lab20 --tail=20
kubectl get events -n lab20 --sort-by=.lastTimestamp
kubectl delete -f load-pod.yaml
```

প্রত্যাশিত ফলাফল ও যাচাই

metrics-server Pod `Running` হবে, `kubectl top nodes` ও `kubectl top pods`-এ CPU ও মেমরি ব্যবহার দেখা যাবে, ব্যস্ত Pod-এর CPU ব্যবহার অন্যদের চেয়ে বেশি হবে। নমুনা (illustrative): `NAME CPU(cores) CPU% MEMORY(bytes) — technova-worker 120m 6% 1200Mi 15%`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **kubectl top-এ তথ্য আসছে না:** metrics-server Pod চলছে কি না দেখুন; ইনস্টলের পর মেট্রিক আসতে এক-দুই মিনিট লাগতে পারে।
- **metrics-server Pod চলছে না:** kind-এ TLS ফ্ল্যাগ দরকার হতে পারে — অফিসিয়াল নির্দেশনা অনুযায়ী `--kubelet-insecure-tls` আর্গুমেন্ট যোগ করুন।
- **ব্যবহার শূন্য দেখাচ্ছে:** Pod সত্যিই নিষ্ক্রিয় থাকতে পারে — ব্যস্ত Pod দিয়ে চাপ তৈরি করে আবার দেখুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab20` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev`/`staging`/`prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. metrics-server কী করে? এটি ছাড়া `kubectl top` কেন কাজ করে না?
2. request ও limit নির্ধারণে মনিটরিংয়ের তথ্য কীভাবে ব্যবহার করা হয়?
3. metrics-server ও Prometheus-এর পার্থক্য কী? কখন কোনটি দরকার?

4. একটি Pod-এর CPU ব্যবহার হঠাৎ বেড়ে গেলে প্রথমে কী কী দেখবেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২১ — Helm চার্ট

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Helm — Kubernetes-এর প্যাকেজ ম্যানেজার — দিয়ে অ্যাপ্লিকেশন ইনস্টল, আপগ্রেড ও রোলব্যাক করা শেখা। YAML ফাইল একে একে প্রয়োগ করা ছোট প্রকল্পে চলে, কিন্তু বড় অ্যাপ্লিকেশনে অনেক ফাইল, অনেক পরিবেশ ও অনেক সংস্করণ পরিচালনা করতে হয়। Helm চার্ট হলো টেমপ্লেটসহ YAML-এর একটি প্যাকেজ: একই চার্টে মান (values) বদলে dev, staging ও prod-এ আলাদা কনফিগারেশন দেওয়া যায়। `helm install`, `helm upgrade`, `helm rollback` ও `helm list` দিয়ে পুরো অ্যাপ্লিকেশনকে একটি একক হিসেবে পরিচালনা করা যায়। এই ল্যাবে আপনি একটি চার্ট তৈরি করবেন, মান বদলে ইনস্টল করবেন, আপগ্রেড করবেন এবং রোলব্যাক করবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

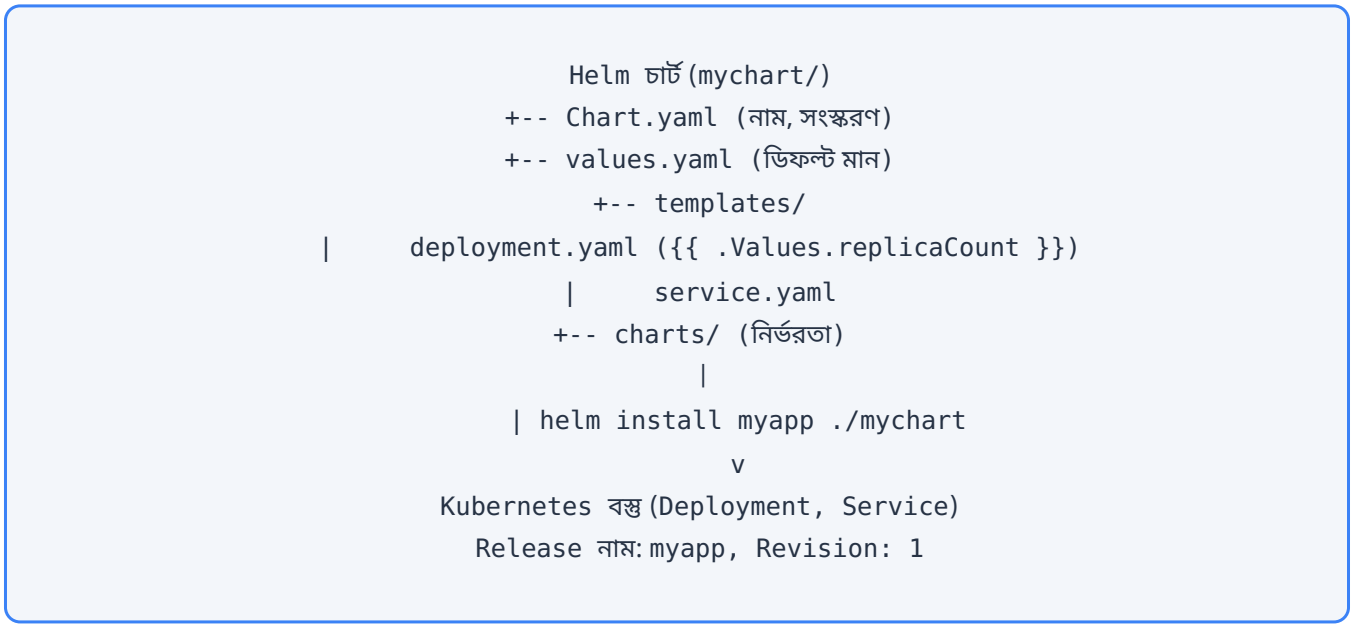
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, Helm। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Helm-এর মূল ধারণা হলো টেমপ্লেট ও মান আলাদা করা। টেমপ্লেটে `{{ .Values.image.tag }}`-এর মতো জায়গায় মান বসে; একই চার্ট ভিন্ন values ফাইল দিয়ে ভিন্ন পরিবেশে চালানো যায়। Release হলো একটি চার্টের একটি ইনস্টল করা সংস্করণ — প্রতিটি আপগ্রেডে নতুন revision তৈরি হয়।

ধাপে ধাপে নির্দেশনা

1. **Helm ইনস্টল যাচাই করুন:** `helm version` চালিয়ে Helm ইনস্টল আছে কি না দেখুন। না থাকলে অফিসিয়াল নির্দেশনা অনুযায়ী ইনস্টল করুন।
2. **একটি চার্ট তৈরি করুন:** `helm create technova-app` চালান। তৈরি হওয়া ডিরেক্টরি ঘুরে দেখুন: `Chart.yaml`, `values.yaml`, `templates/`। `values.yaml`-এ replicaCount, image ও service-এর ডিফল্ট মান দেখুন।
3. **চার্টের টেমপ্লেট পড়ুন:** `templates/deployment.yaml` খুলে দেখুন কীভাবে `{{ .Values.replicaCount }}` ও `{{ .Values.image.repository }}` ব্যবহার করা হয়েছে। টেমপ্লেট রেন্ডার করে দেখুন: `helm template technova-app ./technova-app` — কোনো ক্লাস্টার ছাড়াই চূড়ান্ত YAML দেখা যাবে।
4. **চার্ট ইনস্টল করুন:** `kubectl create namespace lab21` চালিয়ে `helm install myapp ./technova-app -n lab21` চালান। `helm list -n lab21` ও `kubectl get pods -n lab21` দিয়ে দেখুন Release তৈরি হয়েছে ও Pod চলছে।
5. **মান বদলে আপগ্রেড করুন:** `helm upgrade myapp ./technova-app -n lab21 --set replicaCount=3 --set image.tag=1.28` চালান। `helm history myapp -n lab21` দেখুন — revision 2 তৈরি হয়েছে। `kubectl get pods -n lab21` দিয়ে Replica বেড়েছে কি না দেখুন।
6. **values ফাইল দিয়ে আপগ্রেড করুন:** একটি `values-prod.yaml` ফাইল লিখুন (replicaCount: 2) এবং `helm upgrade myapp ./technova-app -n lab21 -f values-prod.yaml` চালান। কমান্ড-লাইনের `--set` দ্রুত পরীক্ষার জন্য, ফাইল দীর্ঘস্থায়ী কনফিগারেশনের জন্য ভালো।
7. **রোলব্যাক করুন:** `helm rollback myapp 1 -n lab21` চালিয়ে প্রথম revision-এ ফিরুন। `helm history myapp -n lab21` ও `get pods` দিয়ে যাচাই করুন। Helm-এর rollback Deployment-এর rollback-এর মতোই কাজ করে, তবে পুরো Release-এর জন্য।

8. **Release মুছে ফেলুন:** `helm uninstall myapp -n lab21` দিয়ে Release মুছে ফেলুন। `kubectl get all -n lab21` দিয়ে দেখুন সব বস্তু চলে গেছে।

কনফিগারেশন ফাইল

values-prod.yaml — প্রোডাকশন-ধাঁচের মান — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
replicaCount: 2
image:
  repository: nginx
  pullPolicy: IfNotPresent
  tag: "1.27"
service:
  type: ClusterIP
  port: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
helm version
helm create technova-app
helm template technova-app ./technova-app
kubectl create namespace lab21
helm install myapp ./technova-app -n lab21
helm list -n lab21
helm upgrade myapp ./technova-app -n lab21 --set replicaCount=3
helm history myapp -n lab21
helm rollback myapp 1 -n lab21
helm uninstall myapp -n lab21
```

প্রত্যাশিত ফলাফল ও যাচাই

চার্ট তৈরি হবে, `helm template`-এ চূড়ান্ত YAML দেখা যাবে, ইনস্টলের পর Pod চলবে, আপগ্রেডে revision বাড়বে ও Replica বদলাবে, rollback-এ প্রথম অবস্থায় ফেরত যাবে। নমুনা (illustrative): `NAME NAMESPACE REVISION STATUS – myapp lab21 2 deployed`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **helm install-এ ত্রুটি আসছে:** চার্টের নাম ও পথ ঠিক আছে কি না দেখুন; `helm lint ./technova-app` দিয়ে চার্ট যাচাই করুন।
- **আপগ্রেডে কিছু বদলাচ্ছে না:** মানটি সঠিক কী (key) দিয়ে বদলাচ্ছেন কি না দেখুন — `helm get values myapp -n lab21` দিয়ে বর্তমান মান দেখুন।
- **rollback-এর পরও নতুন মান দেখাচ্ছে:** সঠিক revision নম্বর দিয়েছেন কি না `helm history` দিয়ে যাচাই করুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab21` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Helm চার্টের তিনটি মূল অংশ (Chart.yaml, values.yaml, templates) কী কী করে?
2. `--set` ও `-f values.yaml` — কখন কোনটি ব্যবহার করবেন?
3. Helm Release ও revision-এর ধারণা কী? rollback কীভাবে কাজ করে?
4. সরাসরি YAML প্রয়োগ করার চেয়ে Helm ব্যবহারের সুবিধা কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২২ — Kustomize ওভারলে

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kustomize — পরিবেশভিত্তিক YAML পরিচালনার টুল — দিয়ে একই বেস কনফিগারেশন থেকে dev, staging ও prod-এর আলাদা সংস্করণ তৈরি করা শেখা। Helm টেমপ্লেট ব্যবহার করে, Kustomize করে না — এটি বেস YAML-এর ওপরে প্যাচ, নামের উপসর্গ, Label ও ConfigMap জেনারেটর প্রয়োগ করে। একই Deployment-এর dev সংস্করণে একটি Replica ও staging সংস্করণে তিনটি Replica — এই পার্থক্য Kustomize ওভারলে দিয়ে প্রকাশ করা যায়, মূল ফাইল না বদলে। kubectl-এর ভেতরে Kustomize অন্তর্ভুক্ত আছে (`kubectl apply -k`), তাই আলাদা টুল না থাকলেও এই ল্যাব করা যায়। এই ল্যাবে আপনি একটি বেস তৈরি করবেন, দুটি ওভারলে লিখবেন এবং প্রতিটি পরিবেশের চূড়ান্ত YAML দেখবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, kustomize। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



Kustomize-এর মূল নীতি: বেস ফাইল কখনো বদলাবেন না, কেবল ওভারলে যোগ করবেন। এতে বোঝা যায় কোন অংশ সবার জন্য এক, আর কোন অংশ পরিবেশভিত্তিক।

ধাপে ধাপে নির্দেশনা

1. **কাজের ডিরেক্টরি তৈরি করুন:** `mkdir -p kustomize-demo/base kustomize-demo/overlays/dev kustomize-demo/overlays/staging` চালান। এই ল্যাবের সব ফাইল এই ডিরেক্টরিতে থাকবে।
2. **বেস লিখুন:** base ডিরেক্টরিতে একটি Deployment, একটি Service ও একটি `kustomization.yaml` লিখুন। বেসে Replica ১টি, ইমেজ nginx:1.27, Namespace নির্ধারিত নয় — ওভারলে তা নির্ধারণ করবে। নিচের YAML-এ বেসের উদাহরণ দেওয়া হয়েছে।
3. **dev ওভারলে লিখুন:** overlays/dev-এ একটি `kustomization.yaml` লিখুন যা বেসকে নির্দেশ করে, Namespace `dev` নির্ধারণ করে ও `namePrefix: dev-` যোগ করে। `kubectl kustomize overlays/dev` চালিয়ে চূড়ান্ত YAML দেখুন।
4. **staging ওভারলে লিখুন:** overlays/staging-এ একটি `kustomization.yaml` লিখুন যা Replica ৩টি করে, Namespace `staging` নির্ধারণ করে ও ইমেজের ট্যাগ 1.28 করে। `kubectl kustomize overlays/staging` দিয়ে দেখুন Replica ও ট্যাগ বদলে গেছে।
5. **dev প্রয়োগ করুন:** `kubectl create namespace dev` চালিয়ে `kubectl apply -k overlays/dev` চালান। `kubectl get pods -n dev` দিয়ে দেখুন একটি Pod চলছে।
6. **staging প্রয়োগ করুন:** `kubectl create namespace staging` চালিয়ে `kubectl apply -k overlays/staging` চালান। `kubectl get pods -n staging` দিয়ে দেখুন তিনটি Pod চলছে। একই বেস থেকে দুটি ভিন্ন পরিবেশ তৈরি হলো।
7. **ConfigMap জেনারেটর ব্যবহার করুন:** ওভারলেতে `configMapGenerator` দিয়ে পরিবেশভিত্তিক ConfigMap তৈরি করুন — dev-এ APP_ENV=dev, staging-এ APP_ENV=staging। `kubectl kustomize` দিয়ে দেখুন

ConfigMap-এর নামের শেষে হ্যাশ যোগ হয়েছে — মান বদলালে নতুন ConfigMap তৈরি হয়, পুরোনো Pod আপডেটের সংকেত পায়।

8. **পরিষ্কার করুন:** `kubectl delete -k overlays/dev` ও `kubectl delete -k overlays/staging` দিয়ে দুটি পরিবেশ মুছে ফেলুন।

কনফিগারেশন ফাইল

kustomization.yaml — বেস ও staging ওভারলে — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  app: web
---
# overlays/staging/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: staging
namePrefix: staging-
resources:
  - ../../base
replicas:
  - name: web-deployment
    count: 3
images:
  - name: nginx
    newTag: "1.28"
configMapGenerator:
  - name: app-config
    literals:
      - APP_ENV=staging
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
mkdir -p kustomize-demo/base kustomize-demo/overlays/dev kustomize-demo/overlays/staging
kubectl kustomize overlays/dev
kubectl kustomize overlays/staging
kubectl apply -k overlays/dev
kubectl apply -k overlays/staging
kubectl get pods -n dev
kubectl get pods -n staging
kubectl delete -k overlays/dev
kubectl delete -k overlays/staging
```

প্রত্যাশিত ফলাফল ও যাচাই

dev ওভারলেতে একটি Replica, staging ওভারলেতে তিনটি Replica ও nginx:1.28 দেখা যাবে; প্রতিটি পরিবেশের Pod নিজের Namespace-এ চলবে। `kubectl kustomize`-এ চূড়ান্ত YAML-এ ওভারলের পরিবর্তনগুলো স্পষ্ট দেখা যাবে। নমুনা (illustrative): staging-এ `replicas: 3` ও `image: nginx:1.28`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **kustomize-এ ত্রুটি আসছে:** kustomization.yaml-এর `resources`-এর পথ ঠিক আছে কি না দেখুন; আপেক্ষিক পথ বেস থেকে গণনা করা হয়।
- **ওভারলের পরিবর্তন দেখা যাচ্ছে না:** `kubectl kustomize` দিয়ে রেন্ডার করা YAML দেখুন — প্রয়োগ করার আগে রেন্ডার দেখাই সঠিক যাচাই।
- **Namespace তৈরি হয়নি:** Kustomize Namespace তৈরি করে না — আগে `kubectl create namespace` চালাতে হয়।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab22` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev`/`staging`/`prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Kustomize ও Helm-এর পার্থক্য কী? কখন কোনটি বেছে নেবেন?
2. বেস ও ওভারলের ধারণা কী? বেস ফাইল কেন বদলানো উচিত নয়?

3. namePrefix ও commonLabels কী কাজে লাগে?
4. ConfigMap জেনারেটরের নামের শেষে হ্যাশ কেন যোগ হয়?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২৩ — ব্যাকআপ ও পুনরুদ্ধারের ধারণা

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য Kubernetes-এ ব্যাকআপ ও পুনরুদ্ধার (disaster recovery)-এর ধারণা ও অনুশীলন শেখা — কোনো ক্লাউড খরচ ছাড়াই, লোকাল ক্লাস্টারে। Kubernetes-এ দুটি জিনিস ব্যাকআপ করতে হয়: ক্লাস্টারের অবস্থা (etcd-তে সংরক্ষিত সব বস্তু — Deployment, Service, ConfigMap, Secret) এবং অ্যাপ্লিকেশনের ডেটা (PersistentVolume-এর ডেটা)। etcd-র ব্যাকআপ নিলে পুরো ক্লাস্টারের কনফিগারেশন ফিরিয়ে আনা যায়, আর PV-র ব্যাকআপ নিলে ডাটাবেসের ডেটা ফিরে পাওয়া যায়। এই ল্যাবে আপনি YAML এক্সপোর্ট করে কনফিগারেশন ব্যাকআপ নেবেন, etcd স্ন্যাপশটের ধারণা শিখবেন, একটি Namespace মুছে তা আবার ফিরিয়ে আনবেন এবং Velero-র মতো টুলের ধারণা নেবেন।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

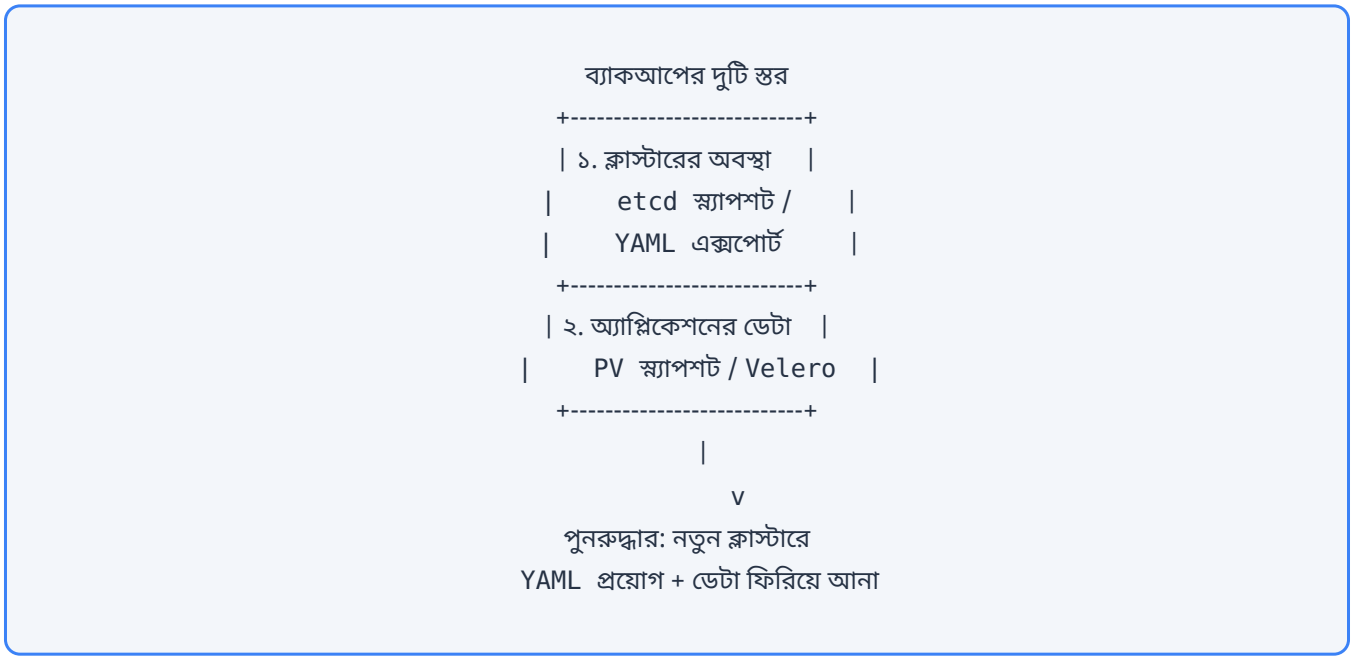
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



ব্যাকআপ নেওয়া হয়নি এমন পুনরুদ্ধার পরীক্ষা করা হয়নি — এই দুটি একসঙ্গে না করলে ব্যাকআপের কোনো মূল্য নেই। ল্যাব পরিবেশে ব্যাকআপের অনুশীলন করাই এই ল্যাবের উদ্দেশ্য, যাতে প্রোডাকশনে প্রথমবার না করতে হয়।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস ও কিছু বস্তু তৈরি করুন:** `kubectl create namespace lab23` চালিয়ে একটি Deployment, একটি Service ও একটি ConfigMap তৈরি করুন। এগুলোই ব্যাকআপের বিষয়বস্তু হবে।
2. **YAML এক্সপোর্ট করে ব্যাকআপ নিন:** `kubectl get all -n lab23 -o yaml > backup-lab23.yaml` চালিয়ে পুরো Namespace-এর বস্তু এক্সপোর্ট করুন। আলাদাভাবে `kubectl get configmap app-config -n lab23 -o yaml > backup-configmap.yaml` চালান। ফাইলগুলো খুলে দেখুন — এগুলোই আপনার ব্যাকআপ।
3. **etcd স্ন্যাপশটের ধারণা শিখুন:** kubeadm দিয়ে তৈরি ক্লাস্টারে `etcdctl snapshot save` দিয়ে etcd-র স্ন্যাপশট নেওয়া যায়। kind-এ etcd কন্টেইনারের ভেতরে চলে, তাই সরাসরি স্ন্যাপশট নেওয়া জটিল — এই ল্যাবে কেবল ধারণাটি শিখুন: স্ন্যাপশট নেওয়ার আগে API সার্ভার থামাতে হয় না, তবে স্ন্যাপশট নিয়মিত নিতে হয়।
4. **Velero-র ধারণা নিন:** Velero একটি জনপ্রিয় ব্যাকআপ টুল, যা ক্লাস্টারের বস্তু ও PV-র স্ন্যাপশট একসঙ্গে নেয় এবং অন্য ক্লাস্টারে পুনরুদ্ধার করতে পারে। এই ল্যাবে Velero ইনস্টল করবেন না — কারণ এতে অবজেক্ট স্টোরেজ দরকার হয়, যা লোকাল পরিবেশে অতিরিক্ত — তবে এর ধারণা ও কমান্ড (`velero backup create`, `velero restore create`) মনে রাখুন।
5. **Namespace মুছে বিপর্যয় তৈরি করুন:** `kubectl delete namespace lab23` চালিয়ে সব মুছে ফেলুন। `kubectl get all -n lab23` দিয়ে নিশ্চিত হোন কিছুই নেই। এটিই অনুশীলনের বিপর্যয়।
6. **ব্যাকআপ থেকে পুনরুদ্ধার করুন:** `kubectl create namespace lab23` চালিয়ে `kubectl apply -f backup-lab23.yaml` দিয়ে বস্তুগুলো ফিরিয়ে আনুন। `kubectl get all -n lab23` দিয়ে দেখুন Deployment, Service ও ConfigMap ফিরে এসেছে কি না।
7. **PV ডেটার ব্যাকআপ পরীক্ষা করুন:** ল্যাব ১৪-এর মতো একটি PVC-তে ফাইল লিখে সেই ফাইলটি হোস্টে কপি করে রাখুন (`kubectl cp`), তারপর Pod মুছে আবার তৈরি করে ফাইলটি ফিরিয়ে আনুন। এটি ছোট আকারে ডেটা ব্যাকআপ ও পুনরুদ্ধারের অনুশীলন।

8. **পুনরুদ্ধার পরীক্ষার নোট লিখুন:** আপনার নোটে লিখুন: ব্যাকআপ কখন নেওয়া হয়েছিল, পুনরুদ্ধারে কত সময় লেগেছে, কী কী ফিরে এসেছে, কী হারিয়েছে। প্রোডাকশনে এই নোটই RTO (কত সময়ে ফিরতে হবে) ও RPO (কতটুকু ডেটা হারানো চলবে) নির্ধারণে সাহায্য করে।

কনফিগারেশন ফাইল

backup-lab23.yaml — এক্সপোর্ট করা ব্যাকআপ (উদাহরণ) — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
# এই ফাইলটি হাতে লেখা নয় — kubectl get -o yaml দিয়ে তৈরি।
# উদাহরণ অংশ:
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: lab23
data:
  APP_NAME: technova
  APP_ENV: dev
---
# ব্যাকআপ কমান্ড:
# kubectl get all -n lab23 -o yaml > backup-lab23.yaml
# kubectl get configmap app-config -n lab23 -o yaml > backup-configmap.yaml
# kubectl get secret db-secret -n lab23 -o yaml > backup-secret.yaml
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace lab23
kubectl get all -n lab23 -o yaml > backup-lab23.yaml
kubectl get configmap app-config -n lab23 -o yaml > backup-configmap.yaml
kubectl delete namespace lab23
kubectl create namespace lab23
kubectl apply -f backup-lab23.yaml
kubectl get all -n lab23
kubectl delete namespace lab23
```

প্রত্যাশিত ফলাফল ও যাচাই

ব্যাকআপ ফাইলে Namespace-এর সব বস্তুর YAML থাকবে, Namespace মুছে ফেলার পর কিছু থাকবে না, আর ব্যাকআপ থেকে প্রয়োগ করার পর Deployment, Service ও ConfigMap আবার ফিরে আসবে। নমুনা (illustrative): পুনরুদ্ধারের পর `kubectl get deployments -n lab23`-এ Deployment আবার দেখা যাবে।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **ব্যাকআপ ফাইলে Secret-এর মান দেখা যাচ্ছে:** Secret এক্সপোর্ট করলে মান base64-এ থাকে — ফাইলটি নিরাপদে রাখুন, Git-এ কমিট করবেন না।
- **পুনরুদ্ধারের পর Pod চলছে না:** ব্যাকআপে Pod-এর IP বা নোডের তথ্য থাকতে পারে — Deployment, Service, ConfigMap-এর মতো ঘোষণামূলক বস্তু ব্যাকআপ করাই সঠিক; Pod সরাসরি ব্যাকআপ করবেন না।
- **PV-র ডেটা ফিরে আসেনি:** YAML ব্যাকআপে কেবল কনফিগারেশন থাকে, ডেটা নয় — ডেটার জন্য আলাদা PV ব্যাকআপ দরকার।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab23` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. Kubernetes-এ কী কী ব্যাকআপ করতে হয়? কেবল YAML ব্যাকআপ যথেষ্ট নয় কেন?
2. etcd স্ন্যাপশট ও YAML এক্সপোর্ট — পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?
3. RTO ও RPO-এর অর্থ কী? ব্যাকআপ পরিকল্পনায় এগুলো কীভাবে ব্যবহার করা হয়?
4. ব্যাকআপ নেওয়া হয়েছে কিন্তু পুনরুদ্ধার পরীক্ষা করা হয়নি — এতে ঝুঁকি কী?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ল্যাব ২৪ — সম্পূর্ণ MERN অ্যাপ্লিকেশন ডেপ্লয়মেন্ট

উদ্দেশ্য

এই ল্যাবের উদ্দেশ্য আগের সব ল্যাবের জ্ঞান একত্র করে একটি সম্পূর্ণ MERN অ্যাপ্লিকেশন — MongoDB, Express, React, Node.js — Kubernetes-এ ডেপ্লয় করা। TechNova-র উদাহরণ অ্যাপ্লিকেশনে তিনটি স্তর আছে: React frontend, Express/Node.js backend API, আর MongoDB ডাটাবেস। frontend ও backend Deployment হিসেবে চলবে, MongoDB StatefulSet বা Deployment হিসেবে PVC-সহ চলবে, backend-এর সঙ্গে ডাটাবেসের সংযোগ ConfigMap ও Secret দিয়ে হবে, frontend ও backend-এর সামনে Service থাকবে, আর Ingress দিয়ে বাইরে থেকে ট্রাফিক আসবে। এই ল্যাবটি বইয়ের সবচেয়ে বড় ল্যাব — ধাপে ধাপে, যাচাই করে করে এগোবেন। এই ল্যাব শেষ করলে আপনি প্রজেক্ট ১ ও ক্যাপস্টোনের জন্য প্রস্তুত।

> **LAB:** এই ল্যাবটি শেষ করার পর আপনি নিজে এই কাজটি করতে পারবেন — বইয়ের ব্যাখ্যা না দেখে, কেবল নিজের নোট ও `kubectl --help` ব্যবহার করে। যদি কোনো ধাপে আটকে যান, তবে আগের ধাপের যাচাইটি আবার চালান; বেশিরভাগ ভুল আগের ধাপেই লুকিয়ে থাকে।

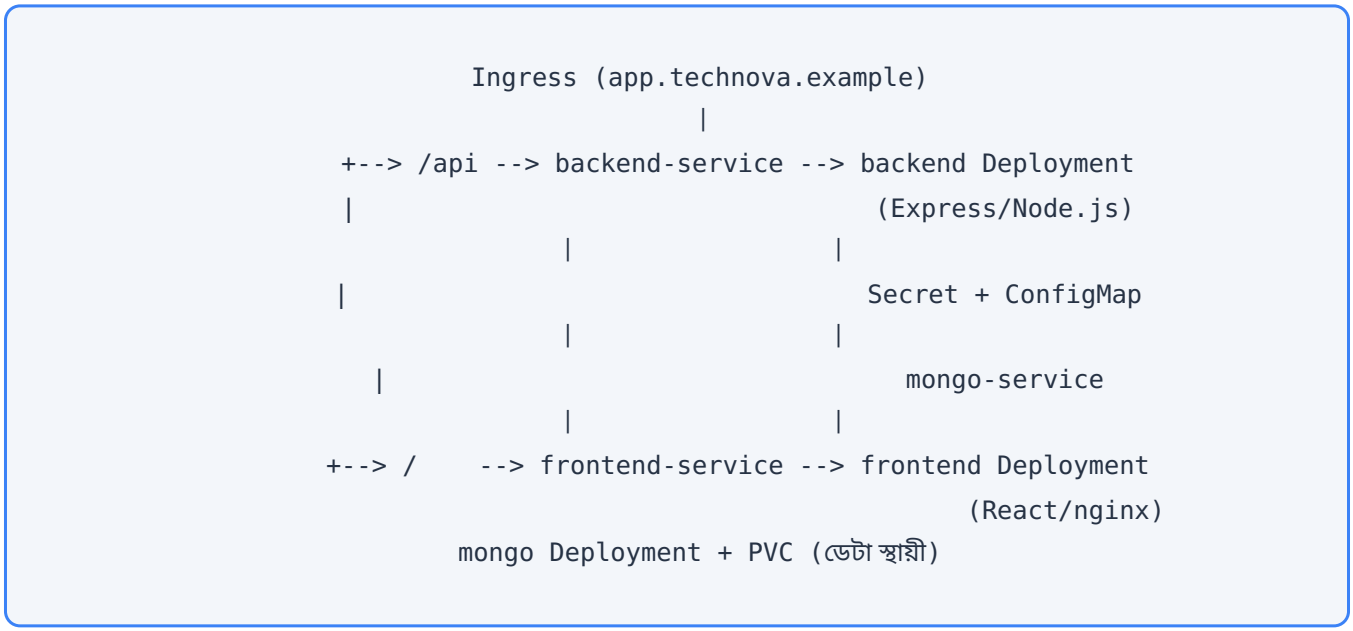
প্রয়োজনীয় টুলস

এই ল্যাবের জন্য প্রয়োজন: kind, kubectl, Helm, Docker। এছাড়া একটি টেক্সট এডিটর ও টার্মিনাল প্রয়োজন। সব কাজ লোকাল ক্লাস্টারে (kind/Minikube/k3d) করুন — কোনো ক্লাউড ধাপ নেই, তাই কোনো খরচ হবে না। যদি কোনো ধাপে ক্লাউড সার্ভিসের নাম আসে, তবে তা কেবল ধারণা বোঝানোর জন্য; সেই ধাপটি লোকাল পরিবেশে অনুশীলন করুন।

পূর্বশর্ত

এই ল্যাব শুরু করার আগে আগের ল্যাবগুলো শেষ করুন, বিশেষ করে ল্যাব ১-এর ক্লাস্টার সেটআপ ও ল্যাব ২-এর কনটেক্সট যাচাই। `kubectl cluster-info` ও `kubectl config current-context` চালিয়ে নিশ্চিত হোন আপনি লোকাল ক্লাস্টারে আছেন। আগের ল্যাবের Namespace মুছে ফেলা থাকলে এই ল্যাব পরিষ্কার পরিবেশে শুরু হবে। YAML ফাইলের ইন্ডেন্টেশন (স্পেস) ঠিক রাখুন — ট্যাব ব্যবহার করবেন না।

আর্কিটেকচার



এই আর্কিটেকচারে প্রতিটি স্তর আলাদা — frontend বদলালে backend থামে না, backend বদলালে ডাটাবেস থামে না। Service-এর নাম দিয়ে স্তরগুলো একে অপরকে ডাকে, আর Secret-এ ডাটাবেসের পাসওয়ার্ড থাকে।

ধাপে ধাপে নির্দেশনা

1. **কাজের নেমস্পেস তৈরি করুন:** `kubectl create namespace technova` চালান। এই ল্যাবের সব বস্তু এই Namespace-এ থাকবে।
2. **MongoDB-র Secret ও PVC তৈরি করুন:** নিচের YAML-এ MongoDB-র জন্য Secret (ব্যবহারকারী নাম ও পাসওয়ার্ড — CHANGE-ME-... প্লেসহোল্ডার) ও PVC আছে। `kubectl apply -f mongo.yaml` চালান। ডাটাবেসের পাসওয়ার্ড কখনো YAML-এ আসল মান দিয়ে লিখবেন না।
3. **MongoDB Deployment তৈরি করুন:** একই YAML-এ MongoDB Deployment আছে, যা PVC মাউন্ট করে ও Secret থেকে পাসওয়ার্ড নেয়। `kubectl get pods -n technova` দিয়ে MongoDB Pod `Running` হওয়া পর্যন্ত অপেক্ষা করুন — ডাটাবেস চালু হতে সময় লাগে।
4. **backend API Deployment তৈরি করুন:** নিচের দ্বিতীয় YAML-এ backend Deployment আছে: ইমেজ `technova/api:1.0.0` (উদাহরণ ইমেজ — আপনার নিজের ইমেজ দিয়ে বদলান), পরিবেশ চলকে MongoDB-র ঠিকানা (`mongo-service`) ও Secret থেকে পাসওয়ার্ড, liveness ও readiness প্রোব, আর request/limit। `kubectl apply -f backend.yaml` চালান।
5. **backend Service তৈরি করুন:** backend-এর সামনে একটি ClusterIP Service তৈরি করুন। `kubectl get endpoints backend-service -n technova` দিয়ে Endpoints দেখুন।
6. **frontend Deployment ও Service তৈরি করুন:** React অ্যাপ্লিকেশন (nginx দিয়ে পরিবেশিত) Deployment হিসেবে তৈরি করুন ও তার সামনে Service বসান। frontend-এর পরিবেশ চলকে backend-এর ঠিকানা দিন — যেমন `REACT_APP_API_URL=http://backend-service` অথবা Ingress-এর মাধ্যমে `/api`।
7. **Ingress তৈরি করুন:** ল্যাব ১৩-এর মতো Ingress তৈরি করুন: `/` পথে frontend, `/api` পথে backend। `kubectl get ingress -n technova` দিয়ে ADDRESS দেখুন।

8. **সম্পূর্ণ অ্যাপ্লিকেশন পরীক্ষা করুন:** port-forward বা Ingress দিয়ে frontend খুলুন, backend-এর `/api/health` পথে অনুরোধ পাঠান, আর MongoDB-তে একটি নথি তৈরি করে দেখুন তা থেকে যাচ্ছে কি না। প্রতিটি স্তর আলাদাভাবে যাচাই করুন — একসঙ্গে সব পরীক্ষা করবেন না।
9. **স্কেল ও রোলিং আপডেট পরীক্ষা করুন:** backend-কে তিনটি Replica-তে স্কেল করুন, তারপর ইমেজের ট্যাগ বদলে rolling update করুন। আপডেটের সময় frontend থেকে অনুরোধ পাঠিয়ে দেখুন সার্ভিস বন্ধ হচ্ছে কি না।
10. **পরিষ্কার করুন:** `kubectl delete namespace technova` দিয়ে পুরো অ্যাপ্লিকেশন মুছে ফেলুন। ডেটা মুছে যাবে — তাই পরিষ্কার করার আগে গুরুত্বপূর্ণ ডেটা ব্যাকআপ নিন (ল্যাব ২৩)।

কনফিগারেশন ফাইল

mongo.yaml — MongoDB Secret, PVC ও Deployment — ফাইলটি সংরক্ষণ করার আগে প্রতিটি ফিল্ড পড়ুন। `apiVersion` ও `kind` ঠিক আছে কি না, Namespace-এর নাম ঠিক আছে কি না, আর কোনো আসল পাসওয়ার্ড বা গোপন মান নেই কি না — তা যাচাই করুন। সব গোপন মানে `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করা হয়েছে।

```
apiVersion: v1
kind: Secret
metadata:
  name: mongo-secret
  namespace: technova
type: Opaque
stringData:
  MONGO_USER: technova_user
  MONGO_PASSWORD: CHANGE-ME-mongo-password
  MONGO_ROOT_PASSWORD: CHANGE-ME-mongo-root-password
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
  namespace: technova
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
  namespace: technova
  labels:
    app: mongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
```

```
      containers:
        - name: mongo
          image: mongo:7.0
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: MONGO_USER
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: MONGO_ROOT_PASSWORD
          volumeMounts:
            - name: mongo-data
              mountPath: /data/db
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
      volumes:
        - name: mongo-data
          persistentVolumeClaim:
            claimName: mongo-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
  namespace: technova
  labels:
    app: mongo
spec:
  type: ClusterIP
  selector:
    app: mongo
  ports:
    - port: 27017
      targetPort: 27017
```

অতিরিক্ত কনফিগারেশন — backend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: technova
  labels:
    app: backend
spec:
  replicas: 2
```

```
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: api
          image: technova/api:1.0.0
          ports:
            - containerPort: 3000
          env:
            - name: MONGO_URI
              value: mongodb://technova_user:CHANGE-ME-mongo-password@mongo-service:27017/
technova?authSource=admin
            - name: NODE_ENV
              value: production
            - name: PORT
              value: "3000"
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: technova
  labels:
    app: backend
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 3000
```

অতিরিক্ত কনফিগারেশন — frontend.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
  namespace: technova
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: web
          image: technova/frontend:1.0.0
          ports:
            - containerPort: 80
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "250m"
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
  namespace: technova
  labels:
    app: frontend
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: technova-ingress
  namespace: technova
spec:
  ingressClassName: nginx
  rules:
    - host: app.technova.example
      http:
        paths:
```

```
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

কমান্ড

নিচের কমান্ডগুলো ক্রমানুসারে চালান। প্রতিটি কমান্ডের আউটপুট পড়ুন — কোনো ত্রুটি দেখলে এগোবেন না, Troubleshooting অংশটি দেখুন।

```
kubectl create namespace technova
kubectl apply -f mongo.yaml
kubectl get pods -n technova
kubectl apply -f backend.yaml
kubectl apply -f frontend.yaml
kubectl apply -f ingress.yaml
kubectl get deployments -n technova
kubectl get services -n technova
kubectl get ingress -n technova
kubectl delete namespace technova
```

প্রত্যাশিত ফলাফল ও যাচাই

MongoDB, backend ও frontend — তিনটি Pod `Running` হবে; Service-গুলোর Endpoints থাকবে; Ingress-এ ADDRESS দেখা যাবে; frontend থেকে backend-এর স্বাস্থ্য পরীক্ষা সফল হবে। নমুনা (illustrative): `backend-service ClusterIP 10.96.x.x 80/TCP – frontend-service ClusterIP 10.96.x.x 80/TCP`।

যাচাইয়ের সাধারণ নিয়ম: `kubectl get pods`-এ STATUS `Running` ও READY `1/1` (বা Replica সংখ্যা অনুযায়ী) হওয়া উচিত; `kubectl describe`-এ কোনো Warning ইভেন্ট থাকা উচিত নয়; আর `kubectl logs`-এ অ্যাপ্লিকেশনের স্বাভাবিক স্টার্টআপ বার্তা দেখা যাওয়া উচিত। এই তিনটি যাচাই না করে পরের ল্যাবে যাবেন না।

সমস্যা সমাধান (Troubleshooting)

নিচের সমস্যাগুলো এই ল্যাবে সবচেয়ে বেশি দেখা যায়। আপনার সমস্যাটি তালিকায় না থাকলে `kubectl describe` ও `kubectl logs` দিয়ে শুরু করুন — বেশিরভাগ ক্ষেত্রে উত্তর সেখানেই আছে।

- **backend MongoDB-তে সংযোগ করতে পারছে না:** Secret-এর নাম, MongoDB-র Service নাম (`mongo-service`) ও পোর্ট মিলছে কি না দেখুন; MongoDB Pod চলছে কি না যাচাই করুন।
- **frontend backend-কে খুঁজে পাচ্ছে না:** frontend-এর পরিবেশ চলকে backend-এর ঠিকানা ঠিক আছে কি না দেখুন; Ingress-এর `/api` নিয়ম যাচাই করুন।

- **MongoDB Pod বারবার restart হচ্ছে:** PVC বাঁধা হয়েছে কি না ও মেমরি limit যথেষ্ট কি না দেখুন; `describe pod` ও লগ পড়ুন।

> **TROUBLE:** কোনো ধাপে আটকে গেলে প্রথমে যাচাই করুন আপনি সঠিক Namespace-এ আছেন কি না (`kubectl config view --minify`), সঠিক কনটেক্সটে আছেন কি না, আর YAML ফাইলের নাম ও Namespace ঠিক আছে কি না। বেশিরভাগ ল্যাব-ভুল এই তিনটির একটিতে হয়।

পরিষ্কার করা (Cleanup)

ল্যাব শেষে এই ল্যাবের সব বস্তু মুছে ফেলুন, যাতে পরের ল্যাবে পুরোনো বস্তু বিভ্রান্তি তৈরি না করে। সাধারণত Namespace মুছে ফেলাই যথেষ্ট: `kubectl delete namespace lab24` (ল্যাব ২৪-এ `technova`, ল্যাব ৭-এ `dev` / `staging` / `prod`)। kind ক্লাস্টারটি রেখে দিন — পরের ল্যাবগুলো একই ক্লাস্টারে চলবে। যদি ক্লাস্টারটি এলোমেলো হয়ে যায়, তবে `kind delete cluster --name technova` দিয়ে মুছে আবার তৈরি করুন; ল্যাব পরিবেশ ফেলে দেওয়া যায় — এটিই এর সুবিধা।

> **WARNING:** Cleanup-এর সময় Namespace-এর নাম দুবার যাচাই করুন। ভুল Namespace মুছলে সেই Namespace-এর সব Deployment, Service, ConfigMap, Secret ও PVC একসঙ্গে মুছে যাবে। কখনো `kube-system` বা `default` Namespace মুছবেন না।

স্ব-মূল্যায়নের প্রশ্ন

1. MERN-এর তিনটি স্তর কীভাবে একে অপরের সঙ্গে কথা বলে? প্রতিটি স্তরের সামনে Service কেন দরকার?
2. MongoDB-র জন্য Deployment না StatefulSet — কখন কোনটি বেছে নেবেন?
3. backend-এর পরিবেশ চলকে ডাটাবেসের পাসওয়ার্ড সরাসরি লেখা কেন ভুল?
4. এই অ্যাপ্লিকেশনে কোন স্তরটি প্রথমে তৈরি করতে হয়, আর কেন?

প্রতিটি প্রশ্নের উত্তর মুখে বা লিখে দিন। দুটির বেশি প্রশ্নের উত্তর দিতে না পারলে ল্যাবটি আবার করুন অথবা সংশ্লিষ্ট অধ্যায়টি আবার পড়ুন — এগিয়ে যাওয়ার আগে ভিত্তি মজবুত করুন।

ব্যবহারিক অ্যাসাইনমেন্ট — ১৫টি

নিচের ১৫টি অ্যাসাইনমেন্ট ল্যাবগুলোর জ্ঞানকে স্বাধীন কাজে রূপান্তর করে। প্রতিটি অ্যাসাইনমেন্ট একা করুন, নিজের ক্লাস্টারে করুন, আর ফলাফল Git রিপোজিটরিতে সংরক্ষণ করুন। প্রতিটি অ্যাসাইনমেন্টের জন্য উদ্দেশ্য, জমা দেওয়ার বিষয় (deliverable) ও মূল্যায়নের ইঙ্গিত দেওয়া হয়েছে। কোনো অ্যাসাইনমেন্টে আসল সিক্রেট ব্যবহার করবেন না — কেবল `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন।

1. **অ্যাসাইনমেন্ট ১ — নিজের ক্লাস্টার ডকুমেন্টেশন।** উদ্দেশ্য: kind দিয়ে একটি ক্লাস্টার তৈরি করে তার আর্কিটেকচার, নোড, সংস্করণ ও কনটেক্সট ডকুমেন্ট করা। ডেলিভারেবল: একটি README ফাইল, যাতে ক্লাস্টার তৈরির কমান্ড, `kubectl get nodes` ও `kubectl get pods -A`-এর আউটপুটের ব্যাখ্যা, আর ক্লাস্টার মুছে আবার তৈরির ধাপ আছে। মূল্যায়নের ইঙ্গিত: নোডের ভূমিকা (control-plane/worker) ঠিকমতো ব্যাখ্যা করা হয়েছে কি না, আর কনটেক্সট যাচাইয়ের ধাপ আছে কি না দেখা হবে।
2. **অ্যাসাইনমেন্ট ২ — Pod জীবনচক্রের প্রতিবেদন।** উদ্দেশ্য: একটি Pod-এর Pending থেকে Running, তারপর মুছে ফেলা পর্যন্ত প্রতিটি অবস্থা পর্যবেক্ষণ করে লেখা। ডেলিভারেবল: প্রতিটি অবস্থার স্ক্রিনশট বা টার্মিনাল আউটপুটসহ একটি প্রতিবেদন, যাতে `describe`-এর Events-এর ব্যাখ্যা আছে। মূল্যায়নের ইঙ্গিত: প্রতিটি অবস্থার কারণ ব্যাখ্যা করা হয়েছে কি না দেখা হবে।
3. **অ্যাসাইনমেন্ট ৩ — প্রোব ডিজাইন।** উদ্দেশ্য: একটি ধীরে চালু হওয়া অ্যাপ্লিকেশনের জন্য liveness, readiness ও startup প্রোবের মান নির্ধারণ করা। ডেলিভারেবল: প্রোবসহ Pod YAML, মানগুলোর যুক্তি লেখা একটি নোট, আর প্রোব ব্যর্থতার পরীক্ষার ফল। মূল্যায়নের ইঙ্গিত: startup প্রোবের প্রয়োজন বোঝা হয়েছে কি না, আর মানগুলো যুক্তিসঙ্গত কি না দেখা হবে।
4. **অ্যাসাইনমেন্ট ৪ — শূন্য-ডাউনটাইম আপডেট।** উদ্দেশ্য: একটি Deployment-এর ইমেজ আপডেট করে প্রমাণ করা যে আপডেটের সময় সার্ভিস বন্ধ হয়নি। ডেলিভারেবল: আপডেটের আগে, মাঝে ও পরে curl-এর ফল, rollout history, আর maxUnavailable/maxSurge-এর ব্যাখ্যা। মূল্যায়নের ইঙ্গিত: আপডেটের সময় কোনো অনুরোধ ব্যর্থ হয়েছে কি না দেখা হবে।
5. **অ্যাসাইনমেন্ট ৫ — Namespace পরিকল্পনা।** উদ্দেশ্য: একটি ছোট দলের জন্য dev, staging ও prod Namespace-এর পরিকল্পনা করা, যাতে নাম, Label ও ResourceQuota-র নিয়ম আছে। ডেলিভারেবল: একটি পরিকল্পনা নথি ও Namespace তৈরির YAML। মূল্যায়নের ইঙ্গিত: Namespace আলাদা করার যুক্তি ও Label-এর সামঞ্জস্য দেখা হবে।
6. **অ্যাসাইনমেন্ট ৬ — ConfigMap ও Secret ব্যবস্থাপনা।** উদ্দেশ্য: একটি অ্যাপ্লিকেশনের সব কনফিগারেশন ConfigMap-এ ও সব গোপন তথ্য Secret-এ সরিয়ে নেওয়া। ডেলিভারেবল: ConfigMap, Secret ও Pod YAML, আর কী কী ইমেজ থেকে সরানো হয়েছে তার তালিকা। মূল্যায়নের ইঙ্গিত: কোনো আসল সিক্রেট Git-এ নেই কি না, আর পরিবেশ চলক ও ভলিউম — দুটি পদ্ধতিই ব্যবহার করা হয়েছে কি না দেখা হবে।
7. **অ্যাসাইনমেন্ট ৭ — রিসোর্স বাজেট।** উদ্দেশ্য: `kubectl top` দিয়ে একটি অ্যাপ্লিকেশনের ব্যবহার মেপে তার request ও limit নির্ধারণ করা। ডেলিভারেবল: মাপের তথ্যসহ একটি টেবিল, request/limit-সহ YAML, আর সীমা ছাড়িয়ে OOMKilled-এর পরীক্ষা। মূল্যায়নের ইঙ্গিত: মানগুলো মাপের ভিত্তিতে নির্ধারিত হয়েছে কি না, অনুমানে নয়, তা দেখা হবে।
8. **অ্যাসাইনমেন্ট ৮ — ব্যাকআপ Job ও CronJob।** উদ্দেশ্য: প্রতিদিন রাতে চলা একটি ব্যাকআপ CronJob তৈরি করা। ডেলিভারেবল: Job ও CronJob YAML, সময়সূচির ব্যাখ্যা, concurrencyPolicy-র যুক্তি, আর একটি হাতে চালানো Job-এর লগ। মূল্যায়নের ইঙ্গিত: সময়সূচি সঠিক কি না ও ব্যর্থ Job-এর পুনরায় চেষ্টার নীতি আছে কি না দেখা হবে।

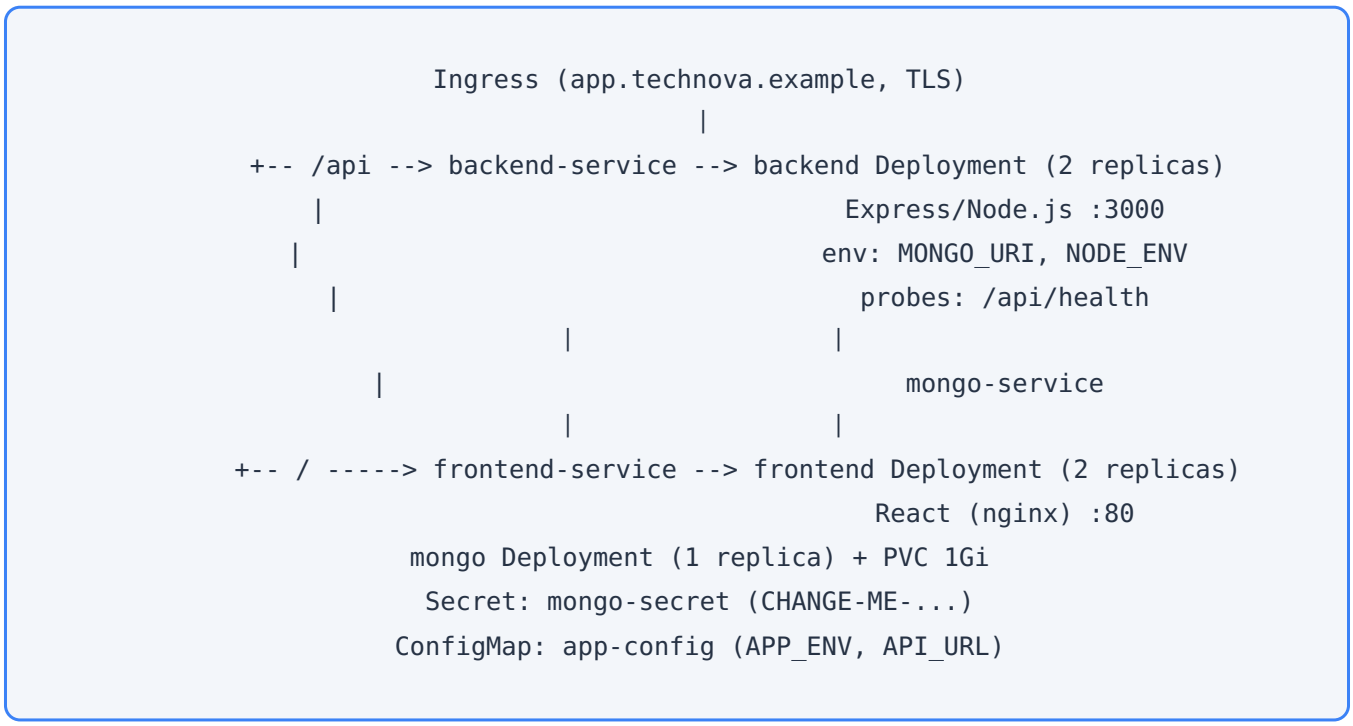
9. **অ্যাসাইনমেন্ট ৯ — Service ও DNS মানচিত্র।** উদ্দেশ্য: তিনটি সার্ভিসের একটি মানচিত্র আঁকা, যাতে প্রতিটি সার্ভিসের DNS নাম, পোর্ট ও Selector দেখানো হয়েছে। ডেলিভারেবল: একটি ডায়াগ্রাম ও Service YAML ফাইলগুলো, আর ভিন্ন Namespace থেকে DNS পরীক্ষার ফল। মূল্যায়নের ইঙ্গিত: FQDN-এর ব্যবহার সঠিক কি না দেখা হবে।
10. **অ্যাসাইনমেন্ট ১০ — Ingress ও TLS।** উদ্দেশ্য: দুটি সার্ভিসের জন্য Ingress তৈরি করা, যাতে একটি হোস্টে দুটি পথ আছে ও TLS চালু আছে। ডেলিভারেবল: Ingress YAML, স্ব-স্বাক্ষরিত শংসাপত্র তৈরির ধাপ, আর HTTP ও HTTPS পরীক্ষার ফল। মূল্যায়নের ইঙ্গিত: TLS Secret-এর নাম ও হোস্ট মিলছে কি না, আর পথের নিয়ম ঠিক আছে কি না দেখা হবে।
11. **অ্যাসাইনমেন্ট ১১ — স্থায়ী স্টোরেজ পরীক্ষা।** উদ্দেশ্য: PVC-সহ একটি অ্যাপ্লিকেশন তৈরি করে প্রমাণ করা যে Pod মুছে গেলেও ডেটা থাকে। ডেলিভারেবল: PV, PVC ও Pod YAML, Pod মুছে আবার তৈরি করার পর ডেটা যাচাইয়ের ফল। মূল্যায়নের ইঙ্গিত: reclaimPolicy-র ব্যাখ্যা আছে কি না দেখা হবে।
12. **অ্যাসাইনমেন্ট ১২ — RBAC ন্যূনতম অনুমতি।** উদ্দেশ্য: একটি ServiceAccount-কে কেবল একটি Namespace-এর Pod পড়ার অনুমতি দেওয়া। ডেলিভারেবল: ServiceAccount, Role ও RoleBinding YAML, `kubectl auth can-i` পরীক্ষার ফল (পড়া যায়, মোছা যায় না)। মূল্যায়নের ইঙ্গিত: অনুমতি প্রয়োজনের চেয়ে বেশি নয় কি না দেখা হবে।
13. **অ্যাসাইনমেন্ট ১৩ — নিরাপত্তা কঠোরকরণ।** উদ্দেশ্য: একটি Pod-কে non-root, read-only ফাইল সিস্টেম ও capability বাদ দিয়ে চালানো। ডেলিভারেবল: SecurityContext-সহ Pod YAML, `id` কমান্ডের আউটপুট, আর লেখার চেষ্টায় ব্যর্থতার প্রমাণ। মূল্যায়নের ইঙ্গিত: প্রতিটি নিরাপত্তা সেটিংয়ের কারণ ব্যাখ্যা করা হয়েছে কি না দেখা হবে।
14. **অ্যাসাইনমেন্ট ১৪ — Helm বনাম Kustomize তুলনা।** উদ্দেশ্য: একই অ্যাপ্লিকেশন Helm চার্ট ও Kustomize ওভারলে — দুইভাবে ডেপ্লয় করে তুলনা করা। ডেলিভারেবল: চার্ট, বেস ও ওভারলে ফাইল, আর একটি তুলনা টেবিল (কখন কোনটি ভালো)। মূল্যায়নের ইঙ্গিত: দুটি টুলের পার্থক্য ও ব্যবহারের ক্ষেত্র বোঝা হয়েছে কি না দেখা হবে।
15. **অ্যাসাইনমেন্ট ১৫ — ঘটনা প্রতিবেদন (Incident Report)।** উদ্দেশ্য: ইচ্ছা করে একটি অ্যাপ্লিকেশন ভাঙা (ভুল ইমেজ, ভুল ConfigMap, সীমা ছাড়ানো) ও তা ঠিক করে একটি ঘটনা প্রতিবেদন লেখা। ডেলিভারেবল: ঘটনার সময়রেখা, লক্ষণ, নির্ণয়ের ধাপ, মূল কারণ, সমাধান ও প্রতিরোধের উপায় — একটি পূর্ণ প্রতিবেদন। মূল্যায়নের ইঙ্গিত: মূল কারণ সঠিকভাবে চিহ্নিত করা হয়েছে কি না, আর প্রতিবেদনটি অন্য কেউ পড়ে বুঝতে পারে কি না দেখা হবে।

> **WARNING:** অ্যাসাইনমেন্টের ফলাফল Git-এ সংরক্ষণ করার সময় Secret-এর YAML ফাইল কমিট করবেন না, অথবা তাতে কেবল `CHANGE-ME-...` প্লেসহোল্ডার রাখুন। আসল পাসওয়ার্ড, টোকেন বা শংসাপত্র কখনো রিপোজিটরিতে তুলবেন না।

প্রজেক্ট ০১ — Kubernetes MERN অ্যাপ্লিকেশন ডেপ্লয়মেন্ট

এই প্রজেক্টে আপনি ল্যাব ২৪-এর MERN অ্যাপ্লিকেশনটিকে একটি সম্পূর্ণ, ডকুমেন্টেড প্রকল্পে রূপান্তর করবেন। TechNova-র উদাহরণ অ্যাপ্লিকেশন: React frontend, Express/Node.js backend API, MongoDB ডাটাবেস — সবই Kubernetes-এ চলবে। এই প্রজেক্টের লক্ষ্য কেবল অ্যাপ্লিকেশন চালানো নয় — একটি পেশাদার ডেপ্লয়মেন্ট প্যাকেজ তৈরি করা, যা অন্য কেউ পড়ে বুঝতে ও চালাতে পারে।

আর্কিটেকচার



প্রয়োজনীয় উপাদান

frontend Deployment, backend API Deployment, frontend ও backend Service, Ingress, ConfigMap, Secret, MongoDB-র PersistentVolumeClaim ও Deployment, প্রতিটি কন্টেইনারে health probe (liveness ও readiness), প্রতিটি Deployment-এ resource request ও limit — এই সবকিছুই এই প্রজেক্টে থাকতে হবে। কোনো উপাদান বাদ গেলে প্রজেক্ট অসম্পূর্ণ ধরা হবে।

কনফিগারেশন ফাইল — backend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: technova
  labels:
    app: backend
spec:
  replicas: 2
  selector:
```

```
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: api
          image: technova/api:1.0.0
          ports:
            - containerPort: 3000
          env:
            - name: NODE_ENV
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: APP_ENV
            - name: MONGO_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: MONGO_PASSWORD
          livenessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /api/health
              port: 3000
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
```

কনফিগারেশন ফাইল — frontend Deployment ও ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: technova
data:
  APP_ENV: production
  API_URL: http://backend-service
---
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: frontend-deployment
  namespace: technova
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: web
          image: technova/frontend:1.0.0
          ports:
            - containerPort: 80
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "250m"
```

ডেপ্লয়মেন্ট নির্দেশনা

1. `kubectl create namespace technova` চালান।
2. Secret ও ConfigMap তৈরি করুন (`kubectl apply -f config.yaml`)।
3. MongoDB-র PVC ও Deployment তৈরি করুন; Pod `Running` হওয়া পর্যন্ত অপেক্ষা করুন।
4. backend Deployment ও Service তৈরি করুন; `/api/health` পরীক্ষা করুন।
5. frontend Deployment ও Service তৈরি করুন; frontend থেকে backend-এ অনুরোধ যাচ্ছে কি না দেখুন।
6. Ingress তৈরি করুন ও `app.technova.example` দিয়ে পরীক্ষা করুন।
7. backend-কে ৩টি Replica-তে স্কেল করে rolling update পরীক্ষা করুন।
8. সব যাচাইয়ের ফল README-তে লিখুন ও `kubectl delete namespace technova` দিয়ে পরিষ্কার করুন।

REAL WORLD: বাস্তব প্রকল্পে frontend ও backend-এর ইমেজ CI/CD পাইপলাইন থেকে আসে, প্রতিটি কমিটে নতুন ট্যাগ তৈরি হয়, আর ডেপ্লয়মেন্ট Git-এর মাধ্যমে হয় (GitOps)। এই প্রজেক্টে আপনি সেই পাইপলাইনের শেষ ধাপটি — ক্লাস্টারে ডেপ্লয়মেন্ট — হাতে করছেন, যাতে পাইপলাইন স্বয়ংক্রিয় হলে তার ভেতরে কী ঘটে তা বুঝতে পারেন।

প্রজেক্ট ০২ — নিরাপদ মাল্টি-এনভায়রনমেন্ট ক্লাস্টার

এই প্রজেক্টে আপনি একটি ক্লাস্টারকে তিনটি পরিবেশে ভাগ করবেন — dev, staging ও prod — যাতে প্রতিটি পরিবেশ আলাদা Namespace-এ থাকে, আলাদা অনুমতি, আলাদা নেটওয়ার্ক নীতি ও আলাদা সম্পদ সীমা পায়। লক্ষ্য হলো একটি পরিবেশের ভুল যাতে অন্য পরিবেশকে ক্ষতিগ্রস্ত না করে। এটি প্রোডাকশন ক্লাস্টারের সবচেয়ে সাধারণ গঠনগুলোর একটি।

আর্কিটেকচার

```
                    ক্লাস্টার (kind / Minikube)
                       +-- namespace: dev
    |     Deployment (1 replica), Service, ConfigMap (APP_ENV=dev)
       |     RoleBinding: dev-team (Pod পড়া ও তৈরি, Secret পড়া নয়)
            |     ResourceQuota: CPU 2, মেমরি 4Gi
                     +-- namespace: staging
     |     Deployment (2 replicas), Ingress, NetworkPolicy
            |     ResourceQuota: CPU 4, মেমরি 8Gi
                       +-- namespace: prod
        Deployment (3 replicas), Ingress + TLS, NetworkPolicy
     RoleBinding: prod-team (কেবল পড়া; ডেপ্লয় CI/CD ServiceAccount দিয়ে)
                   ResourceQuota: CPU 8, মেমরি 16Gi
                 LimitRange: প্রতিটি Pod-এর ডিফল্ট request/limit
```

RBAC ও ServiceAccount

প্রতিটি Namespace-এ আলাদা ServiceAccount তৈরি করুন। dev-এ ডেভেলপারদের Role দিন (Pod, Deployment তৈরি ও পড়া; Secret পড়া নয়)। prod-এ মানুষের সরাসরি ডেপ্লয় নিষিদ্ধ করুন — কেবল CI/CD-এর ServiceAccount ডেপ্লয় করতে পারবে। `kubectl auth can-i` দিয়ে প্রতিটি অনুমতি পরীক্ষা করুন। ClusterRole ব্যবহার করবেন কেবল তখনই, যখন অনুমতি পুরো ক্লাস্টারে দরকার হয়; Namespace-এর অনুমতির জন্য Role-ই যথেষ্ট।

NetworkPolicy ও ResourceQuota

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: prod-quota
  namespace: prod
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    pods: "20"
    services: "10"
```

```
---
apiVersion: v1
kind: LimitRange
metadata:
  name: prod-limits
  namespace: prod
spec:
  limits:
    - default:
        memory: 512Mi
        cpu: "500m"
      defaultRequest:
        memory: 256Mi
        cpu: "250m"
      type: Container
```

NetworkPolicy দিয়ে prod Namespace-এ কেবল Ingress Controller থেকে ট্রাফিক আসতে দিন, আর ডাটাবেস Pod-এ কেবল backend থেকে সংযোগ দিতে দিন (ল্যাব ১৭)। dev Namespace-এ নীতি শিথিল রাখুন, prod-এ কঠোর রাখুন — পরিবেশের ঝুঁকি অনুযায়ী নীতি বদলায়।

Helm ও Kustomize

একই অ্যাপ্লিকেশন Helm চার্ট হিসেবে প্যাকেজ করুন, যাতে values-dev.yaml, values-staging.yaml ও values-prod.yaml — তিনটি মানের ফাইল আছে। অথবা Kustomize বেস ও তিনটি ওভারলে ব্যবহার করুন। দুটি পদ্ধতির যেকোনো একটি বেছে নিন, তবে তিনটি পরিবেশে একই বেস থেকে ডেপ্লয় করতে হবে — প্রতিটি পরিবেশের জন্য আলাদা YAML হাতে লেখা যাবে না। `helm template` বা `kubectl kustomize` দিয়ে প্রতিটি পরিবেশের চূড়ান্ত YAML দেখিয়ে যাচাই করুন।

পরিবেশভিত্তিক কনফিগারেশন

ConfigMap-এ পরিবেশের নাম, API-র ঠিকানা ও ফিচার ফ্ল্যাগ রাখুন; Secret-এ প্রতিটি পরিবেশের আলাদা পাসওয়ার্ড রাখুন (সবগুলো CHANGE-ME-... প্লেসহোল্ডার)। dev-এর Secret prod-এ ব্যবহার করা যাবে না — প্রতিটি Namespace-এ আলাদা Secret তৈরি করুন।

আইসোলেশনের সীমা ও সীমাবদ্ধতা

Namespace আইসোলেশন সম্পূর্ণ নয় — এটি নাম, অনুমতি ও সম্পদের সীমা আলাদা করে, কিন্তু নোড, নেটওয়ার্ক ও কার্নেল ভাগ করে নেয়। একটি Namespace-এর Pod অন্য Namespace-এর Pod-এর নেটওয়ার্কে পৌঁছাতে পারে, যদি না NetworkPolicy বাধা দেয়। একটি Namespace-এর Pod নোডের সম্পদ খেয়ে ফেলতে পারে, যদি না ResourceQuota থাকে। সত্যিকারের শক্ত আইসোলেশন দরকার হলে আলাদা ক্লাস্টার বা আলাদা নোড পুল দরকার হয়। এই সীমাগুলো আপনার প্রজেক্ট ডকুমেন্টেশনে স্পষ্টভাবে লিখুন — সীমা না জানা নিজেই একটি ঝুঁকি।

> **WARNING:** এই প্রজেক্টে prod Namespace-এর নামে পরীক্ষা করলেও এটি আসল প্রোডাকশন নয় — লোকাল ক্লাস্টারের একটি Namespace মাত্র। আসল প্রোডাকশনে Namespace ছাড়াও ব্যাকআপ, মনিটরিং, সতর্কবার্তা ও জরুরি প্রতিক্রিয়া পরিকল্পনা দরকার হয়।

প্রজেক্ট ০৩ — TechNova প্রোডাকশন Kubernetes প্ল্যাটফর্ম

এই প্রজেক্টে আপনি TechNova-র জন্য একটি সম্পূর্ণ প্রোডাকশন-ধাঁচের Kubernetes প্ল্যাটফর্মের নকশা তৈরি করবেন — কেবল অ্যাপ্লিকেশন ডেপ্লয় নয়, পুরো প্ল্যাটফর্ম: ক্লাস্টার, নেটওয়ার্ক, স্টোরেজ, নিরাপত্তা, অটোস্কেলিং, মনিটরিং, ব্যাকআপ, দুর্যোগ পুনরুদ্ধার, আপগ্রেড ও ঘটনা প্রতিক্রিয়া। এই প্রজেক্টের ফলাফল একটি নকশা নথি ও একটি কাজ করা লোকাল প্রমাণ (proof-of-concept) — দুটিই।

আর্কিটেকচার

```
          TechNova Production Platform (203.0.113.x, technova.example)
        +------------------ control-plane (3 নোড, HA) ------------------+
          |  API সার্ভার x3 | etcd x3 | scheduler | controller-manager   |
            +------------------ worker নোড পুল -----------------------------+
         |  app নোড পুল: frontend/backend Deployment (HPA)                |
         |  data নোড পুল: MongoDB StatefulSet + PVC (StorageClass)        |
        |  infra: Ingress Controller, CoreDNS, metrics-server             |
                    +-------------------------------------------------------------+
          | নেটওয়ার্ক: NetworkPolicy, Ingress + TLS, Service (ClusterIP)|
          | নিরাপত্তা: RBAC, Pod Security Standards, non-root, Secret     |
                | পর্যবেক্ষণ: Prometheus, Grafana, লগ সংগ্রহ, সতর্কবার্তা    |
                  | ব্যাকআপ: etcd স্ন্যাপশট, Velero, PV স্ন্যাপশট        |
                    +-------------------------------------------------------------+
```

উচ্চ-উপলব্ধতা (HA) ও ওয়ার্কলোড আর্কিটেকচার

control-plane-এর তিনটি নোড থাকবে, যাতে একটি নোড নষ্ট হলেও ক্লাস্টার চলে। etcd-র তিনটি সদস্য থাকবে। worker নোডগুলো পুলে ভাগ করা হবে: অ্যাপ্লিকেশন পুল ও ডাটা পুল — ডাটাবেসের Pod কেবল ডাটা পুলে চলবে (nodeSelector ও taint/toleration দিয়ে)। frontend ও backend Deployment-এ কমপক্ষে দুটি Replica থাকবে, আর PodDisruptionBudget দিয়ে নিশ্চিত করা হবে যে রক্ষণাবেক্ষণের সময়ও অন্তত একটি Replica চলতে থাকে।

নেটওয়ার্কিং ও স্টোরেজ

Ingress দিয়ে বাইরের ট্রাফিক আসবে, TLS দিয়ে HTTPS শেষ হবে। অভ্যন্তরীণ যোগাযোগ ClusterIP Service ও DNS দিয়ে হবে। NetworkPolicy দিয়ে Namespace ও স্তরের মধ্যে ট্রাফিক সীমিত করা হবে। স্টোরেজের জন্য StorageClass ব্যবহার করা হবে, যাতে PVC তৈরি করলেই স্বয়ংক্রিয়ভাবে PV তৈরি হয়। ডাটাবেসের জন্য StatefulSet ব্যবহার করা হবে, যাতে প্রতিটি Pod-এর নিজস্ব স্থায়ী ভলিউম ও স্থিতিশীল নাম থাকে।

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: technova-standard
```

```
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain
allowVolumeExpansion: true
```

উপরের StorageClass-টি লোকাল প্রমাণের জন্য একটি উদাহরণ — প্রোডাকশনে provisioner হবে ক্লাস্টারের স্টোরেজ ব্যবস্থার আসল provisioner। `reclaimPolicy: Retain` মানে PVC মুছে গেলেও ডেটা থেকে যাবে; `allowVolumeExpansion: true` মানে PVC-র আকার পরে বাড়ানো যায়।

নিরাপত্তা

RBAC দিয়ে ন্যূনতম অনুমতি, Pod Security Standards-এর restricted প্রোফাইল, non-root কন্টেইনার, read-only ফাইল সিস্টেম, Secret-এর এনক্রিপশন, আর ইমেজ স্ক্যানিং — এই সবগুলো নথিতে থাকতে হবে। কোনো Pod-ই ডিফল্ট ServiceAccount-এর বেশি অনুমতি নিয়ে চলবে না।

অটোস্কেলিং, মনিটরিং ও ব্যাকআপ

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
  namespace: technova
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

HorizontalPodAutoscaler CPU ব্যবহার ৭০% ছাড়ালে Replica বাড়াবে, কমলে কমাবে। মনিটরিংয়ের জন্য Prometheus মেট্রিক সংগ্রহ করবে, Grafana ড্যাশবোর্ড দেখাবে, আর সতর্কবার্তা (alert) যাবে দলের কাছে। ব্যাকআপের জন্য etcd স্ন্যাপশট প্রতিদিন নেওয়া হবে, Velero দিয়ে Namespace-এর ব্যাকআপ নেওয়া হবে, আর PV-র স্ন্যাপশট নেওয়া হবে। RTO ও RPO নথিতে লিখতে হবে।

দুর্যোগ পুনরুদ্ধার, ক্লাস্টার আপগ্রেড ও ঘটনা প্রতিক্রিয়া

দুর্যোগ পুনরুদ্ধার পরিকল্পনায় লিখুন: ক্লাস্টার পুরোপুরি নষ্ট হলে কীভাবে নতুন ক্লাস্টারে ব্যাকআপ থেকে ফিরিয়ে আনা হবে, কত সময় লাগবে, আর কে কী করবে। ক্লাস্টার আপগ্রেডের পরিকল্পনায় লিখুন: প্রথমে control-plane, তারপর worker নোড — একে একে, PodDisruptionBudget মেনে। ঘটনা প্রতিক্রিয়ার জন্য রানবুক লিখুন: Pod CrashLoopBackOff হলে কী করবেন, Ingress-এ ৫০২ ত্রুটি এলে কী করবেন, ডাটাবেসের ডিস্ক পূর্ণ হলে কী করবেন — প্রতিটির জন্য ধাপ, দায়িত্ব ও যোগাযোগের পদ্ধতি।

অপারেশন রানবুক

প্রতিটি রানবুকে থাকবে: লক্ষণ, নির্ণয়ের কমান্ড (`kubectl get pods`, `describe`, `logs`), সম্ভাব্য কারণ, সমাধানের ধাপ, আর কখন ঊর্ধ্বতন দলকে জানাতে হবে। রানবুক ছাড়া প্ল্যাটফর্ম অসম্পূর্ণ — কারণ রাত ৩টায় ঘটনা ঘটলে কেউ নকশা নথি পড়ে সিদ্ধান্ত নিতে পারে না, রানবুক পড়ে পারে।

ক্যাপস্টোন — TechNova প্রোডাকশন Kubernetes প্ল্যাটফর্ম

এই ক্যাপস্টোনে আপনি TechNova-র জন্য একটি প্রোডাকশন-ধাঁচের MERN পরিবেশ নিজে ডিজাইন ও ডেপ্লয় করবেন — বইয়ের সব জ্ঞান একত্র করে, কোনো ধাপে ধাপে নির্দেশনা ছাড়াই। আপনাকে একটি কাজ করা পরিবেশ দিতে হবে, আর তার সঙ্গে একটি পেশাদার ডেপ্লয়মেন্ট ডকুমেন্টেশন দিতে হবে, যা পড়ে অন্য একজন প্রকৌশলী একই পরিবেশ আবার তৈরি করতে পারেন। ক্যাপস্টোন শুরু করার আগে সমাধান নির্দেশিকা (`#capstone-solution`) পড়বেন না — প্রথমে নিজে চেষ্টা করুন।

পরিস্থিতি

TechNova একটি ছোট সফটওয়্যার কোম্পানি। তাদের MERN অ্যাপ্লিকেশন (React frontend, Express/Node.js backend API, MongoDB) এখন একটি সার্ভারে চলছে। তারা চায় অ্যাপ্লিকেশনটি Kubernetes-এ নিয়ে যেতে, যাতে স্কেল করা যায়, আপডেটে ডাউনটাইম না হয়, আর একটি Pod বা নোড নষ্ট হলেও সার্ভিস চলে। আপনার কাজ: লোকাল ক্লাস্টারে (kind/Minikube/k3d) একটি প্রোডাকশন-ধাঁচের পরিবেশ তৈরি করা, যা তাদের আসল প্রোডাকশন পরিবেশের একটি ছোট, কাজ করা মডেল হবে। কাল্পনিক প্রসঙ্গ: `technova.example`, `203.0.113.x`।

যা করতে হবে

1. **আর্কিটেকচার পরিকল্পনা:** একটি আর্কিটেকচার ডায়াগ্রাম আঁকুন ও নকশার সিদ্ধান্তগুলো লিখুন — কেন এই আর্কিটেকচার, কী বিকল্প ছিল, কেন তা বাদ দেওয়া হয়েছে।
2. **ক্লাস্টার নির্বাচন:** kind, Minikube বা k3d — যেকোনো একটি বেছে নিন ও পছন্দের কারণ লিখুন। ক্লাস্টার তৈরি করে তার নোড ও সংস্করণ ডকুমেন্ট করুন।
3. **Namespace ডিজাইন:** dev, staging ও prod — তিনটি Namespace তৈরি করুন, প্রতিটির উদ্দেশ্য, Label ও ResourceQuota লিখুন।
4. **অ্যাপ্লিকেশন ডেপ্লয়মেন্ট:** frontend, backend ও MongoDB ডেপ্লয় করুন — প্রতিটিতে Replica, probe, resource request/limit থাকতে হবে।
5. **নেটওয়ার্কিং:** Service, DNS ও Ingress (TLS-সহ) তৈরি করুন; `/` পথে frontend, `/api` পথে backend।
6. **স্থায়ী স্টোরেজ:** MongoDB-র জন্য PVC তৈরি করুন; Pod মুছে ডেটা টিকে থাকে তা প্রমাণ করুন।
7. **সিক্রেট ব্যবস্থাপনা:** সব পাসওয়ার্ড ও টোকেন Secret-এ রাখুন; কোনো আসল মান Git-এ বা ডকুমেন্টেশনে লিখবেন না — কেবল `CHANGE-ME-...` প্লেসহোল্ডার।
8. **নিরাপত্তা নিয়ন্ত্রণ:** RBAC, ServiceAccount, NetworkPolicy ও SecurityContext (non-root, read-only ফাইল সিস্টেম) প্রয়োগ করুন।
9. **সম্পদ ব্যবস্থাপনা:** প্রতিটি ওয়ার্কলোডে request/limit দিন, Namespace-এ ResourceQuota ও LimitRange দিন।
10. **অটোস্কেলিং:** backend-এর জন্য HorizontalPodAutoscaler তৈরি করুন (CPU ৭০%) ও চাপ দিয়ে পরীক্ষা করুন।
11. **মনিটরিং:** metrics-server ইনস্টল করে `kubectl top` দিয়ে ব্যবহার দেখান; Prometheus/Grafana-র পরিকল্পনা নথিতে লিখুন।
12. **ব্যর্থতা অনুকরণ (Failure Simulation):** ইচ্ছা করে একটি Pod মুছুন, একটি নোডের Pod সরান, একটি ভুল ইমেজ দিয়ে আপডেট করুন — প্রতিটিতে সার্ভিস কীভাবে টিকে থাকে তা দেখান।

13. **সমস্যা সমাধান ও পুনরুদ্ধার:** প্রতিটি ব্যর্থতার নির্ণয়, সমাধান ও পুনরুদ্ধারের সময় লিখুন; Namespace মুছে ব্যাকআপ থেকে ফিরিয়ে আনুন।
14. **ডেপ্লয়মেন্ট ডকুমেন্টেশন:** একটি পেশাদার README লিখুন — আর্কিটেকচার, ডেপ্লয়মেন্ট ধাপ, যাচাই, ব্যাকআপ, পুনরুদ্ধার ও রানবুক — সবকিছু।

গ্রহণযোগ্যতার মানদণ্ড (Acceptance Criteria)

- তিনটি Namespace-এ অ্যাপ্লিকেশন চলে; frontend, backend ও MongoDB — তিনটি Pod `Running` ও `Ready`।
- Ingress দিয়ে `app.technova.example`-এ HTTPS-এ অ্যাপ্লিকেশন পৌঁছানো যায়।
- একটি Pod মুছলে সার্ভিস বন্ধ হয় না; নতুন Pod স্বয়ংক্রিয়ভাবে তৈরি হয়।
- ভুল ইমেজের আপডেট rollback করা যায়; rollout history-তে revision দেখা যায়।
- MongoDB-র Pod মুছে আবার তৈরি করলে ডেটা থেকে যায়।
- কেবল অনুমোদিত ServiceAccount-ই Pod মুছতে পারে; অন্যরা `can-i`-তে `no` পায়।
- NetworkPolicy প্রয়োগের পর frontend সরাসরি ডাটাবেসে পৌঁছাতে পারে না।
- HPA চাপ বাড়লে Replica বাড়ায়।
- ব্যাকআপ থেকে Namespace পুনরুদ্ধার করা যায়।
- ডকুমেন্টেশন পড়ে অন্য কেউ পরিবেশটি আবার তৈরি করতে পারে।

গ্রেডিং রুব্রিক

মানদণ্ড	ওজন	চমৎকার (৯০-১০০%)	ভালো (৭০-৮৯%)	অসম্পূর্ণ (৫০%-এর কম)
আর্কিটেকচার পরিকল্পনা	১০%	স্পষ্ট ডায়াগ্রাম, সিদ্ধান্তের যুক্তি, বিকল্পের আলোচনা	ডায়াগ্রাম আছে, যুক্তি আংশিক	ডায়াগ্রাম নেই বা যুক্তি নেই
অ্যাপ্লিকেশন ডেপ্লয়মেন্ট	২০%	তিনটি স্তরই চলে, probe ও resource সীমাসহ	তিনটি স্তর চলে, probe বা সীমা আংশিক	একটি স্তর চলে না
নেটওয়ার্কিং ও TLS	১৫%	Ingress + TLS কাজ করে, পথের নিয়ম সঠিক	Ingress কাজ করে, TLS আংশিক	Ingress কাজ করে না
স্টোরেজ ও সিক্রেট	১০%	PVC-তে ডেটা টিকে থাকে, Secret-এ কোনো আসল মান নেই	PVC কাজ করে, Secret-এ ছোট ত্রুটি	ডেটা হারায় বা Secret Git-এ আছে
নিরাপত্তা (RBAC, NetworkPolicy, SecurityContext)	১৫%	তিনটি নিয়ন্ত্রণই কাজ করে ও পরীক্ষিত	দুটি নিয়ন্ত্রণ কাজ করে	একটি বা কোনোটিই কাজ করে না
অটোস্কেলিং ও মনিটরিং	১০%	HPA চাপে Replica বাড়ায়, মনিটরিং দেখানো হয়েছে	HPA তৈরি, পরীক্ষা আংশিক	HPA নেই
ব্যর্থতা অনুকরণ ও পুনরুদ্ধার	১০%	তিনটি ব্যর্থতা পরীক্ষিত, পুনরুদ্ধার প্রমাণিত	দুটি ব্যর্থতা পরীক্ষিত	কোনো পরীক্ষা নেই

ডকুমেন্টেশন	১০%	অন্য কেউ পড়ে আবার তৈরি করতে পারে	ডকুমেন্টেশন আছে, কিছু ধাপ অস্পষ্ট	ডকুমেন্টেশন নেই বা অসম্পূর্ণ

পাস করতে মোট ৬০% ও প্রতিটি বিভাগে অন্তত ৫০% পেতে হবে। নিরাপত্তা বিভাগে আসল সিক্রেট পাওয়া গেলে পুরো ক্যাপস্টোন অকৃতকার্য ধরা হবে — এটি একটি কঠোর নিয়ম।

> **WARNING:** ক্যাপস্টোনের সব কাজ লোকাল ক্লাস্টারে করুন। কোনো ক্লাউড ধাপে খরচ হতে পারে এমন ক্ষেত্রে তা আগে থেকে চিহ্নিত করুন ও এড়িয়ে চলুন। প্রোডাকশন ক্লাস্টারে ক্যাপস্টোনের পরীক্ষা করবেন না।

ক্যাপস্টোন সমাধান নির্দেশিকা — শিক্ষকের রূপরেখা

> **WARNING:** এই নির্দেশিকাটি পড়ার আগে ক্যাপস্টোনটি নিজে চেষ্টা করুন। প্রথমে নিজে আর্কিটেকচার আঁকুন, নিজে ডেপ্লয় করুন, নিজে ভাঙুন ও ঠিক করুন। এই নির্দেশিকাটি কেবল আপনার কাজ যাচাই করার জন্য — এটি কপি করে জমা দিলে শেখা হবে না।

এই নির্দেশিকাটি একটি রূপরেখা সমাধান — সম্পূর্ণ ধাপে ধাপে নির্দেশনা নয়। এতে আর্কিটেকচারের সিদ্ধান্ত, মূল ম্যানিফেস্টগুলোর সারসংক্ষেপ ও যাচাইয়ের চেকলিস্ট দেওয়া হয়েছে। আপনার সমাধান এর থেকে ভিন্ন হতে পারে — ভিন্ন হওয়া মানেই ভুল নয়, যদি তা গ্রহণযোগ্যতার মানদণ্ড পূরণ করে।

আর্কিটেকচারের সিদ্ধান্ত

- **ক্লাস্টার:** kind — কারণ এটি দ্রুত তৈরি হয়, ফেলে দেওয়া যায়, আর তিনটি নোডের ক্লাস্টার লোকাল মেশিনে চলে। Minikube বা k3d-ও গ্রহণযোগ্য, যদি পছন্দের কারণ লেখা থাকে।
- **Namespace:** dev, staging, prod — প্রতিটিতে আলাদা ResourceQuota; prod-এ সবচেয়ে কঠোর নীতি। একটি Namespace-এর ভুল যাতে অন্যটিকে ক্ষতিগ্রস্ত না করে, সেটিই মূল যুক্তি।
- **ওয়ার্কলোড:** frontend ও backend Deployment (২টি Replica), MongoDB Deployment + PVC (লোকাল মডেলে; প্রোডাকশনে StatefulSet)। Replica ২টি রাখার কারণ: একটি Pod নষ্ট হলেও সার্ভিস চলে।
- **নেটওয়ার্কিং:** ClusterIP Service + NGINX Ingress Controller + TLS (স্ব-স্বাক্ষরিত শংসাপত্র, ল্যাবের জন্য)। `/api` পথে backend, `/` পথে frontend।
- **নিরাপত্তা:** প্রতিটি Namespace-এ আলাদা ServiceAccount; prod-এ ডেপ্লয় কেবল CI/CD ServiceAccount দিয়ে; NetworkPolicy দিয়ে frontend থেকে ডাটাবেসে সরাসরি সংযোগ বন্ধ; সব কন্টেইনার non-root, read-only ফাইল সিস্টেম।
- **অটোস্কেলিং:** backend-এ HPA (CPU ৭০%, min ২, max ১০)। metrics-server দিয়ে মেট্রিক।
- **ব্যাকআপ:** YAML এক্সপোর্ট + etcd স্ন্যাপশটের পরিকল্পনা; PV ডেটার জন্য `kubectl cp` দিয়ে ফাইল ব্যাকআপ (লোকাল মডেলে)।

মূল ম্যানিফেস্টগুলোর সারসংক্ষেপ

প্রয়োজনীয় ম্যানিফেস্ট: Namespace ও ResourceQuota (৩টি), ConfigMap (প্রতিটি পরিবেশে আলাদা), Secret (প্রতিটি পরিবেশে আলাদা, CHANGE-ME-... মান), MongoDB Deployment + Service + PVC, backend Deployment + Service (probe, resource, env — Secret ও ConfigMap থেকে), frontend Deployment + Service, Ingress (TLS-সহ), NetworkPolicy (ডাটাবেস ও frontend-এর জন্য), Role/RoleBinding (প্রতিটি Namespace-এ), HPA (backend)। প্রতিটি ম্যানিফেস্টে `apiVersion` বর্তমান স্থিতিশীল সংস্করণে হতে হবে: `v1`, `apps/v1`, `networking.k8s.io/v1`, `rbac.authorization.k8s.io/v1`, `autoscaling/v2`।

যাচাইয়ের চেকলিস্ট

- `kubectl get pods -n prod` — সব Pod `Running` ও `1/1`।

- `kubectl get ingress -n prod` — ADDRESS দেখা যায়; `curl -k https://app.technova.example` সফল হয়।
- `kubectl get hpa -n prod` — TARGETS-এ CPU ব্যবহার দেখা যায়।
- `kubectl auth can-i delete pods --as=system:serviceaccount:prod:app-reader -n prod` — উত্তর `no`।
- একটি Pod মুছে `kubectl get pods -n prod -w` — নতুন Pod তৈরি হয়, সার্ভিস বন্ধ হয় না।
- MongoDB Pod মুছে আবার তৈরি করে ডেটা যাচাই — ডেটা থেকে যায়।
- Namespace মুছে ব্যাকআপ YAML থেকে পুনরুদ্ধার — সব বস্তু ফিরে আসে।
- ডকুমেন্টেশনে আর্কিটেকচার, ডেপ্লয়মেন্ট ধাপ, ব্যাকআপ, পুনরুদ্ধার ও রানবুক আছে।

এই চেকলিস্টের প্রতিটি আইটেমে টিক দিতে পারলে আপনার সমাধান গ্রহণযোগ্য। কোনো আইটেমে ব্যর্থ হলে সংশ্লিষ্ট ল্যাবে ফিরে যান — ল্যাব ৫ (Deployment), ল্যাব ১১ (Service), ল্যাব ১৩ (Ingress), ল্যাব ১৪ (Storage), ল্যাব ১৫ (RBAC), ল্যাব ১৭ (NetworkPolicy), ল্যাব ২০ (মনিটরিং) ও ল্যাব ২৪ (MERN)।

> **CONCEPT:** ক্যাপস্টোনের আসল মূল্যায়ন নম্বরে নয় — আপনি নিজে একটি সম্পূর্ণ পরিবেশ তৈরি করতে পারেন কি না, তা-ই আসল পরীক্ষা। এই নির্দেশিকার সঙ্গে আপনার কাজ মিলিয়ে দেখুন, পার্থক্যগুলো বুঝুন, আর ডকুমেন্টেশনে লিখুন কেন আপনি ভিন্ন সিদ্ধান্ত নিয়েছেন। সেই যুক্তিই একজন প্রকৌশলীকে আলাদা করে।